uc3m | Universidad Carlos III de Madrid

Grado en Ingeniería Informática
2025-2026

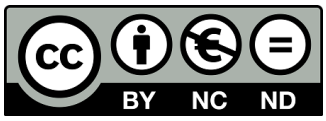
Trabajo Fin de Grado

“Gestión dinámica de recursos de cómputo y E/S: caso de estudio con EpiGraph y Expand Ad-Hoc”

Pablo Navarro Hurtado

Tutora
Paula Sánchez Checa

Colmenarejo, Madrid
Septiembre, 2026



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

Las aplicaciones científicas de altas prestaciones reservan sus recursos al comenzar y los mantienen fijos hasta terminar, con independencia de cómo evolucione su carga. En una simulación epidemiológica esa rigidez resulta especialmente costosa, porque las dos demandas de la aplicación no se comportan igual: el coste de cómputo sigue la curva de infección y varía a lo largo de una misma ejecución, mientras que el volumen de escritura permanece constante y proporcional al tamaño de la población. Dimensionar los recursos para el pico los desperdicia durante las fases de baja incidencia, y dimensionarlos por debajo alarga la simulación cuando la epidemia se acelera.

Este trabajo aborda ese problema integrando el simulador epidemiológico EpiGraph con el sistema de ficheros paralelo y maleable Expand Ad-Hoc y con la biblioteca de maleabilidad FlexMPI, sobre un clúster virtual de contenedores Docker desplegado tanto en un equipo personal como en el clúster de altas prestaciones Tucán de la Universidad Carlos III de Madrid. La entrada/salida de la aplicación se redirige hacia el sistema de ficheros paralelo sin modificar ni recompilar su código, mediante precarga de biblioteca, y tanto el número de procesos de cómputo como el de servidores de almacenamiento se reconfiguran mientras la simulación está en marcha.

La evaluación se organiza en cuatro casos de estudio, precedidos de una cuantificación de la variabilidad del entorno que establece qué diferencias pueden afirmarse y cuáles quedan por debajo del ruido de medida. Redirigir la escritura hacia el sistema de ficheros paralelo abarata la entrada/salida y, con ella, el tiempo total de la simulación, y la ventaja crece con el número de procesos empleados. En el despliegue del almacenamiento pesa más dónde se sitúan los servidores que cuántos son, ya que lo determinante es si la escritura ha de atravesar la red. La maleabilidad del número de procesos no acorta la ejecución frente a una configuración estática dimensionada al pico, pero reduce sensiblemente los recursos que esta ocupa mientras dura, y entre las decisiones que la gobiernan adelantar la ampliación pesa más que agrandarla. El rebalanceo dinámico de la carga entre un número fijo de procesos, en cambio, no compensa en toda condición: su signo depende del tamaño de la población simulada y de la frecuencia con que se evalúa el balance.

Palabras clave: maleabilidad; computación de altas prestaciones; sistemas de ficheros *ad hoc*; simulación epidemiológica; MPI; contenedores

AGRADECIMIENTOS

A mi tutora Paula por su paciencia, su perseverancia y por la ayuda que me ha prestado en este trabajo. Sin sus consejos y aportaciones la realización del mismo no habría sido posible.

A mis padres, que han hecho todo lo posible para que tanto mi carrera como este último proyecto salieran adelante.

A mis hermanos, por hacer de conejillos de indias de muchos trabajos a lo largo de mi trayectoria por la universidad y por su cariño y compañia.

A mis amigos (que no compañeros) de la Carlos III, por los que valía totalmente la pena hacer el largo trayecto hasta Colmenarejo.

ÍNDICE GENERAL

1. INTRODUCCIÓN. 1
1.1. Motivación 1
1.2. Objetivos del proyecto 2
1.3. Estructura de la memoria 3
2. ESTADO DE LA CUESTIÓN 4
2.1. Contextualización y Conceptos clave. 4
2.2. Tecnologías y modelos existentes. 5
2.2.1. Sistemas de ficheros ad hoc 5
2.2.2. Tecnologías para aplicar maleabilidad 7
2.2.3. Tecnologías de despliegue de contenedores 8
2.3. Otros trabajos similares 10
2.3.1. Maleabilidad de aplicaciones 10
2.3.2. Maleabilidad de sistemas de ficheros. 10
2.3.3. Maleabilidad combinada de aplicaciones y sistemas de ficheros. 11
3. ENTORNO DE DESARROLLO. 12
3.1. Contexto del proyecto 12
3.2. Tecnologías empleadas 13
3.2.1. EpiGraph 13
3.2.2. FlexMPI. 15
3.2.3. Expand Ad-Hoc (XPN) 16
3.2.4. Docker 18
3.3. Marco legislativo. 19
3.3.1. Licencias de software 19
3.3.2. Protección de datos 20
3.3.3. Reglamento europeo de inteligencia artificial 21
3.4. Entorno socioeconómico 21
3.4.1. Objetivos de Desarrollo Sostenible 22

4. PROPUESTA 23
4.1. Objetivos, Metodología y Fases de desarrollo 23
4.2. Planificación del proyecto 24
4.3. Análisis de requisitos 26
4.3.1. Requisitos funcionales 27
4.3.2. Requisitos no funcionales 29
4.4. Diseño del sistema 30
4.4.1. EpiGraph: la aplicación maleable de cómputo. 32
4.4.2. FlexMPI: control de maleabilidad y balanceo de carga 33
4.4.3. Expand Ad-Hoc (XPN) e intercepción de la entrada/salida 34
4.4.4. Ciclo de maleabilidad: *checkpoint* y reinicio 35
4.5. Presupuesto 36
4.5.1. Costes directos 36
4.5.2. Costes indirectos 37
4.5.3. Costes totales 38
5. EVALUACIÓN 39
5.1. Descripción de la plataforma de evaluación 39
5.1.1. Equipo local de desarrollo. 39
5.1.2. Clúster Tucán de la UC3M 40
5.1.3. Clúster virtual de contenedores 41
5.1.4. Pila de software y herramientas de observación 45
5.1.5. Métricas de la traza de ejecución 46
5.2. Integración y puesta en marcha del sistema 48
5.2.1. Ejecución local de EpiGraph y FlexMPI 49
5.2.2. Despliegue sobre el clúster virtual de contenedores. 49
5.2.3. Integración de Expand Ad-Hoc y de la maleabilidad 51
5.2.4. Puesta en marcha en el clúster Tucán. 53
5.3. Matriz de trazabilidad 56
5.4. Casos de estudio 58
5.4.1. CE1 — Caracterización de la E/S e interceptación transparente 59
5.4.2. CE2 — Configuraciones de despliegue del almacenamiento. 60

5.4.3. CE3 — Maleabilidad del número de procesos. 62
5.4.4. CE4 — Rebalanceo dinámico de carga. 64
5.5. Resultados 65
5.5.1. Variabilidad de las medidas 66
5.5.2. CE1 — Efecto de la interceptación transparente 69
5.5.3. CE2 — Configuraciones de despliegue del almacenamiento. 73
5.5.4. CE3 — Maleabilidad del número de procesos. 80
5.5.5. CE4 — Rebalanceo dinámico de carga. 96
5.6. Limitaciones del estudio. 101
5.6.1. Entorno de ejecución. 101
5.6.2. Integración con el sistema de ficheros 102
5.6.3. Disponibilidad de la plataforma. 103
5.6.4. Alcance e interpretación de las medidas 103
6. CONCLUSIONES Y TRABAJO FUTURO 105
6.1. Conclusiones 105
6.2. Grado de consecución de los objetivos. 106
6.3. Trabajo futuro 107
BIBLIOGRAFÍA 109

ÍNDICE DE FIGURAS

1.1 Asignación estática frente a asignación maleable 2

3.1 Grafo de contactos y su representación CSC 15
3.2 Vector de costes de EpiGraph a FlexMPI 17
3.3 Reparto de los bloques de un fichero entre los servidores de XPN 17
3.4 XPN como intermediario entre la aplicación y el sistema de ficheros . . . 18
3.5 Despliegue del entorno experimental sobre un nodo del clúster 19

4.1 Ciclo iterativo de desarrollo experimental 23
4.2 Diagrama de Gantt del proyecto 26
4.3 Arquitectura de componentes del sistema 31
4.4 Flujo de datos y de control durante una ejecución 31
4.5 Ciclo de maleabilidad por *checkpoint* y reinicio 35

5.1 Uso de CPU con cuatro y ocho procesos en el equipo local 43
5.2 Topologías de despliegue del clúster virtual de contenedores 44
5.3 Fragmento de traza de un día simulado 47
5.4 Desbalanceo de CPU entre contenedores en Docker 50
5.5 Efecto de corregir la intercepción de XPN sobre el tiempo de E/S en Tucán 54
5.6 Diferencia entre las dos ejecuciones más separadas 67
5.7 Dispersión entre despliegues de las configuraciones estáticas 69
5.8 Descomposición del tiempo de ejecución con NFS y con XPN 71
5.9 Descomposición del tiempo según el número de servidores de almacenamiento 75
5.10 Disposición de los servidores de almacenamiento entre dos nodos 78
5.11 Las tres expansiones y las tres estáticas que las acotan 83
5.12 Seguir la demanda frente a expandir una sola vez 87
5.13 El día en que se pide la expansión decide lo que devuelve 90
5.14 Tiempo y consumo de recursos de las dos configuraciones 93
5.15 Efecto de retirar servidores de almacenamiento a mitad de ejecución . . . 95

5.16 El signo del rebalanceo depende de las condiciones 100

ÍNDICE DE TABLAS

4.1 Estructura para la definición de requisitos. 27

4.2 Requisito funcional RF-01. 27

4.3 Requisito funcional RF-02. 27

4.4 Requisito funcional RF-03. 28

4.5 Requisito funcional RF-04. 28

4.6 Requisito funcional RF-05. 28

4.7 Requisito no funcional RNF-01. 29

4.8 Requisito no funcional RNF-02. 29

4.9 Requisito no funcional RNF-03. 29

4.10 Requisito no funcional RNF-04. 30

4.11 Costes de personal. 37

4.12 Costes de hardware. 37

4.13 Amortización del hardware. 37

4.14 Costes indirectos. 38

4.15 Costes totales. 38

5.1 Características del equipo local de desarrollo. 40

5.2 Características de un nodo de cómputo del clúster Tucán. 41

5.3 Componentes principales de la pila de software. 46

5.4 Campos de la línea `IOPerf`, que EpiGraph emite en cada volcado. 47

5.5 Campos de la línea `Iter`, que FlexMPI emite al cerrar cada intervalo de monitorización. 48

5.6 Principales incidencias de integración en el entorno local, con su causa y la solución aplicada. 52

5.7 Principales incidencias específicas de la puesta en marcha sobre el clúster Tucán, con su causa y la solución aplicada. 55

5.8 Matriz de trazabilidad entre los requisitos y los casos de estudio. 57

5.9 Dispersión entre despliegues de las configuraciones estáticas. 68

5.10 Tiempo total y tiempo de entrada/salida con NFS y con XPN. 70

5.11 Efecto de duplicar el número de procesos sobre cada sistema de ficheros. . 71

5.12 Barrido del número de servidores de almacenamiento sobre un único nodo, con la descomposición del tiempo total. 74

5.13 Disposición de los servidores de almacenamiento entre dos nodos 78

5.14 Tamaño de bloque de 512 frente a 1024 KiB, sobre las nueve configuraciones de un solo nodo. Una ejecución por celda. 79

5.15 Tiempo de ejecución con cada uno de los dos mecanismos de reconfiguración. 81

5.16 Expansiones de distinta magnitud, todas solicitadas el día 45, frente a las configuraciones estáticas que las acotan. Entre corchetes, el intervalo observado. 82

5.17 Efecto del día simulado en que se solicita la expansión, con destino fijo en cuarenta procesos. 85

5.18 Todas las expansiones de una sola etapa, cruzando el día en que se solicitan con el destino al que llegan. 88

5.19 Efecto de retirar la mitad de los servidores de almacenamiento el día en que la aplicación expande. 94

5.20 Efecto del balanceo según la plataforma. Población de 800 826 individuos, ocho procesos, evaluación cada 1440 iteraciones. Tiempos en segundos. 97

5.21 Efecto de la cadencia de evaluación sobre cada una de las dos políticas. Clúster Tucán, población de 800 826 individuos, ocho procesos. 98

5.22 Efecto del balanceo según el tamaño de la población. Clúster Tucán, evaluación cada 144 iteraciones. Tiempos en segundos. 98

1. INTRODUCCIÓN

1.1. Motivación

Hace ya 6 años, el mundo sufrió una pandemia que supuso un reto para la salud pública: la COVID-19, a raíz de este evento, los sistemas sanitarios se vieron obligados a adoptar diferentes medidas de prevención de infecciones, como puede ser la administración de vacunas. En este contexto, la capacidad de evitar o incluso predecir contagios se volvió una herramienta muy útil para mantener orden ante la emergencia.

Este trabajo radica en esta cuestión. EpiGraph, es un simulador epidemiológico de alto rendimiento desarrollado en la Universidad Carlos III de Madrid que ayuda a visualizar claramente la evolución de una pandemia concreta. Esto se consigue simulando la propagación del virus en una población, y proporcionando datos específicos como la edad y la profesión de sus ciudadanos, las interacciones sociales y el uso de mascarillas.

Sin embargo, la simulación de escenarios de este tipo a gran escala conlleva un coste computacional alto, al tratarse de una pandemia, puede producirse el surgimiento de una variante nueva del virus que aumente repentinamente el número de contagios, lo que se traduce en una sección crítica en la que el simulador necesita una gran capacidad de cómputo. También existen periodos en los que se ha inmunizado a gran parte de la población y los infectados son escasos. Esta aparente aleatoriedad en el número de infectados genera un flujo de trabajo en el que la carga computacional y de E/S varía constantemente.

En cuanto a la carga computacional, el problema radica en que, al hablar de supercomputación, las asignaciones de recursos y los sistemas de ficheros suelen ser estáticas. Puede parecer que la mejor solución sería asignar una gran cantidad de recursos en función del pico máximo de infectados en la pandemia, pero esto puede resultar terriblemente ineficiente, especialmente en aplicaciones con fases con distinta carga computacional, ya que se desperdician recursos cuando hay poca demanda de CPU.

A este reto computacional se suman los problemas de entrada/salida (E/S) en entornos HPC. Las aplicaciones de alto rendimiento (dinámicas de fluidos como OpenFOAM, previsiones meteorológicas como WRF o simulaciones a gran escala como GROMACS) cuentan con un elevado número de operaciones de E/S. Cuando decenas o cientos de nodos de cómputo realizan este tipo de transacciones a la vez, el tener todo el conjunto de datos en un solo lugar se convierte en un problema, para resolver este cuello de botella se desarrollaron los sistemas de ficheros distribuidos.

El objetivo de este proyecto viene motivado por este contexto. La solución propuesta en este trabajo consiste en lo siguiente: la implementación de la maleabilidad, un concepto en el que se ha trabajado extensamente dentro del ámbito de los sistemas distribuidos. No es suficiente con simular, la estructura interna de EpiGraph debe adaptarse al comporta-

miento real de la epidemia. Este trabajo propone la integración de EpiGraph con Expand Ad-Hoc, un sistema de ficheros distribuido capaz de reducir o expandir, en función de la necesidad, los servidores de almacenamiento. La motivación reside en dejar que el simulador solicite más potencia de E/S cuando el número de contagios aumente y libere recursos cuando disminuye, manejando el problema de la manera más eficiente posible. La Figura 1.1 resume esa idea: mientras la asignación estática mantiene reservados durante toda la ejecución los recursos que solo hacen falta en el pico —el área sombreada de la figura—, la asignación maleable los solicita cuando la curva sube y los devuelve cuando baja.

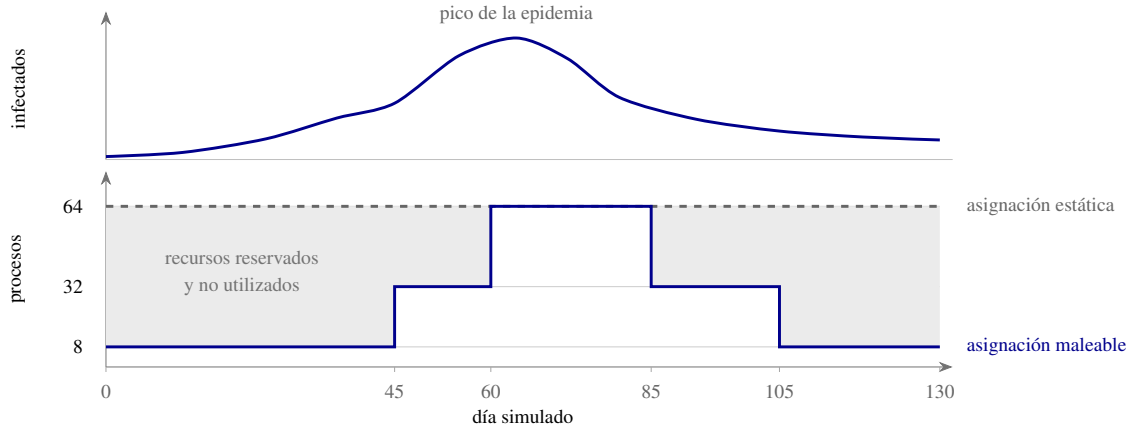


Fig. 1.1. Asignación estática frente a asignación maleable de procesos.

1.2. Objetivos del proyecto

Para definir los objetivos de este trabajo, primero se definirá un objetivo general, el cual se descompondrá en una serie de objetivos específicos. Por un lado, el objetivo principal del trabajo es:

- **Integrar el simulador epidemiológico EpiGraph con el sistema de ficheros distribuido y paralelo Expand Ad-Hoc.** Se busca dotar al proceso completo de maleabilidad, que la estructura de almacenamiento se adapte a la carga de trabajo, la cual varía en función de la fase de ejecución correspondiente. En definitiva, optimizar el uso de recursos y el tiempo de ejecución.

Más concretamente, los objetivos específicos del trabajo son los siguientes:

- **Adquirir las competencias necesarias:** Aprender sobre el funcionamiento de EpiGraph y Expand Ad-Hoc y cómo pueden complementarse para obtener el mejor resultado posible
- **Diseñar estrategias de adaptación dinámica:** Definir cómo aplicar la maleabilidad para tratar de mejorar el rendimiento, desarrollo de tácticas para definir el número de nodos de cómputo, de E/S, etc.

- **Representación de los resultados:** Ya sea a través de gráficos, figuras o similar. Mostrarlos de tal forma que resulte sencilla su correcta interpretación.
- **Evaluar los resultados:** Realizar un análisis de los distintos resultados de la integración y sacar una conclusión final.

1.3. Estructura de la memoria

En primer lugar, en «Estado de la cuestión» se introducen los conceptos sobre los que se apoya el trabajo —la maleabilidad, los sistemas de ficheros *ad hoc* y el despliegue en contenedores—, se revisan las tecnologías disponibles para cada uno de ellos y se recogen los trabajos previos que abordan la maleabilidad de aplicaciones, la de sistemas de ficheros y la combinación de ambas.

A continuación, en «Entorno de desarrollo» se describen los dos entornos sobre los que se ha trabajado —el equipo local y el clúster Tucán de la Universidad Carlos III de Madrid— y las tecnologías que se integran en ellos: EpiGraph, FlexMPI, Expand Ad-Hoc y Docker. El capítulo se cierra con el marco legislativo, donde se analizan las licencias del software empleado y la normativa aplicable en materia de protección de datos, y con el entorno socioeconómico del proyecto.

El capítulo «Propuesta» recoge el trabajo de ingeniería. Comienza por la metodología y las fases de desarrollo, sigue con la planificación del proyecto y el análisis de requisitos, y dedica su parte central al diseño del sistema: cómo se integran los tres componentes, qué papel cumple cada uno y cómo se materializa el ciclo de maleabilidad por *checkpoint* y reinicio. El capítulo termina con el presupuesto.

«Evaluación» es el capítulo más extenso y contiene la parte experimental. Describe primero la plataforma de medida y la puesta en marcha del sistema, con las incidencias que hubo que resolver para que los tres componentes funcionasen juntos, y establece después la trazabilidad entre los requisitos y los experimentos. Se plantean a continuación los cuatro casos de estudio —la caracterización de la entrada/salida, las configuraciones de despliegue del almacenamiento, la maleabilidad del número de procesos y el rebalanceo dinámico de carga—, se analizan sus resultados y se cierra con las limitaciones del estudio.

Finalmente, se incluye un capítulo de «Conclusiones y trabajo futuro» en el que se sintetiza lo obtenido, se revisa el grado de consecución de los objetivos y se apuntan las líneas por las que el trabajo puede continuarse.

2. ESTADO DE LA CUESTIÓN

2.1. Contextualización y Conceptos clave

En este apartado se van a explicar aquellos conceptos que son necesarios para comprender los temas tratados en el resto de la memoria y, por lo tanto, entender los objetivos del trabajo.

Las aplicaciones de computación de alto rendimiento (*High Performance Computing*, HPC) resuelven problemas de gran escala (desde alineamientos de secuencias genómicas a entrenamientos de modelos de deep learning) ejecutándose sobre clústeres de nodos con memoria distribuida. En este entorno, el estándar de comunicación entre procesos es MPI (*Message Passing Interface*): una especificación que define las reglas de envío de mensajes, sincronización y operaciones colectivas necesarias para coordinar decenas o cientos de procesos concurrentes distribuidos en distintos nodos del clúster [1].

Sin embargo, mientras el cómputo se distribuye entre múltiples nodos, el almacenamiento se mantenía tradicionalmente centralizado. Durante las primeras décadas del cómputo de altas prestaciones, todos los nodos escribían sus datos (puntos de control, resultados intermedios) sobre un único sistema de ficheros compartido, servido por un nodo o servidor de almacenamiento central que concentraba el conjunto de datos completo en un solo lugar. Este modelo, accedido habitualmente mediante NFS[1], fue el estándar hasta que el aumento de la complejidad de E/S —impulsado por el auge de la inteligencia artificial, la ciencia de datos y las simulaciones de alta resolución— obligó a revisar el sistema de almacenamiento [2]. Mantener todo el conjunto de datos en un único punto se convirtió en un cuello de botella: el rendimiento se degrada a medida que crece el número de nodos que compiten por el mismo recurso. Para hacer frente a esta limitación surgieron los sistemas de ficheros distribuidos, que reparten los datos entre múltiples servidores de almacenamiento, permitiendo ejecutar las operaciones de E/S de forma concurrente y reduciendo así el tiempo de ejecución de este tipo de aplicaciones.

A partir de esa base se crearon los sistemas de ficheros *ad hoc*: despliegan servidores de almacenamiento directamente sobre los nodos de cómputo y permiten escalar el número de nodos en función de la fase de la ejecución. Expand Ad-Hoc es un ejemplo de este enfoque: un sistema de ficheros distribuido y paralelo, desarrollado en la UC3M, que puede desplegarse de forma temporal sobre los propios nodos de cómputo de la aplicación [3], [4].

No obstante, tanto los sistemas de cómputo como los de almacenamiento en HPC han sido históricamente estáticos: los recursos se reservan al inicio del trabajo y permanecen

[1] *Network File System*, protocolo que permite a un cliente acceder a ficheros alojados en un servidor remoto a través de la red como si residieran en su propio sistema de ficheros local.

fijos hasta su finalización, con independencia de cómo evolucione la carga. Esta rigidez resulta ineficiente en aplicaciones cuya demanda varía a lo largo del tiempo (como una simulación epidemiológica en la que el coste computacional crece con el número de infectados y disminuye en periodos de baja incidencia). El concepto de maleabilidad responde a esta limitación: es la capacidad de ajustar dinámicamente los recursos de cómputo y almacenamiento en tiempo de ejecución para optimizar el rendimiento y la eficiencia [5]. Aplicada al cómputo, permite aumentar o reducir el número de procesos MPI activos sin detener la simulación; aplicada al almacenamiento, permite expandir o contraer los servidores del sistema de ficheros *ad hoc* según la demanda de E/S en cada fase de la ejecución.

2.2. Tecnologías y modelos existentes

2.2.1. Sistemas de ficheros ad hoc

Durante décadas, el almacenamiento principal de los clústeres HPC se basó en discos magnéticos. Aunque efectivos para cargas de trabajo secuenciales, el rendimiento de estos discos presenta deficiencias notables al realizar tareas de acceso aleatorio (acceso a datos localizados en posiciones no contiguas del disco), lo que genera un severo cuello de botella en las aplicaciones modernas [6].

Con el surgimiento de aplicaciones intensivas en datos (IA, Big Data y simulaciones de alta resolución), que se caracterizan precisamente por estos patrones de acceso irregular, el mayor reto de los clústeres HPC se convirtió en cómo hacer frente a la cantidad de datos, su complejidad y la velocidad a la que se generan [6].

Dado que los sistemas de ficheros paralelos tradicionales, como Lustre o GPFS, fueron diseñados asumiendo las capacidades de esos discos magnéticos, los sistemas de ficheros *ad hoc* surgieron como una respuesta directa a sus carencias ante las nuevas demandas de la computación de alto rendimiento [6]. A continuación se describen algunos de los sistemas más relevantes:

GekkoFS

Este sistema se caracteriza por un uso agresivo de funciones *hash* para distribuir de manera equilibrada los metadatos y datos entre los nodos del clúster. Su rasgo más característico es su gestión de directorios. Se crean de forma similar a los ficheros, lo que maximiza el rendimiento, pero convierte una tarea sencilla como *ls* (listar archivos) en una operación muy compleja computacionalmente hablando[6].

BurstFS

En cuanto a diseño básico se refiere, es bastante similar a GekkoFS, la diferencia principal entre ambos es la manera en la que BurstFS escribe datos. Usa el almacenamiento local de cada nodo para hacer escrituras estructuradas en registros (*log-structured*). Esto suprime la latencia de red en la fase de escritura de datos[6].

BeeOND (BeeGFS On Demand)

Para definir este subsistema conviene definir de antemano aquello en lo que está basado. BeeGFS es un sistema de ficheros paralelo y distribuido construido en función de la semántica POSIX [2], permite un acceso de lectura y escritura a sus ficheros transparente para las aplicaciones. BeeOND es una variante de este sistema que permite desplegar instancias de este servicio en los nodos de cómputo asignados a un trabajo [6].

Hercules

Este sistema de ficheros aprovecha la memoria principal de los nodos de cómputo como almacenamiento temporal. Su arquitectura cliente-servidor se apoya en el framework UCX como capa de comunicación, lo que le proporciona portabilidad de red y soporte nativo para protocolos de alto rendimiento como RDMA e InfiniBand, eliminando la intervención del kernel en las transferencias de datos. Adicionalmente, ofrece una interfaz completamente compatible con POSIX y gestiona los metadatos en dos niveles donde la información de POSIX se almacena en el «Bloque 0» de cada dataset, maximizando el paralelismo y evitando servidores de metadatos dedicados [7].

Expand Ad-Hoc

Su elemento diferenciador es la capacidad de aplicar maleabilidad combinada con tolerancia a fallos basada en la replicación de bloques. Se caracteriza por ofrecer algoritmos avanzados de reconfiguración, destacando el método de reconstrucción por metadatos, lo que le permite expandir o reducir el sistema moviendo solo la cantidad mínima de datos. Está construido utilizando el estándar MPI, lo que asegura su portabilidad en supercomputadores y permite optimizar la localidad de los datos de forma transparente para el usuario [4].

Debido a que este trabajo requiere una gestión dinámica de los recursos para adaptarse a las variaciones de la simulación, se utilizará Expand Ad-Hoc por su capacidad única de combinar maleabilidad y fiabilidad.

[2] La semántica POSIX (*Portable Operating System Interface*) define el comportamiento estándar de las operaciones de E/S sobre ficheros, garantizando atomicidad de las lecturas y escrituras y consistencia fuerte, de modo que toda escritura es inmediatamente visible para cualquier lectura posterior.

2.2.2. Tecnologías para aplicar maleabilidad

La maleabilidad es la capacidad de una aplicación paralela de modificar dinámicamente el número de procesos que utiliza durante su ejecución, en respuesta a variaciones de carga o a la disponibilidad de recursos en el sistema [8]. Esta propiedad se opone a las aplicaciones rígidas y moldeables, en las que el número de procesos queda fijado al inicio y no puede modificarse durante la ejecución [9]. Aunque el estándar MPI incorpora desde su versión 2 mecanismos para añadir procesos en tiempo de ejecución mediante `MPI_Comm_spawn`, no especifica cómo reducirlos, lo que ha motivado el desarrollo de múltiples bibliotecas y *frameworks* que extienden MPI para dar soporte completo a la maleabilidad [8]. A continuación se describen algunas de las propuestas más relevantes:

Adaptive MPI (AMPI)

Este sistema aporta maleabilidad mediante virtualización de procesos: mientras que en una implementación MPI tradicional cada proceso MPI se corresponde con un proceso pesado del sistema operativo asignado a un núcleo concreto, en AMPI cada proceso MPI se implementa como un hilo ligero de usuario, lo que permite ejecutar muchos más procesos MPI que núcleos físicos disponibles en el nodo. El *runtime*[3] planifica la ejecución de estos hilos sobre los núcleos reales y los migra entre nodos en tiempo de ejecución, habilitando así tanto el balanceo dinámico de carga como la reconfiguración del número de recursos [9].

PCM/IOS

La biblioteca PCM (*Process Checkpointing and Migration*) extiende MPI con operaciones de *split* y *merge* sobre aplicaciones iterativas, permitiendo tanto modificar el número de procesos como migrarlos entre nodos. Distingue entre operaciones *1-a-N* (más adecuadas para fluctuaciones pequeñas) y *M-a-N* (útiles ante grandes variaciones de recursos). Integra además un sistema de monitorización del hardware (IOS) que decide automáticamente cuándo aplicar cada tipo de operación [10].

ULFM + MPI_Comm_spawn

ULFM (*User Level Failure Mitigation*) es una extensión propuesta del estándar MPI diseñada originalmente para tolerancia a fallos, permitiendo que las aplicaciones puedan seguir ejecutándose tras la caída inesperada de uno o varios procesos. Lemarinier et al. [11] observaron que estos mismos mecanismos podían reutilizarse para aplicar maleabilidad de forma controlada: en lugar de detener la aplicación y guardar su estado en disco

[3] El *runtime* es la capa de software que acompaña a la aplicación durante su ejecución y se encarga de gestionar tareas como la planificación de hilos, la comunicación entre procesos o la asignación de recursos, de forma transparente para el programador.

para relanzarla con un número distinto de procesos, los datos se transfieren directamente entre los procesos en memoria, lo que reduce drásticamente el tiempo de reconfiguración [11].

MPI Sessions + PMIx

Esta propuesta integra la maleabilidad directamente en el modelo de *MPI Sessions*, apoyándose en el estándar PMIx como canal de comunicación entre la aplicación, el *runtime* de MPI y el gestor de recursos del sistema. Su aportación más original es representar los recursos como conjuntos de procesos y los cambios sobre ellos como operaciones de conjuntos (unión, diferencia, intersección), de modo que tanto añadir como eliminar procesos se expresan bajo un mismo formalismo común [12].

FlexMPI

Desarrollado en la Universidad Carlos III de Madrid, FlexMPI se implementa como una capa sobre MPICH e integra monitorización, balanceo de carga, redistribución de datos y gestión dinámica de procesos. Su característica diferencial es la reconfiguración guiada por rendimiento (*performance-aware*): mediante contadores hardware (vía PAPI) y *profiling* de comunicaciones (vía PMPI), recoge métricas en tiempo real y las alimenta a un modelo de predicción computacional que decide si añadir o eliminar procesos resulta realmente beneficioso para la aplicación. Esto permite reconfigurar bajo criterios definidos por el usuario, como objetivos de tiempo de ejecución, eficiencia paralela o coste operacional [9].

Dado que este trabajo requiere maleabilidad sobre una aplicación MPI ya existente (EpiGraph, desarrollada por el mismo grupo de UC3M y empleada como benchmark de validación de FlexMPI [9]), se utilizará FlexMPI por su integración directa sobre MPICH y por su capacidad de tomar decisiones de reconfiguración basadas en métricas de rendimiento reales.

2.2.3. Tecnologías de despliegue de contenedores

Un contenedor es una unidad de software que empaqueta una aplicación junto con todas sus dependencias (bibliotecas, configuración, variables de entorno) para poder ejecutarla de forma aislada y reproducible en cualquier sistema. A diferencia de la virtualización tradicional, que requiere un hipervisor[4] y simula un sistema operativo completo para cada máquina virtual, los contenedores se ejecutan directamente sobre el núcleo del sistema operativo anfitrión, lo que reduce drásticamente la sobrecarga y permite densidades

[4]Un hipervisor es la capa de software que permite ejecutar varios sistemas operativos sobre un mismo hardware físico, gestionando el aislamiento y el reparto de recursos entre las máquinas virtuales.

de despliegue mucho mayores. Esta ligereza, unida a la garantía de reproducibilidad entre entornos, ha convertido a los contenedores en una herramienta clave para el despliegue de aplicaciones en HPC, desarrollo y producción. A continuación se describen algunas de las tecnologías más relevantes:

LXC (Linux Containers)

LXC es una de las tecnologías pioneras de virtualización a nivel de sistema operativo en Linux, y constituye la base sobre la que se construyeron muchas soluciones posteriores. Su rasgo característico es ofrecer contenedores que se comportan de forma muy similar a una máquina virtual completa — con su propio conjunto de procesos, red y sistema de ficheros — pero sin el coste de un hipervisor. Está orientado a usuarios avanzados que necesitan un control preciso sobre los recursos y el aislamiento del sistema [13].

Podman

Podman es un motor de contenedores compatible con las imágenes de Docker cuya principal aportación es su arquitectura sin demonio (*daemonless*) y su capacidad de ejecutar contenedores en modo *rootless*, es decir, sin privilegios de superusuario. Estas características lo convierten en una opción especialmente atractiva en entornos donde la seguridad es prioritaria, como infraestructuras compartidas o clústeres HPC en los que los usuarios no disponen de permisos de administración [14].

Singularity / Apptainer

Singularity (renombrado como Apptainer tras su incorporación a la Linux Foundation) es una tecnología de contenedores diseñada específicamente para entornos HPC y computación científica. Su característica diferencial es un modelo de seguridad en el que el contenedor se ejecuta con los permisos del usuario que lo lanza, integrándose de forma natural con los gestores de colas (Slurm, PBS) y con redes de alto rendimiento como InfiniBand [15].

Docker

Docker es la plataforma de contenedores más extendida en la actualidad. Su característica diferencial es haber popularizado un ecosistema completo en torno al contenedor: un formato estándar de imágenes, un registro público (Docker Hub) con miles de imágenes preconstruidas, herramientas declarativas para definir entornos Dockerfile y para orquestar despliegues multi-contenedor docker-compose. Esto facilita enormemente la reproducibilidad de entornos complejos y la portabilidad del software entre máquinas de desarrollo, pruebas y producción [13].

Para el desarrollo de este trabajo se utilizará Docker como herramienta de despliegue, dado que se utilizarán distintos sistemas sobre los que se ejecutarán EpiGraph, FlexMPI y Expand Ad-Hoc que requieren coordinar varios servicios con dependencias específicas (MPICH, PAPI, bibliotecas de XPN), y el ecosistema de Docker — en particular docker-compose — permite definir esta infraestructura de forma declarativa y reproducible.

2.3. Otros trabajos similares

2.3.1. Maleabilidad de aplicaciones

Más allá de los trabajos que combinan ambas dimensiones, existe una amplia producción centrada exclusivamente en la maleabilidad de las aplicaciones MPI. Martín et al. [9] presentan FlexMPI, una biblioteca que permite añadir y eliminar procesos en tiempo de ejecución guiándose por un modelo de predicción de rendimiento, y la validan sobre Jacobi, Conjugate Gradient y EpiGraph. Posteriormente, Fernández et al. proponen una estrategia compatible con MPI 4.0 que permite añadir y eliminar nodos de forma controlada utilizando tanto `MPI_Comm_spawn` como `MPI_Comm_accept/connect`, validándola sobre la aplicación CFD WaComM++ [8]. En cuanto a la redistribución de datos — la fase más costosa de toda reconfiguración —, Martín-Álvarez et al. [16] comparan diferentes combinaciones de métodos de expansión/contracción (*Baseline* y *Merge*) con estrategias de comunicación punto-a-punto y colectivas, concluyendo que el método *Merge* con colectivas síncronas ofrece el menor tiempo de reconfiguración.

2.3.2. Maleabilidad de sistemas de ficheros

Otra línea de investigación se centra en dotar de maleabilidad a los propios sistemas de ficheros *ad hoc*, permitiendo expandir o reducir el número de servidores de datos sin detener la aplicación. Sánchez-Gallegos et al. [17] presentan una versión maleable de Hercules capaz de redistribuir los datos almacenados en memoria entre un número variable de nodos, adaptándose a las demandas cambiantes de E/S de la aplicación en tiempo de ejecución. En la misma línea, dentro del proyecto ADMIRE, GekkoFS ha sido extendido con mecanismos que permiten añadir y retirar servidores de E/S durante la ejecución de un trabajo, aprovechando su arquitectura desacoplada en espacio de usuario [18]. Por su parte, Muñoz-Muñoz et al. presentan el diseño de maleabilidad de Expand Ad-Hoc, cuya principal innovación reside en combinar la capacidad de expansión y contracción del sistema de ficheros con mecanismos de tolerancia a fallos basados en replicación de datos, consiguiendo tiempos de reconfiguración muy reducidos y mejoras de hasta un 100 % respecto al sistema de ficheros nativo en aplicaciones reales [4].

Cabe destacar que todos los trabajos revisados sobre maleabilidad de sistemas de ficheros *ad hoc* se han evaluado exclusivamente sobre clústeres HPC reales gestionados mediante Slurm. No se han encontrado en la literatura trabajos que combinen maleabili-

dad de sistemas de ficheros con despliegue en entornos containerizados. El presente TFG contribuye a explorar esta combinación por primera vez, desplegando Expand Ad-Hoc sobre un clúster virtualizado de contenedores Docker.

2.3.3. Maleabilidad combinada de aplicaciones y sistemas de ficheros

El trabajo más relacionado con este proyecto es el realizado por Sánchez-Checa et al. [5]. En él se propone un sistema de evaluación dinámica que combina maleabilidad computacional y maleabilidad de sistema de ficheros sobre el simulador epidemiológico EpiGraph. La solución propuesta integra FlexMPI para gestionar de forma dinámica el número de procesos además de Hercules que funciona como sistema de ficheros distribuidos *ad hoc* en memoria.

Con el objetivo de averiguar el mejor momento para aplicar los rebalanceos, se emplea un modelo predictivo basado en redes LSTM que estima el tiempo de CPU de ejecución de la simulación en función de las trazas de FlexMPI. Los resultados finales muestran reducciones del 80 % en el tiempo de CPU, de más del 14 % en el de E/S y del 42 % en el tiempo total de ejecución, ejecutando el sistema en un clúster HPC real gestionado con SLURM [5].

Un antecedente relevante es el trabajo de Singh y Carretero que integra FlexMPI con CLARISSE, un *runtime* de control de E/S entre capas, para coordinar la maleabilidad de las aplicaciones con la planificación de las operaciones de E/S [19]. Su contribución principal es demostrar que, cuando varias aplicaciones se ejecutan simultáneamente y compiten por el acceso al sistema de ficheros compartido, la combinación de ambos mecanismos permite reducir la contención de E/S redistribuyendo dinámicamente los procesos entre las aplicaciones en ejecución.

Ambos trabajos se enmarcan dentro del proyecto europeo ADMIRE (*Adaptive Multi-tier Intelligent Data Manager for Exascale*), financiado por EuroHPC, cuyo objetivo es crear una pila de E/S activa que ajuste dinámicamente los recursos de cómputo y almacenamiento mediante coordinación global y maleabilidad [20].

Este TFG parte de la misma arquitectura que combina EpiGraph con FlexMPI, pero explora un entorno diferente: los nodos de cómputo son contenedores Docker y el sistema de ficheros *ad hoc* empleado es Expand Ad-Hoc en lugar de Hercules. Este contexto introduce restricciones distintas — red virtual, CPU compartida entre contenedores — y permite estudiar cómo se comporta la maleabilidad en un entorno reproducible y de bajo coste, complementando los resultados obtenidos sobre clústeres HPC reales.

3. ENTORNO DE DESARROLLO

3.1. Contexto del proyecto

El entorno en el que se ejecuta este trabajo consiste en un despliegue en un equipo personal que cuenta con un procesador Intel Core i5-12450H. Este viene provisto de 4 núcleos de alto rendimiento (*P-cores*) y 4 núcleos de eficiencia (*E-cores*). Para garantizar la reproducibilidad de los experimentos, reducir el ruido y el margen de error y aislar las dependencias del software, todas las ejecuciones se llevan a cabo sobre un cierto número de contenedores Docker, cada uno con una imagen Ubuntu, anclados a los P-cores del procesador para evitar interferencias con los núcleos de eficiencia e incongruencias entre los resultados. Dentro de estos contenedores se integran tres componentes principales. Para empezar, EpiGraph [21], el simulador epidemiológico, configurado para simular el brote de las variantes Delta y Omicron del SARS-CoV-2 en España durante 130 días, con una población de 800.826 individuos. Este escenario presenta dos olas epidémicas diferenciadas con picos y valles de infectados bien marcados, provocando una clara variación en la carga computacional, esta situación es idónea para introducir la gestión dinámica de recursos.

En segundo lugar FlexMPI [22], que actúa como capa de maleabilidad y monitorización, ajustando dinámicamente el número de procesos en tiempo de ejecución en función de la carga computacional. Además, recoge métricas de rendimiento en tiempo real, lo que resulta de gran ayuda para su posterior evaluación.

Por último, Expand Ad-Hoc [3], integrado como sistema de ficheros distribuido en memoria sobre los propios contenedores, proporcionando almacenamiento paralelo.

Para evaluar el comportamiento del sistema más allá de los límites de un único equipo personal, los experimentos a gran escala se ejecutan sobre el clúster Tucán del grupo de investigación de la Universidad Carlos III de Madrid. De sus nodos se emplean los identificados como `compute-15-1` a `compute-15-4`, cada uno equipado con dos procesadores Intel Xeon Silver 4316 —20 núcleos y 40 hilos por socket, lo que suma 40 núcleos físicos y 80 hilos lógicos por nodo— a 2,3 GHz y 128 GiB de memoria RAM, bajo el sistema operativo Ubuntu 24.04.

El uso de dos entornos responde a objetivos complementarios. El equipo personal permite el desarrollo, la depuración y la ejecución de experimentos de escala reducida con una población sintética de 800.826 individuos. El clúster Tucán, en cambio, habilita la ejecución con una población notablemente mayor —de 3.511.110 individuos— y con un número de procesos muy superior, lo que permite comprobar la escalabilidad del sistema y el comportamiento de la maleabilidad y del almacenamiento paralelo en condiciones representativas de un entorno HPC real.

El modelo de despliegue es análogo en ambos entornos: los tres componentes se integran en contenedores Docker anclados uno a uno a los núcleos físicos para evitar interferencias. La diferencia esencial aparece en los experimentos multinodo, en los que EpiGraph y los servidores de almacenamiento de Expand Ad-Hoc se distribuyen entre nodos físicos distintos —por ejemplo, la aplicación en `compute-15-1` y los servidores XPN en `compute-15-3`—. Para ello se emplea Docker Swarm con una red *overlay* que interconecta los contenedores de ambos nodos, de modo que las operaciones de E/S atraviesan la red física real del clúster en lugar de resolverse mediante la interfaz de *loopback* local. El acceso al clúster y la reserva de nodos se gestionan mediante el planificador SLURM.

3.2. Tecnologías empleadas

Este trabajo integra tres componentes de software desarrollados en el grupo de investigación de la Universidad Carlos III de Madrid: el simulador epidemiológico EpiGraph, la biblioteca de maleabilidad FlexMPI y el sistema de ficheros distribuido Expand Ad-Hoc (XPN). Además, se utiliza Docker como sistema en el que se integran los tres componentes.

3.2.1. EpiGraph

EpiGraph [21] es un simulador epidemiológico basado en agentes que modela la propagación de enfermedades infecciosas a nivel individual. Diseñado para ejecutarse de forma paralela sobre clústeres HPC, ha sido utilizado para apoyar la toma de decisiones de las autoridades sanitarias españolas y de la Unión Europea [21], [23] . En octubre de 2021, el equipo de Epigraph se incorporó al comité del Ministerio de Sanidad de España para apoyar la campaña de vacunación contra la COVID-19 [24].

Se diferencia de otros modelos en los siguientes aspectos: Al ser un simulador basado en agentes, este modelo tiene en cuenta el comportamiento de cada individuo y es capaz de captar de manera eficaz patrones de interacción complejos, lo que permite realizar predicciones más precisas en comparación con otros enfoques tradicionales [5]. Asimismo, el sistema destaca por ser altamente escalable y por representar directamente tanto las características particulares de los individuos como sus interconexiones. Esta flexibilidad estructural también permite modelar con precisión el efecto de factores externos no relacionados directamente con el virus en sí, tales como las estrategias de vacunación o diversas intervenciones no farmacológicas, entre las que se incluyen el uso de mascarilla o el mantenimiento de la distancia social entre los individuos [21].

EpiGraph integra a su vez varios modelos que interactúan entre sí durante la simulación [21]:

- **Modelo social:** reproduce las actividades diarias y los contactos sociales de cada

individuo, estructurados en torno al trabajo o estudio, tiempo de ocio y tiempo en familia. Considera cuatro colectivos diferenciados (trabajadores, estudiantes, desempleados y mayores) así como nueve tipos de profesiones distintas, utilizando datos demográficos reales del censo español.

- **Modelo epidémico:** simula la propagación de la enfermedad y su evolución en cada individuo infectado mediante un modelo que incluye estados como susceptible, expuesto, asintomático, infectado, hospitalizado, recuperado y fallecido.
- **Modelo de transporte:** representa los desplazamientos de individuos entre ciudades, diferenciando entre desplazamientos de corta y larga distancia.
- **Modelo de intervenciones no farmacológicas:** reproduce las medidas adoptadas por las autoridades, incluyendo distanciamiento social (diferenciado por tipo de contacto), restricciones de movilidad, uso de mascarillas quirúrgicas y FFP2, y estrategias de testeo poblacional.
- **Modelo de vacunación:** determina la eficacia de la vacuna para cada individuo en función de la variante del virus, el tipo de vacuna, si se trata de la primera o segunda dosis, y las características personales como la edad o si ha sido previamente infectado [21].

Internamente, cada persona del modelo está representada como un nodo de un grafo de contacto, almacenado como una matriz dispersa en formato CSC (*Compressed Sparse Column*)[5], recorriendo en cada iteración únicamente los nodos en estado infeccioso y sus aristas (véase Fig. 3.1). En esa figura, el array `p` marca dónde empiezan dentro de `i` los contactos de cada persona, y en rojo aparece la columna del individuo infectado, la única que se recorre.

De esta manera el coste de cómputo es proporcional al número de individuos infectados en cada momento [5]. A más infectados, mayor carga computacional.

A su vez, las operaciones de escritura guardan el estado completo de los N individuos con independencia del nivel de infección (no importa si el individuo está infectado, sano, muerto, etc.), de modo que el tiempo de E/S se mantiene prácticamente constante a lo largo de la ejecución [5].

Al final de cada día simulado, EpiGraph escribe un *checkpoint* con el estado completo de la simulación, lo que permite reanudar la ejecución o redistribuir la carga de trabajo entre procesos sin perder progreso.

Esta aparente asimetría estructural —una carga de cómputo variable que crece con la curva epidémica y una E/S fija proporcional a la población total — provoca que la

[5]El formato CSC (Compressed Sparse Column) es una estructura de datos para almacenar matrices dispersas (con muchos ceros) de forma eficiente en memoria, EpiGraph guarda los contactos de cada persona en esta matriz.

aplicación sea variable en cómputo y estable en carga de E/S, lo que provoca la necesidad de utilizar la maleabilidad para alcanzar el mejor rendimiento de la aplicación. Sin embargo, conviene matizar lo siguiente: aunque la carga de E/S es la misma en toda la ejecución, esta carga se encuentra menos espaciada cuanto menor es la carga computacional. Por ello (en términos temporales) la carga de E/S es inversamente proporcional a la computacional.

En este trabajo se configura para simular la doble ola de las variantes Delta y Ómicron del SARS-CoV-2 en España a lo largo de 130 días simulados, con 1440 iteraciones por día, una por cada minuto simulado. Se emplean dos poblaciones sintéticas: una reducida, de 800.826 individuos, con la que se trabaja en el equipo local, y la completa de la ciudad de Madrid, de 3.511.110, reservada al clúster. La frecuencia con que se vuelca el estado es asimismo configurable: un único volcado por día simulado o diez, esto es, cada 1440 o cada 144 iteraciones. Ambos parámetros determinan la escala de la carga y se detallan, campaña por campaña, en la Sección 5.4.

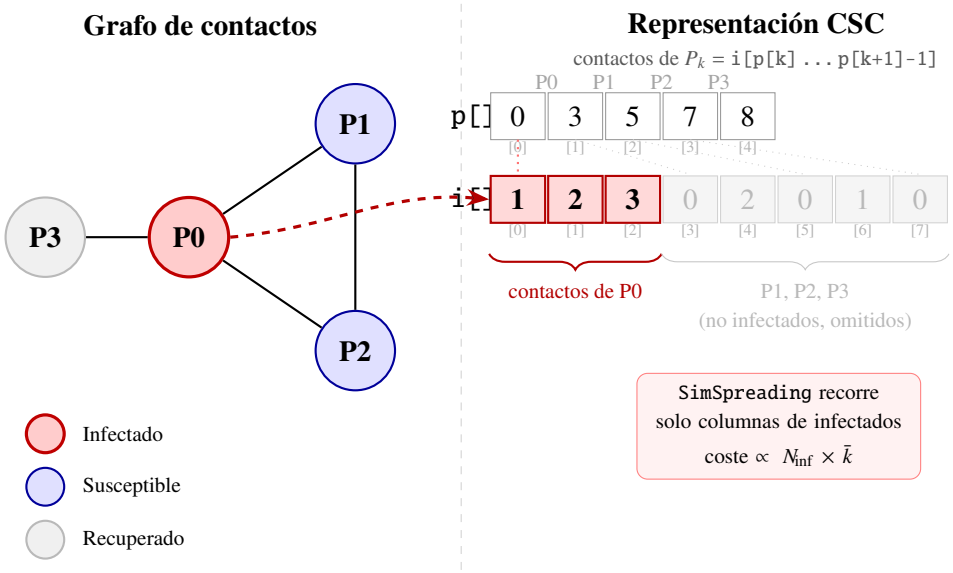


Fig. 3.1. Grafo de contactos de EpiGraph y su representación interna en formato CSC.

3.2.2. FlexMPI

FlexMPI [22] es una extensión de MPI desarrollada en la UC3M que añade capacidades de maleabilidad y balanceo dinámico de carga a aplicaciones paralelas con patrón SPMD (*Single Program, Multiple Data*). Está diseñada para adaptarse a variaciones en la carga de trabajo (tanto internas, derivadas de la propia aplicación, como externas, provocadas por otros procesos que compiten por los mismos recursos)[1].

Sus tres funcionalidades principales son: la monitorización, que recoge métricas de rendimiento en tiempo real mediante contadores hardware a través de la librería PAPI (operaciones de cómputo) y la interfaz de perfilado de MPI (comunicaciones), con un impacto prácticamente imperceptible sobre el rendimiento; el balanceo dinámico de carga,

que evalúa el Poder de Cómputo Relativo (RCP) de cada proceso y, si detecta un desequilibrio, calcula una nueva distribución del trabajo proporcional al rendimiento medido; y la redistribución de datos, que ejecuta la transferencia de datos entre procesos activos para materializar la distribución calculada [5], [22].

Adicionalmente, FlexMPI aplica maleabilidad al número de procesos MPI activos en tiempo de ejecución mediante dos mecanismos: el modo dinámico[6]; y el modo stop/restart, que detiene la aplicación en un *checkpoint*, reconfigura el número de procesos y la relanza desde ese punto [5].

La integración con EpiGraph se realiza con dos funciones que componen la monitorización de los procesos activos. Al inicio de cada día simulado FlexMPI inicializa los contadores de rendimiento; al final evalúa si la distribución actual de trabajo es adecuada, de modo que las posibles reconfiguraciones se evalúan una vez por día de simulación.

Para decidir cómo redistribuir el trabajo, EpiGraph proporciona a FlexMPI un vector de costes con una entrada por cada individuo de la partición local del proceso. Cada entrada refleja la carga computacional real de ese individuo: los nodos infectados con muchos contactos tienen coste elevado, mientras que los nodos susceptibles, recuperados o fallecidos tienen coste cero, ya que la fase de propagación epidémica solo recorre los nodos en estado infeccioso, tal y como se describe en la sección 3.2.1.

La Fig. 3.2 ilustra este mecanismo. FlexMPI agrega estos costes por proceso y, si detecta un desequilibrio entre procesos, calcula una nueva distribución y ejecuta la reconfiguración. Tras cada reconfiguración, EpiGraph actualiza sus estructuras internas para reflejar la nueva distribución de datos entre procesos.

3.2.3. Expand Ad-Hoc (XPN)

Expand Ad-Hoc (XPN) es un sistema de ficheros paralelo ad hoc diseñado para entornos HPC [3], [25] . A diferencia de los sistemas de ficheros tradicionales, XPN no requiere hardware de almacenamiento permanente: puede desplegar procesos servidores sobre cualquier nodo disponible en el entorno —ya sean nodos compartidos con la aplicación o nodos reservados exclusivamente para almacenamiento—, que guardan los datos en memoria RAM y exponen un espacio de nombres de ficheros compartido accesible desde todos los procesos de la aplicación [4].

El mecanismo central de XPN es la distribución de datos en bloques: cuando se escribe un fichero, XPN lo divide en bloques de tamaño fijo configurado por el parámetro tamaño de bloque (*bsize*) y los reparte entre los servidores disponibles siguiendo una política de reparto round-robin[7] [3]. Esta política garantiza que los bloques de un mismo fichero

[6] El modo dinámico emplea `MPI_Comm_spawn` para añadir o eliminar procesos sin detener la aplicación.

[7] Política de asignación cíclica en la que los recursos se distribuyen de forma rotativa y equitativa entre los participantes, asignando uno a cada uno en orden antes de volver al primero.

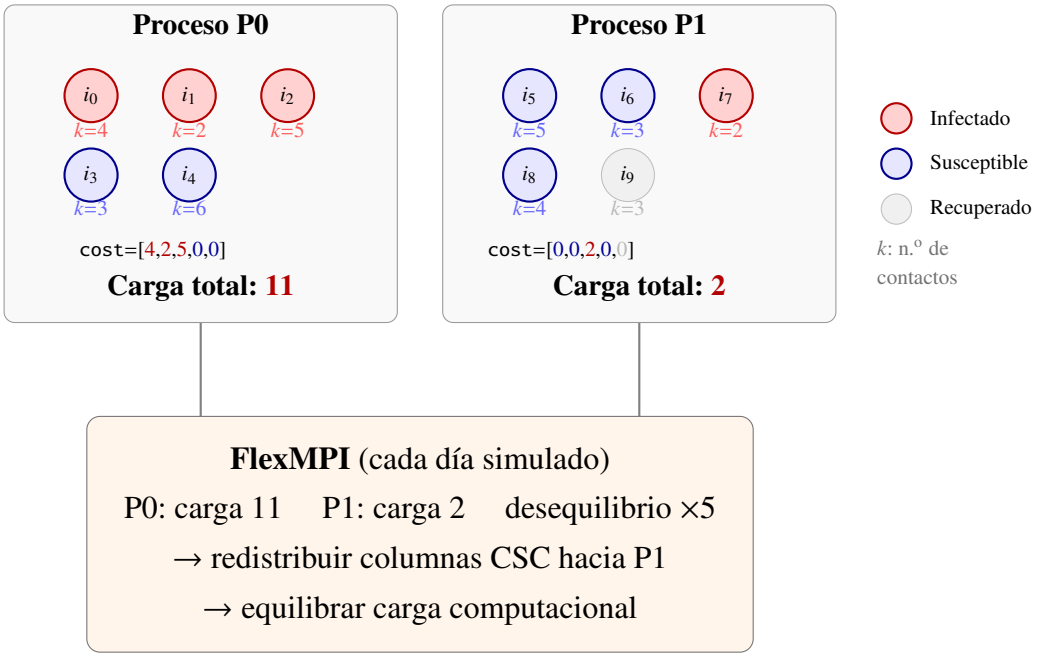


Fig. 3.2. Vector de costes que EpiGraph entrega a FlexMPI al final de cada día simulado.

quedan distribuidos de forma equilibrada entre todos los servidores, tal y como ilustra la Fig. 3.3.

XPN gestiona los metadatos de forma distribuida: los metadatos de cada bloque se almacenan como cabecera junto al bloque en el mismo servidor que lo contiene, sin necesidad de servidores dedicados exclusivamente a metadatos. Los metadatos globales del fichero (tamaño de bloque, servidor con el primer bloque, etc.) recaen sobre uno de los propios servidores de datos, que ejerce de servidor maestro para ese fichero sin abandonar su rol de almacenamiento [4]. De este modo, cualquier nodo puede actuar simultáneamente como servidor de datos, servidor maestro de algunos ficheros y nodo de cómputo de la aplicación.

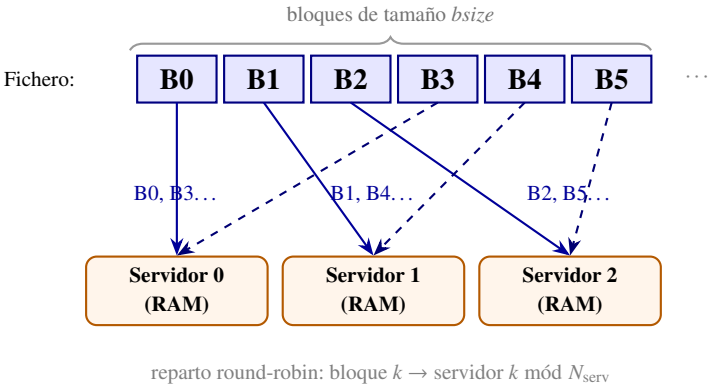


Fig. 3.3. Reparto en *round-robin* de los bloques de un fichero entre tres servidores de XPN.

En el caso en el que un proceso MPI escribe tres bloques consecutivos de su partición del *checkpoint*, XPN descompone esa operación en tres sub-peticiones independientes

lanzadas en paralelo: B0 va al Servidor 0, B1 al Servidor 1 y B2 al Servidor 2 de forma simultánea. En ese mismo instante, otro proceso MPI puede estar escribiendo sus propios bloques (B3, B4, B5...) sobre esos mismos servidores sin necesidad de coordinarse: XPN determina el destino de cada bloque de forma independiente, por lo que ambas escrituras progresan sin interferirse.

Los procesos servidor de XPN admiten dos modalidades de despliegue. En el modo colocalizado, cada nodo ejecuta simultáneamente un servidor XPN y procesos MPI de la aplicación: los servidores aprovechan la RAM de los nodos de cómputo sin consumir nodos adicionales. En el modo exclusivo, un subconjunto de nodos se reserva íntegramente para los servidores XPN, de modo que la aplicación y el sistema de ficheros no compiten por los mismos núcleos. En este trabajo se han empleado las dos modalidades.

La integración con la aplicación no exige modificar su código fuente. XPN proporciona una biblioteca de cliente que, precargada mediante `LD_PRELOAD`, se interpone entre la aplicación y el sistema de ficheros: intercepta las llamadas de entrada/salida y las redirige hacia los servidores de datos, que las atienden desde memoria. La Figura 3.4 resume ese papel de intermediario. Sin esa precarga —el trazo discontinuo de la figura— las mismas llamadas descienden hasta el sistema de ficheros compartido del clúster y XPN no llega a intervenir, un extremo que resultó determinante durante la puesta en marcha (Sección 5.2.3).

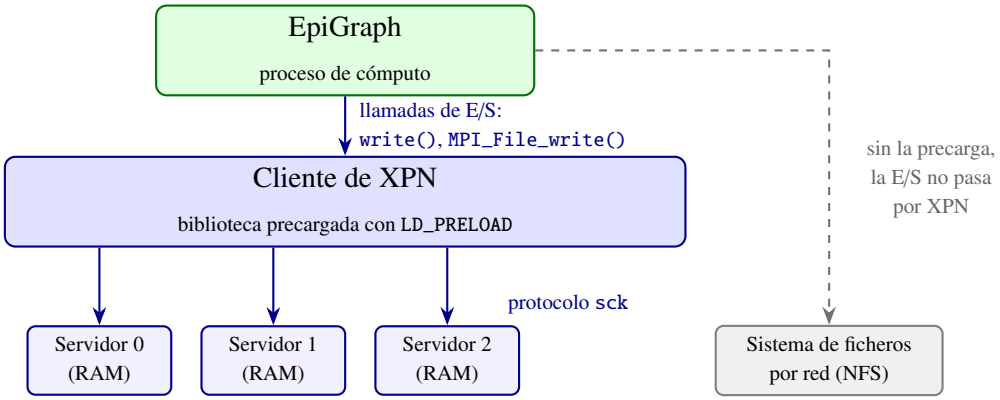


Fig. 3.4. XPN como intermediario entre la aplicación y el sistema de ficheros.

3.2.4. Docker

Docker es una plataforma de contenedores que automatiza el despliegue, la ejecución y la distribución de aplicaciones [13]. Los contenedores son unidades estándar que empaquetan el software junto con todas sus dependencias y archivos de configuración, garantizando que la aplicación se ejecute de forma idéntica con independencia del entorno [13].

A diferencia de las máquinas virtuales, los contenedores se ejecutan directamente sobre el núcleo del sistema operativo anfitrión, sin virtualizar hardware, lo que reduce

significativamente el consumo de recursos y el tiempo de arranque. Docker sigue una arquitectura cliente-servidor: el usuario se comunica con el Docker Daemon, que gestiona el ciclo de vida de los contenedores.

Los dos elementos fundamentales son: las imágenes, plantillas inmutables que especifican el sistema operativo base, las dependencias y la configuración de la aplicación; y los contenedores, instancias ejecutables de una imagen, aisladas entre sí y del sistema anfitrión.

En este trabajo, Docker se emplea para empaquetar el entorno experimental completo y garantizar su reproducibilidad. Cada contenedor actúa como un nodo virtual independiente y ejecuta de forma colocalizada los tres componentes del sistema: el proceso de cómputo de EpiGraph, la biblioteca de maleabilidad FlexMPI y un servidor de Expand Ad-Hoc. Los contenedores se comunican a través de la red virtual de Docker, que transporta tanto los mensajes MPI como el protocolo `sck` de XPN, y cada uno se restringe mediante `cpuset-cpus` al conjunto de núcleos del procesador anfitrión sobre el que debe ejecutarse, de modo que el reparto entre nodos virtuales y núcleos reales queda fijado de antemano y no lo decide el planificador del anfitrión. El controlador de FlexMPI se ejecuta en el primer contenedor. El despliegue resultante se resume en la Figura 3.5, y las decisiones de diseño que lo sustentan se detallan en la Sección 4.4.

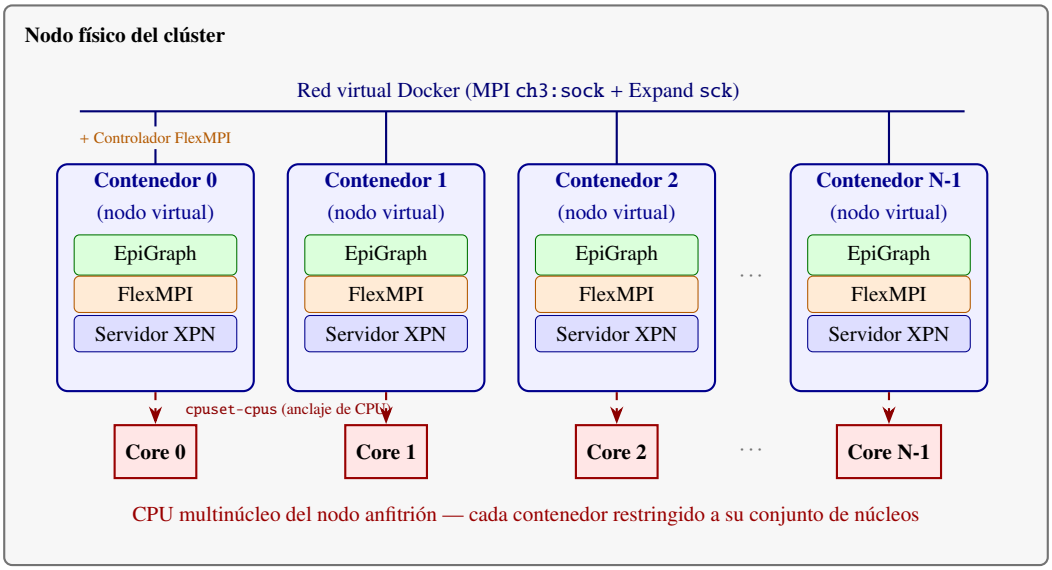


Fig. 3.5. Despliegue del entorno experimental sobre un nodo del clúster.

3.3. Marco legislativo

3.3.1. Licencias de software

Las herramientas de terceros empleadas en este trabajo se distribuyen bajo licencias que permiten su uso académico. PAPI se publica bajo la licencia BSD de tres cláusulas

[26], y Docker bajo la licencia Apache 2.0 [27]; ambas permiten la redistribución y el uso no comercial con la única condición de mantener el aviso de copyright original.

EpiGraph, FlexMPI y Expand Ad-Hoc son herramientas de investigación desarrolladas por el grupo ARCOS de la Universidad Carlos III de Madrid, y se distribuyen bajo licencias diferentes. EpiGraph se acoge a una licencia académica propia de la universidad, la Licencia Académica UC3M (LAUC3M) [28]; no se trata de una licencia de código abierto homologada, sino que autoriza el uso, la distribución y la creación de obras derivadas únicamente con fines no comerciales, y se distribuye sin garantías. FlexMPI se publica bajo la GNU General Public License versión 3 (GPLv3) [29], una licencia copyleft que obliga a distribuir cualquier obra derivada bajo esos mismos términos. Expand Ad-Hoc (XPN) se distribuye bajo la GNU Lesser General Public License versión 3 (LGPLv3) [30]; a diferencia de la GPL, la LGPL permite enlazar la biblioteca desde aplicaciones con otra licencia sin someterlas al copyleft, algo pertinente en este trabajo, donde la E/S se intercepta enlazando el cliente de XPN con EpiGraph.

El uso de las tres herramientas en este trabajo se enmarca en la investigación académica dentro de la propia UC3M, donde fueron desarrolladas, por lo que no existe conflicto de licencia. En ningún momento se redistribuyen estos componentes ni se hace uso comercial de ellos, que es la restricción común a las tres licencias.

3.3.2. Protección de datos

Los datos manejados en este trabajo son de naturaleza sintética. EpiGraph genera internamente un grafo de contactos artificiales a partir de parámetros estadísticos de población; en ningún momento se utilizan datos clínicos, de movilidad o de identificación personal reales. Por tanto, el tratamiento realizado no está sujeto a las obligaciones del Reglamento General de protección de datos (RGPD, UE 2016/679) [31] ni a la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [32].

No obstante, conviene precisar el marco normativo que resultaría de aplicación si en el futuro se quisiera alimentar el simulador con datos clínicos o epidemiológicos reales. El RGPD, por su condición de reglamento de la Unión Europea, es directamente aplicable en todos los Estados miembros sin necesidad de transposición, pero no agota la regulación: reserva a los legisladores nacionales una serie de «cláusulas de apertura» para concretar determinados aspectos. En España, esa concreción corresponde a la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [32], que adapta el ordenamiento español al RGPD, lo desarrolla y añade un título propio sobre la garantía de los derechos digitales. En la práctica, por tanto, el tratamiento de datos personales en territorio español se rige por la aplicación conjunta de ambas normas —el RGPD como marco común europeo y la LOPDGDD como norma nacional que lo desarrolla—, bajo la supervisión de la Agencia Española de Protección de Datos (AEPD).

Dentro de ese marco, los datos de salud constituyen una categoría especial de datos personales conforme al artículo 9 del RGPD y al artículo 9 de la LOPDGDD, cuyo tratamiento está prohibido con carácter general salvo que concurra alguna de las excepciones legalmente previstas. Su uso exigiría, por tanto, una base jurídica explícita —como el consentimiento explícito del interesado o razones de interés público en el ámbito de la salud pública—, una evaluación de impacto relativa a la protección de datos, la adopción de medidas técnicas y organizativas reforzadas (por ejemplo, la seudonimización de los registros) y, en su caso, la designación de un delegado de protección de datos.

3.3.3. Reglamento europeo de inteligencia artificial

El Reglamento (UE) 2024/1689 sobre inteligencia artificial (AI Act) [33] clasifica los sistemas de IA en función del riesgo que presentan. Los componentes desarrollados en este trabajo son herramientas de simulación e infraestructura HPC; no toman decisiones autónomas sobre personas ni se encuadran en ninguna de las categorías de alto riesgo recogidas en el Anexo III del reglamento. Por consiguiente, no les son aplicables los requisitos de conformidad, transparencia ni supervisión humana que el AI Act impone a los sistemas de alto riesgo.

3.4. Entorno socioeconómico

La simulación epidemiológica a gran escala desempeña un papel crucial en la respuesta a emergencias sanitarias. La pandemia de COVID-19 generó pérdidas económicas globales cercanas a los 13,8 billones de dólares según estimaciones del Fondo Monetario Internacional [34], lo que pone de manifiesto el valor de disponer de herramientas capaces de anticipar la propagación de enfermedades y apoyar la toma de decisiones de las autoridades sanitarias. Sin embargo, ejecutar simulaciones de este tipo a escala nacional o continental requiere recursos de computación de alto rendimiento que son costosos tanto económicamente como en términos de energía. En este contexto, la maleabilidad dinámica de procesos MPI representa una mejora directa sobre la eficiencia de uso de dichos recursos: al ajustar el número de procesos activos en función de la carga real de la simulación, se evita mantener procesadores ociosos durante las fases de baja incidencia epidémica y se concentra la potencia de cómputo cuando la propagación se acelera.

Desde el punto de vista técnico, el mecanismo de maleabilidad dinámica implementado en este trabajo reduce significativamente el coste de reconfiguración respecto del enfoque tradicional de parada y relanzamiento [5], lo que hace viable aplicar la maleabilidad con mayor frecuencia y granularidad. Esta mejora es directamente trasladable a entornos HPC de producción, donde el coste de las horas de cómputo tiene un impacto económico real para las instituciones que ejecutan simulaciones epidemiológicas. Por otro lado, el entorno experimental basado en contenedores Docker empleado en este trabajo permite reproducir el experimento completo en hardware de bajo coste, lo que reduce la

barrera de entrada para grupos de investigación con recursos limitados.

3.4.1. Objetivos de Desarrollo Sostenible

En lo relativo a los Objetivos de Desarrollo Sostenible establecidos por la Asamblea General de las Naciones Unidas en la Agenda 2030 [35], este proyecto se alinea principalmente con los objetivos de salud y bienestar, industria e infraestructura, acción por el clima, y alianzas para el logro de los objetivos.

En cuanto al ODS 3 (Salud y Bienestar), una simulación epidemiológica más eficiente permite a las autoridades sanitarias evaluar escenarios de propagación con mayor rapidez y menor coste, contribuyendo directamente a una mejor gestión de emergencias de salud pública. La maleabilidad dinámica hace posible además ejecutar estas simulaciones en infraestructuras más modestas, ampliando el acceso a este tipo de herramientas más allá de los grandes centros de supercomputación.

Respecto al ODS 9 (Industria, Innovación e Infraestructura), el mecanismo de reconfiguración dinámica desarrollado supone un avance técnico en la gestión de recursos en entornos HPC, con potencial de aplicación en cualquier simulación cuya carga computacional varíe a lo largo del tiempo, más allá del dominio epidemiológico.

En lo relativo al ODS 13 (Acción por el Clima), reducir el número de procesos MPI activos durante las fases de baja carga disminuye el consumo energético de los nodos de cómputo. Los centros de datos representan aproximadamente el 1,5 % del consumo eléctrico mundial [36], por lo que mejoras en la eficiencia de uso de los recursos HPC tienen un impacto acumulado no despreciable sobre las emisiones de carbono del sector.

Finalmente, en cuanto al ODS 17 (Alianzas para el Logro de los Objetivos), este trabajo hace uso íntegro de software de código abierto y herramientas de investigación reproducibles, contribuyendo a un ecosistema científico abierto y accesible para grupos con recursos limitados.

4. PROPUESTA

4.1. Objetivos, Metodología y Fases de desarrollo

El desarrollo de este trabajo siguió una metodología iterativa e incremental de carácter experimental, representada en la Figura 4.1. A diferencia de un modelo en cascada clásico, en el que cada fase debe cerrarse por completo antes de comenzar la siguiente, en este proyecto no era posible fijar la totalidad de los requisitos al inicio. El comportamiento del sistema resultante de integrar EpiGraph, FlexMPI y Expand Ad-Hoc sobre un clúster virtualizado con Docker era desconocido a priori y únicamente podía caracterizarse de forma empírica. Por este motivo, el trabajo se organizó en incrementos sucesivos en los que cada componente se integraba y validaba experimentalmente antes de abordar el siguiente, y en los que los resultados obtenidos en cada iteración redefinían las decisiones de la iteración posterior.

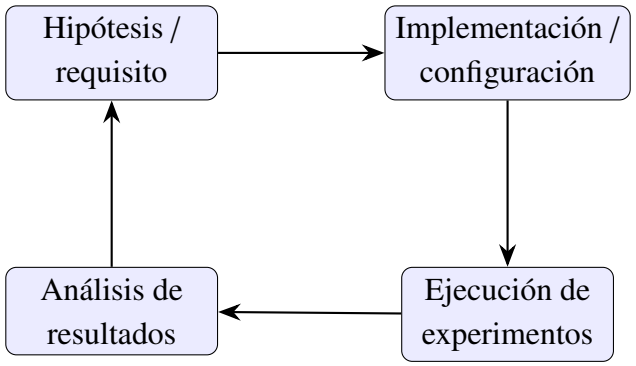


Fig. 4.1. Ciclo iterativo de desarrollo experimental.

Cada iteración siguió un ciclo común de cuatro etapas: el planteamiento de una hipótesis o requisito, su implementación o configuración, la ejecución de los experimentos correspondientes y el análisis de los resultados obtenidos. Así, la identificación del desbalanceo de carga derivado de la propagación no uniforme de la infección entre las particiones o la caracterización del coste de la entrada/salida no fueron requisitos de partida, sino hallazgos experimentales que motivaron las decisiones de diseño descritas en los capítulos siguientes. Este enfoque resultó el más apropiado dada la naturaleza del proyecto, orientado a medir el impacto de la maleabilidad y del sistema de ficheros paralelo sobre el rendimiento de la simulación. No obstante, se conservaron rasgos propios del desarrollo en cascada en la ordenación general del trabajo, que mantuvo una secuencia lógica desde la documentación inicial hasta la evaluación final, tal y como se detalla más adelante.

4.2. Planificación del proyecto

El proyecto fue formalizado en diciembre del año 2025 y se planificó para una duración aproximada de siete meses, hasta junio de 2026; en la práctica se extendió dos meses más, hasta agosto, por los motivos que se detallan al final de la sección. La planificación se organiza en torno a las grandes etapas identificadas en la Sección 4.1, cada una de ellas subdividida en las subfases que se describen a continuación y que se representan en el diagrama de Gantt de la Figura 4.2. Conviene advertir que, por el carácter iterativo del desarrollo, estas etapas no fueron estrictamente secuenciales: la integración y la experimentación se solaparon durante buena parte del proyecto y la primera se prolongó más allá de lo previsto, por los motivos que se detallan al final de la sección.

La etapa de documentación se desarrolló durante diciembre y enero y comprendió una única fase de formalización del trabajo con la tutora y de estudio de los artículos de referencia sobre EpiGraph, FlexMPI y Expand Ad-Hoc, a partir de la cual se redactaron las primeras secciones del estado de la cuestión.

La preparación del entorno experimental se llevó a cabo entre febrero y marzo. Consistió en la configuración del clúster virtualizado con Docker y en la puesta en marcha de EpiGraph y FlexMPI de forma local, de modo que se dispusiera de una plataforma reproducible sobre la que integrar el resto de componentes.

La integración de los tres sistemas se planificó inicialmente para el periodo de febrero a marzo y se abordó de forma incremental, validando cada capa antes de incorporar la siguiente (Sección 5.2). Se distinguen tres subfases: la ejecución local de EpiGraph enlazado con FlexMPI, que exigió resolver incompatibilidades de la pila de MPI y de los ficheros de compilación; el despliegue sobre el clúster virtual de contenedores, que obligó a construir un mapeo de nombres de host único por nodo para el lanzador y a corregir el desbalanceo de carga entre contenedores; y, por último, la incorporación del sistema de ficheros paralelo Expand Ad-Hoc junto con el mecanismo de maleabilidad basado en *checkpoint* y restauración.

En contraste con lo planificado, la integración fue la fase que más se prolongó, extendiéndose de facto hasta julio. El motivo es que la validación de cada capa reveló problemas que no podían anticiparse y cuya resolución requirió modificar el código fuente de FlexMPI y de EpiGraph: la activación del rebalanceo de carga, que no llegaba a dispararse en el entorno contenerizado; la corrección del contador de operaciones en coma flotante, que la instalación de PAPI del sistema notificaba como nulo; o la intercepción transparente de la E/S por parte de XPN, que en un primer momento eclipsaba la inicialización de MPI de FlexMPI y abortaba el arranque. Estas incidencias, recogidas de forma pormenorizada en la Sección 5.2 y sintetizadas en la Tabla 5.6, constituyeron la principal dificultad del trabajo. Un aspecto especialmente costoso fue que varias de ellas no se detectaron hasta que la fase de experimentación estaba ya en marcha —por ejemplo, que las métricas registrasen operaciones en coma flotante nulas o que XPN no estuviera interceptando

realmente la E/S de la aplicación—, de modo que su corrección invalidaba las medidas obtenidas hasta ese momento y obligaba a repetir campañas de experimentos completas. Esta realimentación entre integración y experimentación, propia de la metodología iterativa adoptada, es la que explica el amplio solapamiento de ambas fases en el diagrama de Gantt y el retraso acumulado respecto a la planificación inicial. Las celdas en naranja de la Figura 4.2 señalan ese retrabajo —las correcciones de código y las reejecuciones de experimentos que estas motivaron—, que se prolongó sobre la fase de experimentación, más allá del periodo inicialmente previsto para la integración.

La fase de experimentación se extendió desde marzo hasta agosto y se articuló en cuatro subfases parcialmente solapadas. La primera fue la caracterización del rendimiento de la E/S con distintas configuraciones de despliegue de XPN —colocalizado frente a exclusivo y variando el número de servidores—. La segunda consistió en la evaluación de los mecanismos de maleabilidad, tanto el basado en *checkpoint* y reinicio como el modo dinámico nativo de FlexMPI. La tercera fue la réplica de los experimentos sobre el clúster Tucán de la UC3M, que permitió validar los resultados a mayor escala. Como se ha indicado, varias de estas campañas hubieron de repetirse a raíz de las correcciones introducidas durante la integración. La cuarta, no prevista en la planificación inicial, fue la repetición sistemática de las configuraciones que sostienen las afirmaciones centrales del trabajo: las primeras campañas midieron una sola vez cada configuración, y al cuantificar la variabilidad del entorno se comprobó que varias de las diferencias observadas no superaban el ruido de medida. Elevar el número de repeticiones ocupó el último mes de trabajo y obligó a revisar algunas conclusiones ya redactadas.

Finalmente, la redacción de la memoria transcurrió de forma paralela a las fases de integración y experimentación, desde enero hasta el cierre del trabajo en agosto, incorporando de manera incremental los resultados a medida que se obtenían.

2025–2026									
Fase / subfase	Dic	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago
Documentación									
Formalización y estudio bibliográfico									
Preparación del entorno									
Clúster virtual y ejecución local									
Integración									
Ejecución local de EpiGraph y FlexMPI									
Despliegue en contenedores									
Expand Ad-Hoc y maleabilidad									
Correcciones y reejecuciones									
Experimentación									
Caracterización de la E/S (XPN)									
Mecanismos de maleabilidad									
Clúster Tucán									
Repetición de campañas									
Redacción de la memoria									

Fig. 4.2. Diagrama de Gantt del proyecto. Las celdas azules son la planificación original y las naranjas, el trabajo no previsto.

4.3. Análisis de requisitos

Aunque el desarrollo siguió la metodología iterativa descrita en la Sección 4.1, en la que parte de las decisiones surgieron de hallazgos experimentales, sí fue posible delimitar desde el inicio un conjunto mínimo de requisitos que acotaron el alcance del trabajo. Estos se dividen en requisitos funcionales, que describen las capacidades que debía ofrecer el sistema resultante de integrar EpiGraph, FlexMPI y Expand Ad-Hoc, y requisitos no funcionales, que recogen las restricciones tecnológicas y de calidad. Cada requisito se especifica siguiendo el formato de la Tabla 4.1.

TABLA 4.1. ESTRUCTURA PARA LA DEFINICIÓN DE REQUISITOS.

Identificador	Código único del requisito (RF/RNF-0x).
Nombre	Título breve y descriptivo.
Prioridad	Importancia relativa: alta, media o baja.
Necesidad	Grado de obligatoriedad: esencial o deseable.
Estabilidad	Probabilidad de que el requisito no varíe.
Descripción	Explicación detallada del requisito.

4.3.1. Requisitos funcionales

TABLA 4.2. REQUISITO FUNCIONAL RF-01.

Identificador	RF-01
Nombre	Interceptación transparente de la E/S
Prioridad	Alta
Necesidad	Esencial
Estabilidad	Alta
Descripción	El sistema debe redirigir las operaciones de entrada/-salida de EpiGraph hacia el sistema de ficheros paralelo XPN sin modificar el código fuente de la aplicación, mediante precarga de biblioteca.

TABLA 4.3. REQUISITO FUNCIONAL RF-02.

Identificador	RF-02
Nombre	Maleabilidad del número de procesos
Prioridad	Alta
Necesidad	Esencial
Estabilidad	Media
Descripción	Debe ser posible modificar el número de procesos MPI de EpiGraph durante la ejecución de una simulación, sin reiniciarla manualmente.

TABLA 4.4. REQUISITO FUNCIONAL RF-03.

Identificador	RF-03
Nombre	Rebalanceo dinámico de carga
Prioridad	Alta
Necesidad	Esencial
Estabilidad	Media
Descripción	El sistema debe redistribuir la carga de cómputo entre los procesos en respuesta al desbalanceo introducido por la propagación no uniforme de la infección entre las particiones de la población simulada.

TABLA 4.5. REQUISITO FUNCIONAL RF-04.

Identificador	RF-04
Nombre	Persistencia del estado en XPN
Prioridad	Media
Necesidad	Esencial
Estabilidad	Alta
Descripción	El estado de la simulación generado en las operaciones de *checkpoint* debe almacenarse en el sistema de ficheros paralelo XPN.

TABLA 4.6. REQUISITO FUNCIONAL RF-05.

Identificador	RF-05
Nombre	Configuración del despliegue del almacenamiento
Prioridad	Media
Necesidad	Deseable
Estabilidad	Media
Descripción	Debe permitirse desplegar los servidores XPN en modo colocalizado y exclusivo, así como variar su número, para evaluar distintas configuraciones de almacenamiento.

4.3.2. Requisitos no funcionales

TABLA 4.7. REQUISITO NO FUNCIONAL RNF-01.

Identificador	RNF-01
Nombre	Reproducibilidad del entorno
Prioridad	Alta
Necesidad	Esencial
Estabilidad	Alta
Descripción	El entorno experimental debe empaquetarse mediante contenedores Docker y scripts, de modo que cualquier ejecución sea reproducible.

TABLA 4.8. REQUISITO NO FUNCIONAL RNF-02.

Identificador	RNF-02
Nombre	Equivalencia de resultados
Prioridad	Alta
Necesidad	Esencial
Estabilidad	Alta
Descripción	La integración de XPN no debe alterar la semántica de la simulación: los resultados epidemiológicos deben ser equivalentes a los obtenidos con el sistema de ficheros de referencia.

TABLA 4.9. REQUISITO NO FUNCIONAL RNF-03.

Identificador	RNF-03
Nombre	Portabilidad del entorno
Prioridad	Media
Necesidad	Deseable
Estabilidad	Media
Descripción	El entorno debe poder ejecutarse tanto en un equipo local de desarrollo como en el clúster Tucán de la UC3M.

TABLA 4.10. REQUISITO NO FUNCIONAL RNF-04.

Identificador	RNF-04
Nombre	Tecnologías de implementación
Prioridad	Media
Necesidad	Esencial
Estabilidad	Alta
Descripción	El sistema debe construirse sobre C y el estándar MPI (implementación MPICH), empleando PAPI para la lectura de contadores hardware.

4.4. Diseño del sistema

El sistema resultante se compone de tres subsistemas que cooperan dentro de una estructura de contenedores Docker. La Figura 4.3 presenta dichos componentes y las interfaces que los conectan. EpiGraph actúa como aplicación de cómputo y, en cada nodo virtual, se enlaza con la biblioteca de FlexMPI, que le confiere maleabilidad. Por su parte, la simulación realiza volcados periódicos de su estado que, sobre un almacenamiento centralizado, llegan a dominar el tiempo de ejecución; para mitigar ese coste se incorpora Expand Ad-Hoc (XPN), un sistema de ficheros paralelo que reparte cada volcado entre varios servidores de datos y paraleliza así la escritura (véase la Figura 3.3). Un único proceso controlador de FlexMPI supervisa la ejecución global. Los componentes se relacionan a través de dos planos. El plano de control —en naranja en la figura— conecta cada proceso con el controlador de FlexMPI: por él ascienden los vectores de coste y descienden las órdenes de redistribución. El plano de datos —en azul discontinuo— encamina la E/S de los *checkpoints* hacia los servidores de XPN, que la reparten entre sí.

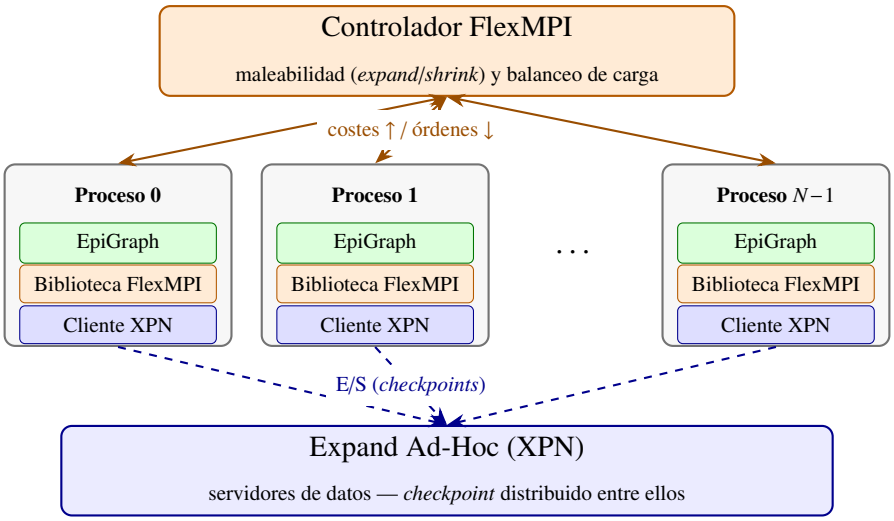


Fig. 4.3. Arquitectura de componentes del sistema y sus puntos de integración.

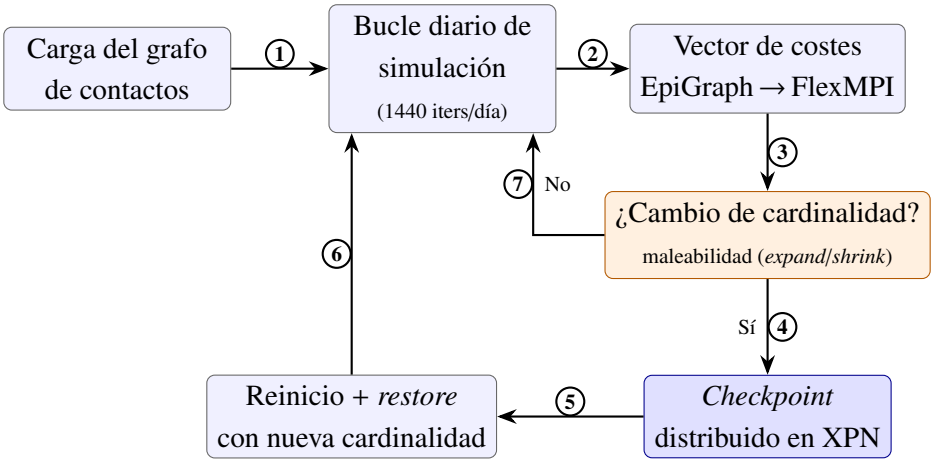


Fig. 4.4. Flujo de datos y de control durante una ejecución.

Para comprender el recorrido de la información conviene atender a la Figura 4.4, cuyas flechas numeradas siguen la secuencia que se describe a continuación. Inicialmente, cada proceso carga su partición del grafo de contactos (Flecha 1) y comienza el bucle de simulación, que avanza la epidemia en pasos de un día (Flecha 2). Al término de cada día simulado, EpiGraph entrega a FlexMPI un vector de costes que cuantifica la carga real de cada proceso (Flecha 3, véase la Figura 3.2).

Con ese vector el controlador gobierna dos mecanismos distintos. El ordinario es el balanceo de carga: cuando detecta un desequilibrio, desplaza columnas del grafo entre los procesos que ya están en ejecución, sin alterar su número ni interrumpir la simulación, y el bucle continúa con el día siguiente (Flecha 7). El excepcional es la maleabilidad, que sí modifica la cardinalidad de la aplicación ante un cambio de fase de la simulación que justifique asignarle más o menos recursos. En ese caso el estado se vuelca en forma de *checkpoint* binario que XPN distribuye entre sus servidores, evitando el cuello de botella de un sistema de ficheros único (Flecha 4); la aplicación se reinicia con el nuevo número

de procesos (Flecha 5) y éstos recuperan el estado desde XPN para retomar el bucle (Flecha 6). Uno y otro mecanismo se detallan en la Sección 4.4.2.

El conjunto de estos subsistemas se ejecuta sobre una estructura de contenedores Docker que constituye el sustrato de despliegue del sistema. Docker unifica EpiGraph, FlexMPI y Expand Ad-Hoc en un entorno reproducible y controlado, aislando las dependencias de cada componente y evitando los conflictos de compatibilidad que surgirían al integrarlos sobre un mismo sistema. Cada contenedor se comporta como un nodo independiente del clúster virtual, con su propia instancia de los procesos MPI, y se comunica con el resto a través de la red virtual de Docker. Los tres componentes se despliegan de forma colocalizada en cada nodo y se restringen mediante `cpuset` al conjunto de núcleos del procesador anfitrión sobre el que deben ejecutarse, lo que permite reproducir un clúster de varios nodos sobre una única máquina con un reparto de CPU controlado. Este despliegue, ilustrado en la Figura 3.5 de la Sección 3.2.4, es el que hace viable evaluar la integración de los tres sistemas de forma controlada y repetible.

4.4.1. EpiGraph: la aplicación maleable de cómputo

EpiGraph es la aplicación que realiza el cómputo de la simulación. La población se modela como un grafo de contactos, almacenado de forma comprimida (Figura 3.1), y se reparte entre los procesos dividiendo el conjunto de individuos en bloques contiguos de tamaño aproximadamente igual, de manera que cada proceso simula uno de esos bloques. La ejecución progresa por un bucle que avanza la epidemia en pasos de un día simulado, cada uno compuesto por un número fijo de iteraciones. El origen del desbalanceo de carga reside en este reparto: dado que el coste de cómputo de cada proceso depende del número de individuos infectados que contiene su bloque —y la infección ni se distribuye de manera uniforme entre los bloques ni permanece constante a lo largo de la simulación—, los procesos dejan de estar equilibrados de forma dinámica. Para que ese desbalanceo pueda corregirse, al término de cada día EpiGraph entrega a FlexMPI un vector de costes que cuantifica la carga efectiva de cada proceso (Figura 3.2); este vector constituye la interfaz de integración entre el cómputo y el sistema de maleabilidad.

Esta evolución de la carga de cómputo es observable en la traza de ejecución: al término de cada día simulado, cada proceso registra su tiempo de cómputo (campo `RTIME`), cuyo perfil reproduce la curva epidémica y permite cuantificar el desbalanceo entre procesos. El análisis detallado de estas métricas se aborda en el capítulo de evaluación.

Internamente, EpiGraph es un modelo basado en agentes: cada individuo es un agente que en cada instante se encuentra en un estado epidemiológico —susceptible, infectado, recuperado o fallecido— y la simulación avanza en iteraciones que representan un minuto simulado, durante las cuales la enfermedad se propaga a través de las aristas del grafo de contactos. Como se detalló en la Sección 3.2.1, la fase de propagación recorre únicamente las columnas correspondientes a los individuos infectados, de modo que el coste de cómputo es proporcional al número de infectados y a su número medio de contactos, y

sigue por tanto la curva epidémica: crece en los picos de la epidemia y decae conforme esta remite. Frente a esa carga de cómputo variable, las operaciones de escritura tienen un tamaño fijo, ya que el *checkpoint* guarda el estado de los N individuos con independencia de su nivel de infección. Esta asimetría —una carga de cómputo que varía con la dinámica de la epidemia y una carga de entrada/salida estable y proporcional al tamaño de la población— es la que justifica el diseño del sistema: adaptar dinámicamente los recursos de cómputo mediante la maleabilidad de FlexMPI y, al mismo tiempo, sostener el coste de la E/S mediante un sistema de ficheros paralelo, dos necesidades que se abordan en las secciones siguientes.

4.4.2. FlexMPI: control de maleabilidad y balanceo de carga

FlexMPI es la biblioteca de maleabilidad que dota a la aplicación de balanceo de carga y reconfiguración dinámicos. Se integra de dos formas complementarias: por un lado, una biblioteca que se enlaza con cada proceso de EpiGraph e interpone los mecanismos de monitorización y reconfiguración sobre las llamadas MPI; por otro, un proceso *controlador* independiente —ejecutado en el primer nodo del clúster— que centraliza la toma de decisiones. La biblioteca instrumenta la ejecución de cada proceso e informa al controlador del estado de su carga, y el controlador, con una visión global del sistema, decide cuándo y cómo reconfigurar la aplicación y emite las órdenes correspondientes.

Al inicio de cada día simulado la biblioteca inicializa los contadores de rendimiento de cada proceso y, al término del día, EpiGraph le entrega el vector de costes descrito en la Sección 4.4.1: una entrada por individuo de la partición local cuyo valor es el grado del nodo si está infectado y cero en caso contrario (Figura 3.2). El controlador agrega esos costes por proceso y, siguiendo una política de balanceo basada en el recuento de columnas (`lbpolicy-counts`), compara la carga de cada uno. Si detecta un desequilibrio, recalcula los límites de la partición —esto es, desplaza columnas del grafo CSC desde los procesos sobrecargados hacia los menos cargados— de modo que todos converjan a una carga de cómputo similar sin alterar el número de procesos. Este balanceo se evalúa de forma regular, una vez por día simulado.

El segundo mecanismo, la *maleabilidad* propiamente dicha, se gobierna mediante una política disparada por eventos (`MALLEABILITY_TRIG`). A diferencia del balanceo, que actúa con una cadencia fija, la reconfiguración del número de procesos se desencadena ante eventos concretos, como un cambio de fase planificado de la simulación que justifica asignar más o menos recursos de cómputo. Cuando se dispara, el controlador recibe la cardinalidad de destino y ordena la ampliación (*expand*) o la reducción (*shrink*) del conjunto de procesos; a diferencia del balanceo, ese número no lo calcula la biblioteca, sino que se le indica desde fuera. La materialización de ese cambio —que exige reconstruir la partición de los datos sobre un número distinto de procesos— se realiza mediante el ciclo de *checkpoint* y reinicio descrito en la Sección 4.4.4.

La actividad de FlexMPI también queda reflejada en la traza: el controlador registra

cada evaluación del balance y cada reconfiguración del número de procesos, mientras que el tiempo invertido en la comunicación y coordinación entre procesos se contabiliza en el campo `CTIME`. Estas métricas se examinan en el capítulo de evaluación.

4.4.3. Expand Ad-Hoc (XPN) e intercepción de la entrada/salida

Expand Ad-Hoc (XPN) es el tercer subsistema de la integración: el sistema de ficheros paralelo que se interpone entre EpiGraph y el almacenamiento para que la entrada/salida deje de limitar la simulación. Se incorpora porque en este tipo de simulaciones la E/S no es un coste accesorio del cómputo, sino una fracción sustancial del tiempo de ejecución.

La simulación vuelca periódicamente su estado, y el coste de esas escrituras crece con el tamaño de la población: cada volcado guarda el estado de todos los individuos, de modo que su volumen es proporcional al número de agentes simulados y, a escala de millones de individuos, alcanza cientos de megabytes por día simulado. Sobre un sistema de ficheros centralizado, estas escrituras se serializan y llegan a dominar el tiempo de ejecución. XPN mitiga ese coste desplegando un sistema de ficheros paralelo y distribuido de tipo *ad hoc*: cada fichero se divide en bloques de tamaño fijo —configurable mediante el parámetro *bsize*— que se reparten entre varios servidores de datos siguiendo una política *round-robin* (el bloque k se asigna al servidor k mód N_{serv}, como ilustra la Figura 3.3). De esta forma la escritura se realiza en paralelo sobre todos los servidores y el ancho de banda agregado crece con su número, en lugar de quedar limitado por un único punto de almacenamiento.

La integración con EpiGraph es transparente y no exige modificar su código fuente. Mediante la precarga de una biblioteca (`LD_PRELOAD`), las llamadas de entrada/salida que la aplicación dirige al sistema de ficheros se interceptan y se redirigen hacia XPN. Así, el *checkpoint* binario que EpiGraph escribe al final de cada día pasa a almacenarse de forma distribuida sobre los servidores de XPN en lugar de sobre el sistema de ficheros del nodo anfitrión, sin que la aplicación sea consciente de ello.

El despliegue de los servidores de datos admite dos configuraciones de diseño que se evalúan más adelante. En la configuración *colocalizada*, cada nodo virtual ejecuta a la vez el proceso de cómputo y un servidor de XPN, lo que aproxima los datos al proceso que los genera y aprovecha el almacenamiento local de cada nodo. En la configuración *exclusiva*, un subconjunto de nodos se dedica únicamente al almacenamiento, liberando de esa carga a los nodos de cómputo a costa de transferir los datos por la red. La elección entre ambas determina el equilibrio entre la localidad de los datos y el aislamiento de la carga de entrada/salida respecto a la de cómputo.

El rendimiento de la E/S a través de XPN queda registrado en la traza de ejecución mediante las líneas `IOPerf`, que recogen para cada *checkpoint* su latencia (campo `dt`) y el ancho de banda alcanzado, expresado en MB/s; estas métricas, junto con el tiempo total de ejecución (`Spreading time`), se analizan en el capítulo de evaluación.

4.4.4. Ciclo de maleabilidad: *checkpoint* y reinicio

Cuando la reconfiguración decidida por FlexMPI implica un cambio en el número de procesos, no basta con incorporar o retirar procesos sobre la marcha: el estado de la simulación está distribuido según la partición vigente —cada proceso mantiene el estado del bloque de individuos que le corresponde—, de modo que variar el número de procesos obliga a reconstruir por completo esa partición. El diseño resuelve esta necesidad reutilizando el mecanismo de *checkpoint* que EpiGraph ya emplea para tolerancia a fallos: en lugar de redistribuir el estado en caliente, la maleabilidad se materializa mediante un ciclo de volcado y reinicio.

El ciclo se desarrolla en cuatro pasos, que la Figura 4.5 sitúa sobre la línea temporal de la ejecución. Al alcanzar el día simulado en que debe producirse el cambio, la aplicación vuelca su estado completo en un *checkpoint* binario y finaliza (*save* y *exit*). A continuación se relanza con el nuevo número de procesos (*restart*), que reparten de nuevo el conjunto de individuos —en tantos bloques contiguos como procesos haya ahora— y reconstruyen su estado local a partir del *checkpoint* (*restore*), prosiguiendo la simulación desde el día guardado. El guardado y la recuperación se gobiernan mediante variables de entorno que indican a la aplicación el día en el que debe volcar su estado y el día desde el que debe restaurarlo.

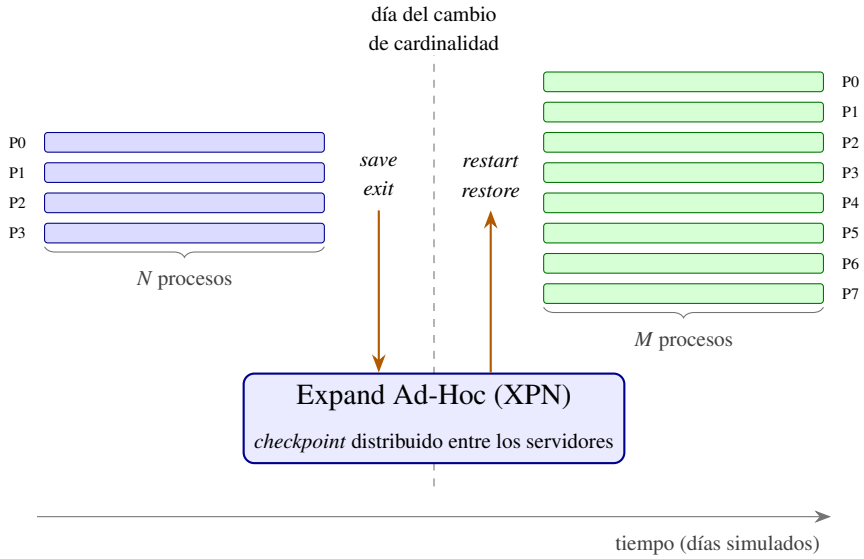


Fig. 4.5. Ciclo de maleabilidad por *checkpoint* y reinicio.

Un rasgo central del diseño es que desacopla los dos tipos de recurso que la aplicación consume. Los recursos de cómputo —el número de procesos de EpiGraph— se ajustan en tiempo de ejecución mediante la maleabilidad de FlexMPI a través del ciclo descrito. Los recursos de entrada/salida —el número y la disposición de los servidores de XPN— constituyen una dimensión de configuración independiente, que XPN permite expandir o reducir por sí misma. De este modo, cada recurso puede dimensionarse con arreglo a la fase de la simulación que lo demanda —más cómputo en los picos de la curva epidémi-

ca, una E/S sostenida durante todo el volcado periódico de estado— sin que el ajuste de uno condicione al otro. Esta independencia entre la gestión del cómputo y la de la E/S es la propiedad que se explora de forma sistemática en la evaluación, contrastando distintas configuraciones de procesos frente a distintas configuraciones de servidores de XPN (colocalizada y exclusiva, con número variable de servidores).

Este ciclo es el punto en el que confluyen los tres subsistemas (Flechas 4–6 de la Figura 4.4): FlexMPI ejecuta el cambio de cardinalidad y reconstruye el reparto del dominio, XPN absorbe y distribuye la entrada/salida del volcado y de la posterior recuperación del estado, y EpiGraph reanuda el cómputo con la configuración resultante. La biblioteca decide por sí sola el rebalanceo de la carga entre los procesos existentes, para el que le basta el vector de costes; en cambio, cuántos procesos debe tener la aplicación tras el cambio y cuántos servidores de datos la atienden son decisiones externas a ella, que en este trabajo fija el diseño de cada experimento.

Como contrapartida, el ciclo de *checkpoint* y reinicio introduce un sobrecoste —el del volcado, el relanzamiento y la recuperación del estado— que la nueva configuración debe compensar para que la reconfiguración resulte rentable. En la traza de ejecución, cada ciclo queda señalado por las marcas de volcado y restauración del estado; tanto ese sobrecoste como el coste de E/S asociado se cuantifican en el capítulo de evaluación.

4.5. Presupuesto

En esta sección se analizan los costes asociados a la elaboración del proyecto, desglosados en costes directos e indirectos. Conviene destacar que el clúster Tucán de la UC3M, empleado en la fase de experimentación, constituye infraestructura cedida por la universidad y no supone un coste directo para el proyecto, y que la totalidad del software utilizado (EpiGraph, FlexMPI, Expand Ad-Hoc, Docker, Visual Studio Code y la distribución TEX) es de código abierto o uso gratuito.

4.5.1. Costes directos

Los costes de personal se calculan en la Tabla 4.11. Dado que el proyecto fue desarrollado por una única persona bajo la supervisión de la tutora, a efectos de presupuesto las tareas se agrupan en los roles profesionales que intervendrían en un desarrollo equivalente, en correspondencia directa con las fases de la metodología descritas en la Sección 4.1 y reflejadas en el diagrama de Gantt de la Figura 4.2. El rol de *analista/investigador* asume las fases de documentación, análisis de requisitos y diseño del sistema, en las que se estudió el estado de la cuestión y se delimitó el alcance del trabajo. El rol de *ingeniero de software* concentra la mayor dedicación y abarca las fases de preparación del entorno, integración de EpiGraph, FlexMPI y Expand Ad-Hoc, y experimentación, junto con la redacción técnica asociada. Por encima de ambos, el rol de *director de proyecto* cubre

las tareas transversales de planificación, seguimiento y supervisión a lo largo de toda la duración del trabajo. La dedicación en meses de cada rol se ha estimado a partir del peso relativo de sus fases en la planificación, y para el salario bruto se toma como referencia el valor medio publicado en Glassdoor [37], sobre el que se aplica un 23,5 % adicional en concepto de cuota patronal a la Seguridad Social [38].

TABLA 4.11. COSTES DE PERSONAL.

Rol	**N.º**	**Salario bruto**	**Meses**	**S. Social (23,5 %)**	**Coste total**
Director de proyecto	1	3.500,00 €	1,0	822,50 €	4.322,50 €
Analista / Investigador	1	2.800,00 €	1,5	987,00 €	5.187,00 €
Ingeniero de software	1	2.357,00 €	9,0	4.985,06 €	26.198,06 €
Total					**35.707,56 €**

Los costes de hardware se recogen en la Tabla 4.12. Dado que el equipo de desarrollo se reutiliza más allá de la duración del proyecto, en la Tabla 4.13 se calcula su amortización para los nueve meses de duración del trabajo.

TABLA 4.12. COSTES DE HARDWARE.

Producto	**Unidades**	**Precio sin IVA**	**Coste total**
Equipo de desarrollo (portátil)	1	1.200,00 €	1.200,00 €
Clúster Tucán (cedido por la UC3M)	—	—	0,00 €
Total			**1.200,00 €**

TABLA 4.13. AMORTIZACIÓN DEL HARDWARE.

Coste adquisición	**Periodo amort. (años)**	**Cuota mensual**	**Duración (meses)**	**Amortización**
1.200,00 €	5	20,00 €/mes	9	180,00 €

4.5.2. Costes indirectos

En la Tabla 4.14 se recogen los costes asociados al consumo eléctrico del equipo de desarrollo y a la conexión a Internet. Las licencias de software son nulas, ya que todas las herramientas empleadas son de código abierto o de uso académico gratuito.

TABLA 4.14. COSTES INDIRECTOS.

Concepto	**Consumo**	**Duración**	**Precio sin IVA**	**Coste total**
Energía equipo de desarrollo	65 W	1150 h	0,15 €/kWh	11,21 €
Conexión a Internet	—	9 meses	30,00 €/mes	270,00 €
Licencias de software (código abierto y uso académico)	—	—	0,00 €	0,00 €
Total				**281,21 €**

4.5.3. Costes totales

Sumando los costes de personal, la amortización del hardware y los costes indirectos se obtiene el coste base del proyecto. En el supuesto de que el proyecto se comercializara, se aplican un margen de beneficio del 10 % y un margen de riesgo del ±15 %, así como el 21 % de IVA. El resultado se resume en la Tabla 4.15.

TABLA 4.15. COSTES TOTALES.

Concepto	**Coste**
Personal	35.707,56 €
Amortización de hardware	180,00 €
Costes indirectos	281,21 €
Total sin IVA	**36.168,77 €**
Total sin IVA (beneficio 10 % y riesgo +15 %)	45.753,49 €
Total sin IVA (beneficio 10 % y riesgo −15 %)	33.817,80 €
Total con IVA (beneficio 10 % y riesgo +15 %)	55.361,72 €
Total con IVA (beneficio 10 % y riesgo −15 %)	40.919,54 €

5. EVALUACIÓN

5.1. Descripción de la plataforma de evaluación

La evaluación de la integración de EpiGraph, FlexMPI y Expand Ad-Hoc se ha llevado a cabo sobre dos plataformas de cómputo complementarias. La primera es un equipo local de desarrollo, empleado durante las fases iniciales para la puesta en marcha, la depuración y la validación funcional del sistema, donde la capacidad de iterar con rapidez prima sobre la escala del problema. La segunda es el clúster de altas prestaciones Tucán de la Universidad Carlos III de Madrid, sobre el que se han ejecutado las campañas experimentales de mayor tamaño —en particular las que emplean la población completa de la ciudad de Madrid, del orden de tres millones y medio de individuos— y los escenarios genuinamente multinodo. Ambos entornos comparten exactamente la misma pila de software, ya que la ejecución se produce en todos los casos sobre un clúster virtual de contenedores Docker (Sección 3.2.4); esta uniformidad es la que garantiza que los resultados obtenidos en el equipo local y en el clúster sean directamente comparables y responde al requisito de reproducibilidad del entorno. En las subsecciones siguientes se describen, en primer lugar, las dos infraestructuras físicas; a continuación, el clúster virtual que se despliega sobre ellas y que constituye la plataforma de ejecución real; y, por último, la pila de software y las herramientas de observación empleadas para la toma de medidas.

5.1.1. Equipo local de desarrollo

El equipo local es un ordenador portátil equipado con un procesador Intel Core i5-12450H de duodécima generación. Se trata de una arquitectura híbrida, en la que conviven dos tipos de núcleos físicos: cuatro núcleos de rendimiento (*performance cores*), con ejecución simultánea de dos hilos por núcleo y una frecuencia máxima de 4,4 GHz, y cuatro núcleos de eficiencia (*efficiency cores*), de un único hilo y una frecuencia máxima de 3,3 GHz. En total, el procesador ofrece ocho núcleos físicos y doce hilos lógicos sobre un único nodo NUMA. Esta heterogeneidad tiene implicaciones directas sobre el trabajo: por un lado, obliga a anclar los procesos de la simulación a núcleos concretos para evitar que el planificador del sistema operativo los reubique entre núcleos de distinta potencia y distorsione las medidas; por otro, es también relevante para la instrumentación con contadores *hardware* del procesador, cuestión que se discute en la Sección 5.2. Las características completas del equipo se recogen en la Tabla 5.1.

TABLA 5.1. CARACTERÍSTICAS DEL EQUIPO LOCAL DE DESARROLLO.

Componente	**Especificación**
Procesador	Intel Core i5-12450H (12.ª generación, arquitectura híbrida)
Núcleos	8 físicos (4 de rendimiento + 4 de eficiencia), 12 hilos lógicos
Frecuencia	Hasta 4,4 GHz (rendimiento) / 3,3 GHz (eficiencia)
Caché	L1 704 KiB, L2 7 MiB, L3 12 MiB
Memoria	16 GiB
Almacenamiento	SSD NVMe de 512 GB
Nodos NUMA	1
Sistema operativo	Fedora Linux 40, núcleo 6.14

5.1.2. Clúster Tucán de la UC3M

El clúster Tucán es una infraestructura de cómputo de altas prestaciones cedida por la Universidad Carlos III de Madrid. Está organizado en torno a un nodo frontal (*frontend*), desde el que los usuarios acceden al sistema y envían los trabajos, y un conjunto de nodos de cómputo interconectados por red, cuyo uso se gestiona mediante el planificador de recursos SLURM. El acceso se realiza a través de un nodo pasarela (*jump host*) que da entrada a la red interna del laboratorio. El sistema de ficheros de usuario está compartido por red entre todos los nodos, de modo que el código y los datos son visibles de forma homogénea desde cualquiera de ellos. Los experimentos descritos en este trabajo se han ejecutado sobre los nodos de cómputo del grupo `compute-15`, que son máquinas de un elevado número de núcleos —cuarenta núcleos físicos, ochenta hilos lógicos, por nodo—, lo que permite ejecutar en un único nodo físico configuraciones con decenas de procesos sin recurrir a la sobresuscripción. Se ha empleado uno de estos nodos en las pruebas de un solo nodo físico y dos de ellos, interconectados mediante una red *overlay*, en las pruebas multinodo en las que los procesos de EpiGraph y los servidores de Expand Ad-Hoc residen en máquinas físicas distintas. Las características de estos nodos de cómputo se resumen en la Tabla 5.2.

TABLA 5.2. CARACTERÍSTICAS DE UN NODO DE CÓMPUTO DEL CLÚSTER TUCÁN.

Componente	**Especificación**
Procesador	2 × Intel Xeon Silver 4316 (Ice Lake-SP)
Núcleos	40 físicos (20 por zócalo), 80 hilos lógicos
Frecuencia	2,3 GHz nominal, hasta 3,4 GHz
Caché	L1 48 KiB + 32 KiB por núcleo, L2 1,3 MiB por núcleo, L3 30 MiB por zócalo
Memoria	128 GB
Nodos NUMA	2 (uno por zócalo)
Red de interconexión	Ethernet de 10 Gbit s^{-1}
Almacenamiento compartido	Sistema de ficheros de red (NFS) sobre el directorio de usuario
Planificador	SLURM
Nodos empleados	Grupo `compute-15` (1 nodo en pruebas de un solo nodo; 2 nodos en pruebas multinodo)
Sistema operativo	Ubuntu 24.04.2 LTS, núcleo 6.8

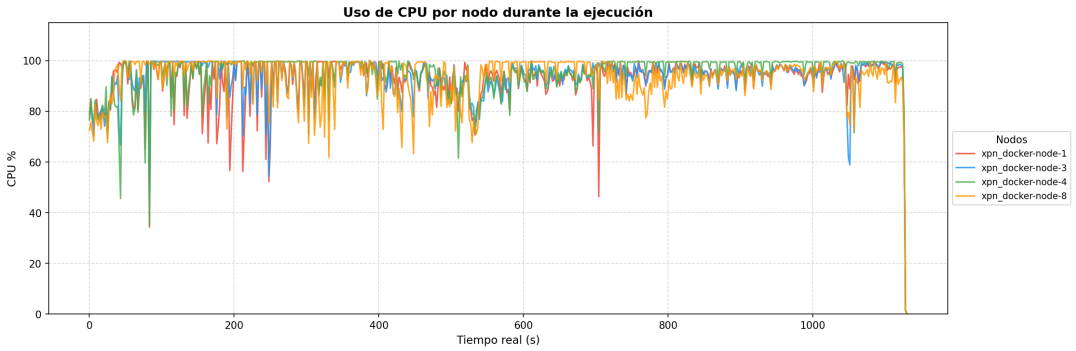
Conviene señalar que, sobre esta infraestructura, el usuario no dispone de privilegios de administración, lo que condiciona algunas decisiones de la puesta en marcha —como el despliegue de los contenedores sin permisos de superusuario— que se detallan en la Sección 5.2. Además, los nodos de cómputo entran en modo de ahorro de energía cuando no tienen carga asignada, por lo que su disponibilidad depende de la reserva previa a través del planificador.

5.1.3. Clúster virtual de contenedores

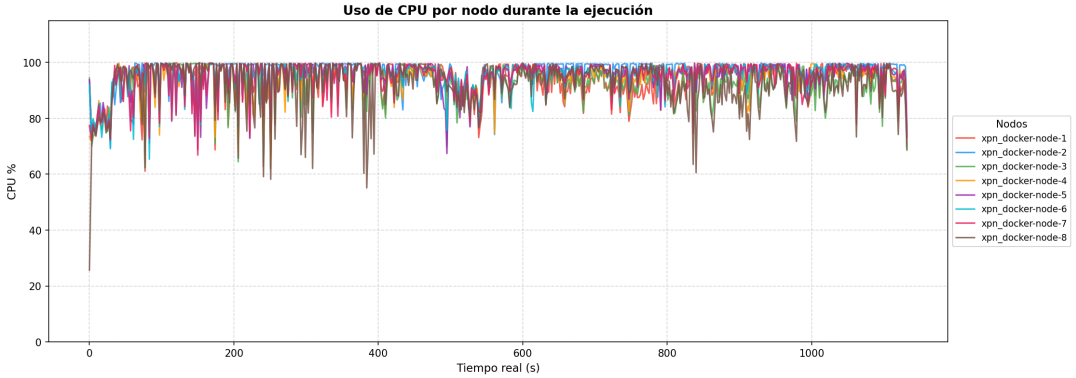
Con independencia de la infraestructura física subyacente, la plataforma sobre la que realmente se ejecutan los experimentos es un clúster virtual construido con contenedores Docker, según el diseño de despliegue introducido en la Sección 4.4. Cada contenedor desempeña el papel de un nodo del clúster y ejecuta uno o varios procesos MPI de la simulación; el conjunto de contenedores conforma así un clúster de tamaño configurable sobre una única máquina física, lo que permite reproducir de forma controlada distintas topologías sin necesidad de disponer de tantas máquinas reales como procesos se deseen evaluar. Todos los contenedores parten de una misma imagen basada en Ubuntu 24.04, que incorpora la pila de software descrita en la Sección 5.1.4, y comparten un volumen común que hace las veces de directorio de trabajo para el código, los guiones de lanzamiento y los registros de ejecución.

Para que las medidas de rendimiento sean fiables, el conjunto de CPU sobre el que puede ejecutarse cada contenedor se fija de antemano (`cpuset`), tal y como ilustra la Figura 3.5. Con ello se impide que el planificador del sistema operativo desplace los procesos entre núcleos de distinta potencia durante la ejecución, lo que introduciría ruido en los tiempos de cómputo y en los contadores *hardware*. En el equipo local ese conjunto son los cuatro núcleos de rendimiento del procesador híbrido descrito en la Sección 5.1.1, de manera que la comparación entre configuraciones no se ve afectada por la asimetría entre núcleos de rendimiento y de eficiencia; el reparto es de un contenedor por núcleo mientras no se rebasan los cuatro, y por encima de esa cifra dos contenedores comparten núcleo, con las consecuencias que se describen a continuación. En el clúster, cuyos nodos ofrecen cuarenta núcleos físicos, todas las configuraciones ensayadas caben con un núcleo por contenedor.

Esta característica del equipo local explica, además, un resultado observado durante la validación: las ejecuciones con cuatro y con ocho procesos arrojaban tiempos prácticamente idénticos —1131,7 s con cuatro procesos y 1131,8 s con ocho, medidos como *spreading time* o tiempo total de simulación, esto es, una diferencia inferior a dos décimas de segundo— (Figura 5.1). Al disponer el procesador de cuatro núcleos de rendimiento con ejecución simultánea de dos hilos por núcleo (*Hyperthreading*), con cuatro procesos cada uno dispone en exclusiva de un núcleo físico, mientras que con ocho procesos dos de ellos comparten cada núcleo y compiten por sus cachés de primer y segundo nivel y por el ancho de banda de memoria. Dado que la simulación de EpiGraph está limitada por el ancho de banda de memoria —en cada iteración recorre vectores de estado de cientos de miles de individuos—, la ejecución simultánea de hilos apenas aporta ganancia, por lo que ocho procesos no mejoran de forma apreciable el rendimiento de cuatro. Este comportamiento se tuvo en cuenta al interpretar las medidas obtenidas en el equipo local y refuerza la conveniencia de ejecutar las campañas de mayor tamaño sobre el clúster Tucán, cuyos nodos ofrecen un número de núcleos físicos muy superior.



(a) Cuatro procesos



(b) Ocho procesos

Fig. 5.1. Uso de CPU de cada contenedor en el equipo local, con XPN colocalizado.

Sobre este clúster virtual se han empleado cuatro disposiciones distintas de los contenedores, que se recogen en la Figura 5.2 y a las que se hará referencia a lo largo de todo el capítulo. Lo que las diferencia es dónde se sitúan los servidores de almacenamiento de Expand Ad-Hoc respecto de los procesos de cómputo de EpiGraph, y hasta qué punto esa separación es real o meramente lógica. En todas ellas el número de contenedores de cada tipo es configurable, lo que permite barrer distintas escalas sin alterar la topología.

Las dos primeras se despliegan sobre un único nodo físico y se comunican por la red virtual de Docker: en la colocalizada cada contenedor reúne cómputo y almacenamiento, mientras que en la exclusiva los contenedores se especializan, si bien la separación sigue siendo solo lógica. Las dos restantes reparten los contenedores entre dos nodos físicos del clúster Tucán unidos por una red superpuesta: en la separada por función cada nodo alberga un único tipo de contenedor, de modo que toda la entrada/salida atraviesa la red; en la distribuida ambos tipos se reparten a partes iguales entre los dos nodos, lo que recupera parte de la localidad.

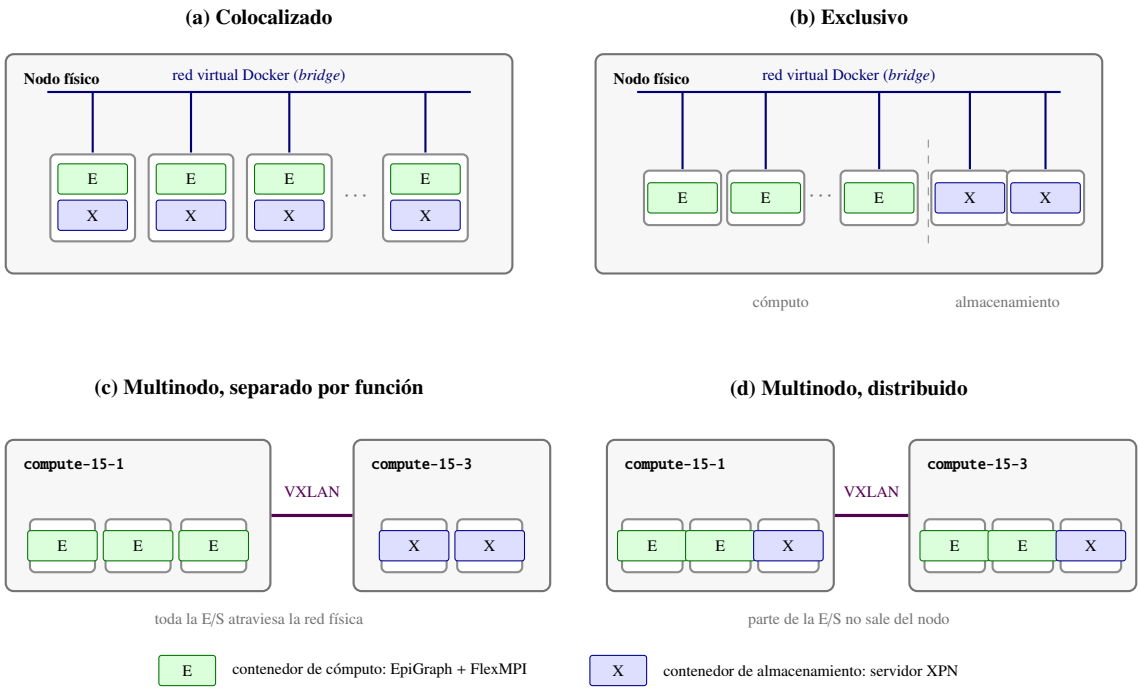


Fig. 5.2. Las cuatro topologías de despliegue empleadas en la evaluación. Las dos superiores ocupan un solo nodo físico; las dos inferiores, dos nodos de Tucán.

Las dos primeras se despliegan sobre un único nodo físico y emplean la red virtual de Docker en modo puente. En la configuración *colocalizada* cada contenedor ejecuta a la vez un proceso de cómputo y un servidor de almacenamiento, de modo que una parte de las escrituras de cada proceso se resuelve sin salir del contenedor. En la configuración *exclusiva*, en cambio, los contenedores se especializan: los primeros ejecutan únicamente la aplicación y los últimos únicamente servidores de almacenamiento. Conviene subrayar que en esta segunda configuración la separación es solo lógica, pues todos los contenedores siguen residiendo en la misma máquina y el tráfico de entrada/salida no llega a atravesar una red física. Esta limitación es precisamente la que motiva las dos configuraciones siguientes.

Las dos restantes reparten los contenedores entre dos nodos físicos del clúster Tucán —`compute-15-1` y `compute-15-3`—, comunicados mediante una red superpuesta de Docker en modo *swarm*. Recurrir a dos máquinas no responde únicamente a la necesidad de que el tráfico de entrada/salida atraviese una red real: duplica también los recursos de cómputo disponibles. Dado que el anclaje de cada contenedor a un núcleo distinto limita el número de procesos que pueden ejecutarse sin que compitan entre sí, disponer de dos nodos es lo que hace viables las configuraciones de mayor tamaño y, con ellas, reducciones sustanciales del tiempo de ejecución en las fases dominadas por el cómputo. Este efecto, y el límite a partir del cual deja de compensar añadir procesos, se analizan en la Sección 5.5.

En la disposición *separada por función*, todos los procesos de cómputo se ubican en un nodo y todos los servidores de almacenamiento en el otro, con lo que la totalidad del

tráfico de entrada/salida atraviesa la red del clúster y no la interfaz de bucle local; es la topología que aísla por completo el almacenamiento del cómputo. En la disposición *distribuida*, por último, ambos tipos de contenedor se reparten a partes iguales entre los dos nodos, de forma determinista y con el fichero de máquinas intercalado para que los procesos de la aplicación queden equilibrados entre ambos. Con ello se recupera parte de la localidad que se pierde en la disposición anterior —una fracción de las escrituras encuentra un servidor en su mismo nodo físico— sin renunciar a emplear los recursos de las dos máquinas; es la topología utilizada en las campañas de mayor escala.

5.1.4. Pila de software y herramientas de observación

La pila de software es idéntica en las dos infraestructuras, al residir íntegramente dentro de la imagen de los contenedores. La comunicación entre procesos se apoya en la implementación MPICH en su versión 4.3.1, compilada con el dispositivo de comunicaciones CH4 y configurada para emplear el proveedor de red basado en *sockets*, de modo que el transporte MPI opera sobre TCP/IP tanto dentro de un mismo nodo como entre nodos del clúster virtual. Sobre esta base se enlazan la biblioteca de EpiGraph, la biblioteca de maleabilidad y balanceo de FlexMPI y el cliente de Expand Ad-Hoc, este último precargado mediante `LD_PRELOAD` para interceptar de forma transparente las llamadas de entrada/salida de la aplicación. La instrumentación del rendimiento se realiza con la biblioteca PAPI en su versión 7.3.0, que da acceso a los contadores *hardware* del procesador; su elección frente a la versión instalada en el sistema obedece a un problema de compatibilidad que se detalla en la Sección 5.2. La Tabla 5.3 recoge los componentes principales y sus versiones.

TABLA 5.3. COMPONENTES PRINCIPALES DE LA PILA DE SOFTWARE.

Componente	**Versión**	**Función**
Imagen base	Ubuntu 24.04	Sistema operativo de los contenedores
Docker	*Engine* / *Swarm*	Virtualización ligera y clúster multinodo
MPICH	4.3.1 (CH4, *sockets*)	Biblioteca de paso de mensajes
PAPI	7.3.0	Lectura de contadores *hardware*
GLPK	de sistema	Resolución del problema de balanceo en FlexMPI
libxml2	de sistema	Análisis de los ficheros de configuración de EpiGraph
GSL	de sistema	Rutinas numéricas de la simulación
EpiGraph	modificada	Aplicación de simulación epidemiológica
FlexMPI	modificada	Maleabilidad y balanceo dinámico de carga
Expand Ad-Hoc	modificada	Sistema de ficheros paralelo *ad hoc*

Para la toma de medidas se han empleado, además, las propias trazas de ejecución que generan EpiGraph y FlexMPI —de las que se extraen los tiempos de cómputo, comunicación y entrada/salida, así como el número de operaciones en coma flotante— y un guión de monitorización que, a partir de la información de `docker stats`, registra periódicamente el uso de CPU y de entrada/salida de cada contenedor durante la ejecución. El significado exacto de cada métrica se describe en la subsección siguiente, y el uso que se hace de cada una, en la Sección 5.4, junto con los casos de estudio en los que interviene.

5.1.5. Métricas de la traza de ejecución

Todas las medidas de este capítulo proceden de la traza que EpiGraph y FlexMPI escriben durante la propia ejecución, sin instrumentación añadida. Esa traza contiene dos tipos de registro que se alternan a lo largo de la simulación, y que la Figura 5.3 reproduce tal y como aparecen en un fichero de *log*. La aplicación emite una línea `IOPerf` cada vez que vuelca su estado, y la biblioteca de FlexMPI emite una línea `Iter` al cerrar cada intervalo de monitorización.

El fragmento corresponde al segundo día de una ejecución colocalizada con dieciséis procesos y la población de 3 511 110 individuos, configurada para volcar su estado cada

```
 IOTimeStamp 11.656285 NumRanks: 16 s: IOPerf 1584 1 3511110 0.140658 0 6589.98 MB/s
 IOTimeStamp 12.641563 NumRanks: 16 s: IOPerf 1728 1 3511110 0.110338 0 8400.83 MB/s
 [...]
 IOTimeStamp 18.529508 NumRanks: 16 s: IOPerf 2736 1 3511110 0.146879 0 6310.84 MB/s
 IOTimeStamp 19.399040 NumRanks: 16 s: IOPerf 2880 1 3511110 0.153322 0 6045.66 MB/s
Iter: 2880 FLOPs: 3351889 MFLOPS:: 0.006151 RTIME:: 544.943711
           PTIME:: 389.582589 CTIME:: 5.846006 IOTime:: 0.000000 Size: 16
```

Fig. 5.3. Fragmento de traza de un día simulado con dieciséis procesos.

144 iteraciones. Las diez líneas `IOPerf` de ese día —de las que la figura recoge cuatro, con la última partida en dos para que quepa en el margen— cubren las iteraciones 1584 a 2880, y la línea `Iter` que las cierra resume el intervalo de monitorización que abarca esas mismas iteraciones. La Tabla 5.4 describe los campos del primer registro y la Tabla 5.5 los del segundo.

TABLA 5.4. CAMPOS DE LA LÍNEA `IOPERF`, QUE EPIGRAPH EMITE EN CADA VOLCADO.

Campo	**Ejemplo**	**Significado**
`IOTimeStamp`	19.399040	Instante en que termina el volcado, en segundos desde el inicio de la simulación
`NumRanks`	16	Procesos de cómputo activos en ese instante
Iteración	2880	Iteración de la simulación en que se produce el volcado
Modo	1	Tipo de volcado: 1 para la escritura periódica de estado
Población	3511110	Número total de individuos simulados
`dt`	0.153322	Latencia del volcado, en segundos. Es el observable de entrada/salida de todo el capítulo
Reservado	0	Campo sin uso
Ancho de banda	6045.66	Volumen escrito dividido entre `dt`, en MB/s

TABLA 5.5. CAMPOS DE LA LÍNEA `ITER`, QUE FLEXMPI EMITE AL CERRAR CADA INTERVALO DE MONITORIZACIÓN.

Campo	**Ejemplo**	**Significado**
`Iter`	2880	Iteración acumulada en que se cierra el intervalo
`FLOPs`	3351889	Operaciones en coma flotante ejecutadas en el intervalo, leídas de los contadores *hardware* mediante PAPI
`MFLOPS`	0.006151	Rendimiento medio del intervalo, obtenido de los dos campos anteriores
`RTIME`	544.943711	Tiempo transcurrido en el intervalo descontada la comunicación, en segundos. Se emplea como estimador del tiempo de cómputo
`PTIME`	389.582589	Tiempo de procesador consumido por el proceso, en segundos
`CTIME`	5.846006	Tiempo empleado en operaciones colectivas de MPI, en segundos
`IOTime`	0.000000	Tiempo de entrada/salida que contabiliza FlexMPI
`Size`	16	Procesos MPI activos, que varía cuando la ejecución es maleable

Dos particularidades de estos registros condicionan su uso. La primera es que `IOTime` vale cero en todas las ejecuciones con XPN: FlexMPI contabiliza el tiempo de las escrituras colectivas de MPI-IO, y el cliente de XPN intercepta esas llamadas antes de que lleguen a su instrumentación, de modo que la latencia real de cada volcado hay que leerla del campo `dt` de las líneas `IOPerf`. La segunda es que los campos que dependen de los contadores *hardware* son sensibles al entorno de contenedores, que restringe el acceso a la unidad de monitorización del procesador; la Sección 5.2 describe el problema y la configuración con la que se resolvió. Por ese motivo el tiempo total de una ejecución no se toma de esos campos, sino de la diferencia entre marcas `IOTimeStamp` consecutivas, que procede del reloj de MPI y es independiente de PAPI.

5.2. Integración y puesta en marcha del sistema

Antes de abordar las campañas experimentales, fue necesario integrar y poner en funcionamiento los tres subsistemas —EpiGraph, FlexMPI y Expand Ad-Hoc— sobre el entorno de desarrollo. En la primera fase del proyecto, dicho entorno es la propia máquina de trabajo, sobre la que Docker simula un clúster de varios nodos (Sección 3.2.4); la

vía canónica de uso de XPN es la precarga de biblioteca mediante `LD_PRELOAD`, como establece el requisito RF-01. La integración no fue un paso único, sino un proceso incremental en el que cada capa se validó antes de añadir la siguiente: primero la ejecución local de la aplicación de cómputo, después su despliegue sobre el clúster virtual de contenedores y, por último, la incorporación del sistema de ficheros paralelo y del mecanismo de maleabilidad. Esta sección describe, en orden de aparición, los procedimientos seguidos y los problemas resueltos en cada uno de esos hitos sobre el equipo local; la réplica del entorno en el clúster Tucán se aborda más adelante. La Tabla 5.6 recoge de forma sintética las principales incidencias encontradas, su causa y la solución aplicada.

5.2.1. Ejecución local de EpiGraph y FlexMPI

El primer objetivo fue lograr que EpiGraph, enlazado con la biblioteca de FlexMPI, compilase y ejecutase correctamente sobre la máquina de desarrollo sin la complejidad adicional de la contenerización ni del sistema de ficheros paralelo. La puesta a punto de la pila de MPI reveló las primeras incompatibilidades: la versión moderna de MPICH, basada en el motor CH4, ignoraba la variable heredada con la que se seleccionaba el proveedor de red, por lo que fue preciso fijar explícitamente el transporte (`FI_PROVIDER=tcp`) y emplear una compilación de MPICH sobre el dispositivo de sockets. A continuación se corrigieron los ficheros de compilación de ambos proyectos, cuyas rutas de cabeceras y bibliotecas (PAPI, GLPK, `libxml2` y GSL) no se correspondían con las del sistema. Superada la compilación, la primera ejecución requirió depurar con `gdb` varios fallos de inicialización hasta conseguir que la simulación avanzara con cuatro procesos en la máquina local.

Con la aplicación ya funcional se estableció la infraestructura de observación que se utilizaría durante toda la evaluación: una convención de nombres para los registros que codifica el número de nodos, procesos y fases de cada ejecución, y el procesado posterior de las trazas. Para esto último se partió de unos guiones en Python ya disponibles, que hubo que adaptar a la nueva convención de nombres y a la comprobación del número de procesos, y se desarrolló un guion de visualización propio que genera las gráficas de tiempo de cómputo por iteración, tiempo acumulado y uso de CPU por nodo. Esta ejecución local, en la que todos los procesos operaban de forma equilibrada, sirvió además como referencia de correcto funcionamiento frente a la que contrastar el comportamiento posterior en el entorno distribuido.

5.2.2. Despliegue sobre el clúster virtual de contenedores

El segundo hito consistió en trasladar la ejecución al clúster virtual de contenedores Docker, sustrato sobre el que se realizan finalmente todos los experimentos. Tras construir la imagen y arrancar los contenedores, el binario compilado fuera del contenedor no encontraba sus dependencias dinámicas, por lo que fue necesario recompilar EpiGraph y

FlexMPI dentro de la imagen —resolviendo de paso la dependencia con una versión de PAPI ausente en el contenedor— y ajustar las rutas de búsqueda de bibliotecas para que se localizasen tanto la biblioteca de FlexMPI como la instalación de MPICH empleada para compilar. También hubo que reconciliar la herramienta de lanzamiento (`mpiexec`) con esa misma instalación, ya que la del sistema no disponía del proceso auxiliar de arranque correspondiente.

El obstáculo más específico de la contenerización fue la resolución de nombres. Los contenedores exponían identificadores de host generados aleatoriamente y no únicos desde el punto de vista del lanzador, de modo que `mpiexec` interpretaba que todos los nodos eran el mismo y acumulaba la totalidad de los procesos en un único contenedor. La solución pasó por construir explícitamente un mapeo entre direcciones IP y nombres de host únicos por nodo, y a partir de él un fichero de máquinas con una única ranura por contenedor, garantizando así que cada proceso MPI se ubicara en un nodo distinto. A ello se añadió la configuración de acceso por SSH sin contraseña entre contenedores —exigido por el lanzador para distribuir los procesos— y la depuración, mediante `gdb` y `strace`, de varios bloqueos en el arranque en los que los procesos quedaban a la espera en la capa de sockets de MPI con la CPU inactiva, atribuibles a conflictos de puerto y al orden de inicialización de los servicios.

El desbalanceo de la carga de CPU fue el obstáculo principal de esta fase: las ejecuciones sobre contenedores resultaban notablemente más largas de lo esperado. Para caracterizarlo se desarrolló un guion que registra en tiempo real el uso de CPU de cada contenedor, lo que permitió constatar que prácticamente toda la carga de cómputo recaía sobre un único contenedor —el que aloja al proceso maestro, `xpn_docker-node-3`— mientras el resto permanecía ocioso (Figura 5.4). Este hallazgo motivó el análisis de sus causas y la intervención sobre el mecanismo de balanceo, que se detallan en la Sección 5.2.3.

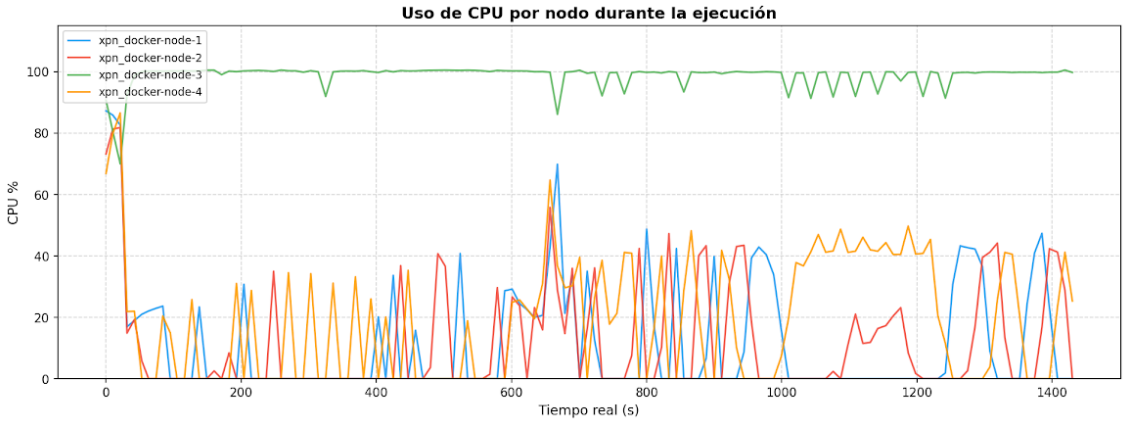


Fig. 5.4. Uso de CPU de cada contenedor durante una ejecución estática sobre el clúster virtual.

5.2.3. Integración de Expand Ad-Hoc y de la maleabilidad

Con la aplicación ejecutándose de forma distribuida se acometió la integración de los dos elementos que dan sentido al trabajo: el sistema de ficheros paralelo Expand Ad-Hoc y el mecanismo de maleabilidad de FlexMPI. Se puso en marcha el servidor de XPN y el proceso controlador de FlexMPI, y se configuró la política de maleabilidad basada en costes junto con el ciclo de *checkpoint* y reinicio que permite variar el número de procesos entre fases de la simulación (Sección 4.4).

Durante estas pruebas se observó el fenómeno que motiva el uso de la maleabilidad: al alcanzarse el pico de la infección, un proceso saturaba su núcleo al 100 % mientras el resto quedaba prácticamente ocioso. El diagnóstico permitió descartar un error de reparto en EpiGraph —la población se divide en bloques de tamaño equivalente y las ejecuciones locales estaban equilibradas— y atribuir el desbalanceo a su origen real: el reparto por bloques combinado con la propagación no uniforme de la infección, que es precisamente lo que el balanceo de carga de FlexMPI debe corregir (Sección 4.4.1). El problema es que dicho balanceo no llegaba a activarse en el entorno contenerizado, con procesadores híbridos en los que los procesos MPI no se clasificaban como dedicados. Fue necesario, por tanto, intervenir en el código de FlexMPI —habilitando la invocación del rebalanceo y relajando las condiciones que lo inhibían en este entorno— para que la redistribución de carga se disparase efectivamente; estas modificaciones, junto con el resto de cambios no triviales sobre las versiones originales, quedan documentadas para garantizar la reproducibilidad.

En esta fase se resolvieron además dos problemas que condicionaban la validez de las medidas. Por un lado, las métricas de rendimiento notificaban un número de operaciones en coma flotante nulo, debido a que la instalación de PAPI del sistema no soportaba el contador solicitado en el hardware de desarrollo; se corrigió seleccionando el evento adecuado y recompilando contra la instalación de PAPI apropiada. Por otro, XPN no llegaba a interceptar la E/S de la aplicación: al precargarse, la biblioteca de intercepción redefinía la rutina de inicialización de MPI y eclipsaba a la de FlexMPI, dejando sin inicializar su comunicador global y abortando el arranque. La solución consistió en evitar que el mecanismo de intercepción redefiniera esa rutina, delegando en la variante que sí podía interceptarse sin conflicto, tras lo cual se verificó que los volcados binarios de estado se materializaban efectivamente sobre el almacenamiento de XPN. Con ello, el entorno local quedó plenamente funcional y reproducible, constituyendo la base sobre la que se diseñaron los casos de estudio y que posteriormente se replicó en el clúster.

TABLA 5.6. PRINCIPALES INCIDENCIAS DE INTEGRACIÓN EN EL ENTORNO LOCAL, CON SU CAUSA Y LA SOLUCIÓN APLICADA.

Problema	**Causa**	**Solución**
Avisos de proveedor de red no soportado en MPICH/CH4	La versión moderna de MPICH ignora la variable heredada de selección de proveedor	Fijar `FI_PROVIDER=tcp` y usar MPICH sobre el dispositivo de sockets
EpiGraph y FlexMPI no compilaban	Rutas de cabeceras y bibliotecas incorrectas en los *Makefiles* (PAPI, GLPK, `libxml2`, GSL)	Corregir las rutas de inclusión y enlazado en ambos ficheros de compilación
Fallos de inicialización en la primera ejecución local	Errores de arranque de la aplicación	Depuración con `gdb` hasta ejecutar con cuatro procesos
Biblioteca de PAPI ausente en el contenedor	El binario se enlazaba contra la versión de PAPI del sistema, no incluida en la imagen	Recompilar dentro del contenedor contra la instalación propia de PAPI y ajustar la ruta de búsqueda
No se encontraba la biblioteca de FlexMPI (`libempi`) ni el lanzador	Rutas de búsqueda de bibliotecas y de `mpiexec` inconsistentes con la instalación empleada	Exportar las rutas de FlexMPI y MPICH y usar el `mpiexec` correspondiente
`mpiexec` acumulaba todos los procesos en un solo nodo	Los contenedores exponían nombres de host aleatorios y no únicos para el lanzador	Construir un mapeo IP → nombre único por nodo y un fichero de máquinas con una ranura por contenedor
Bloqueos en el arranque con la CPU inactiva	Conflictos de puerto y orden de inicialización; SSH entre contenedores pedía contraseña	Configurar SSH sin contraseña, liberar puertos y ordenar el arranque de los servicios

continúa en la página siguiente

TABLA 5.6 — *continuación*

Problema	**Causa**	**Solución**
Un nodo al 100 % y el resto ocioso	Desbalanceo inherente al reparto por bloques con infección no uniforme; el balanceo de FlexMPI no se activaba en el entorno contenerizado	Habilitar el rebalanceo y relajar las condiciones que lo inhibían en el código de FlexMPI
Número de operaciones en coma flotante nulo en las métricas	PAPI del sistema no soportaba el contador solicitado en el hardware	Seleccionar el evento adecuado y recompilar contra la instalación de PAPI apropiada
XPN no interceptaba la E/S y abortaba el arranque de MPI	La biblioteca de intercepción redefinía la inicialización de MPI y eclipsaba a la de FlexMPI, dejando su comunicador sin inicializar	Evitar esa redefinición y delegar en la variante interceptable sin conflicto

5.2.4. Puesta en marcha en el clúster Tucán

Una vez validado el entorno en el equipo local, se trasladó al clúster Tucán de la UC3M (Sección 5.1.2) para evaluar el sistema a mayor escala. La migración no fue inmediata: el acceso se realiza a través del servidor de acceso Guernika, que actúa como host de salto hacia los nodos de cómputo, y del planificador de tareas SLURM, sin privilegios de administrador sobre los nodos, lo que obligó a habilitar la ejecución de Docker sin superusuario y a instalar el complemento de composición en el directorio de usuario, compartido por red entre los nodos; la política de ahorro de energía del clúster exigió, además, un cierre ordenado de los contenedores y de las reservas al término de cada campaña para no dejar nodos en estado de drenaje. Estas incidencias, junto con las que surgieron al desplegar las configuraciones exclusiva y multinodo, se recogen en la Tabla 5.7.

El clúster obligó también a ajustar la propia carga de trabajo para poder apreciar el efecto de XPN. El volumen de E/S de EpiGraph crece de forma lineal con dos factores: el tamaño de la población simulada —en torno a 264 bytes por individuo y día— y la frecuencia con la que se vuelca el fichero de estado. Con la población reducida empleada al principio y un único volcado por día simulado (cada 1440 iteraciones), la E/S apenas suponía unas 130 escrituras de alrededor de 200 MB, en torno a un 2 % del tiempo de ejecución, de modo que la diferencia entre XPN y NFS resultaba imperceptible. Se actuó, por tanto, sobre ambos factores: por un lado se pasó a la población completa de tres millones y medio de individuos, lo que multiplica el tamaño de cada volcado; por otro, se aumentó

en un factor de diez la frecuencia de escritura, volcando cada `EMPI_GLOBAL_niter_lb` iteraciones (144, esto es, cada 0,1 días simulados) en lugar de una vez al día, con lo que el número de escrituras —y de registros de rendimiento `IOPerf`— se multiplicó en la misma proporción. Con estos dos cambios la E/S pasó a ser un factor apreciable frente al cómputo, condición necesaria para que la ventaja de XPN pudiera manifestarse.

Aun con esa mayor carga de E/S, las primeras campañas sobre el clúster arrojaban tiempos prácticamente idénticos con y sin XPN, como si el sistema de ficheros paralelo no llegara a intervenir. Fue precisamente este escenario el que puso de manifiesto el problema de intercepción descrito en la sección anterior. El diagnóstico confirmó la sospecha: al no precargarse la biblioteca de intercepción, toda la E/S —incluidos los volcados binarios de estado— se dirigía en realidad al sistema de ficheros por red, de modo que la comparación que se creía estar realizando entre XPN y NFS era, en la práctica, una comparación de NFS contra sí mismo. La Figura 5.5 cuantifica el efecto de la corrección sobre el clúster con dieciséis procesos, con la ejecución sin XPN en gris y las dos ramas de XPN en naranja y en verde: antes de aplicar la corrección, el tiempo de E/S se mantiene en torno a los 1050 s en todas las configuraciones, indistinguible del obtenido sin XPN (1075 s); una vez corregida la intercepción, la E/S se dirige efectivamente a XPN y el tiempo se desploma hasta 234 s en la configuración colocalizada. Esta figura constituye la justificación empírica de la corrección: sin ella, toda la campaña de medición sobre el clúster carecía de validez, pues comparaba el sistema de ficheros por red consigo mismo.

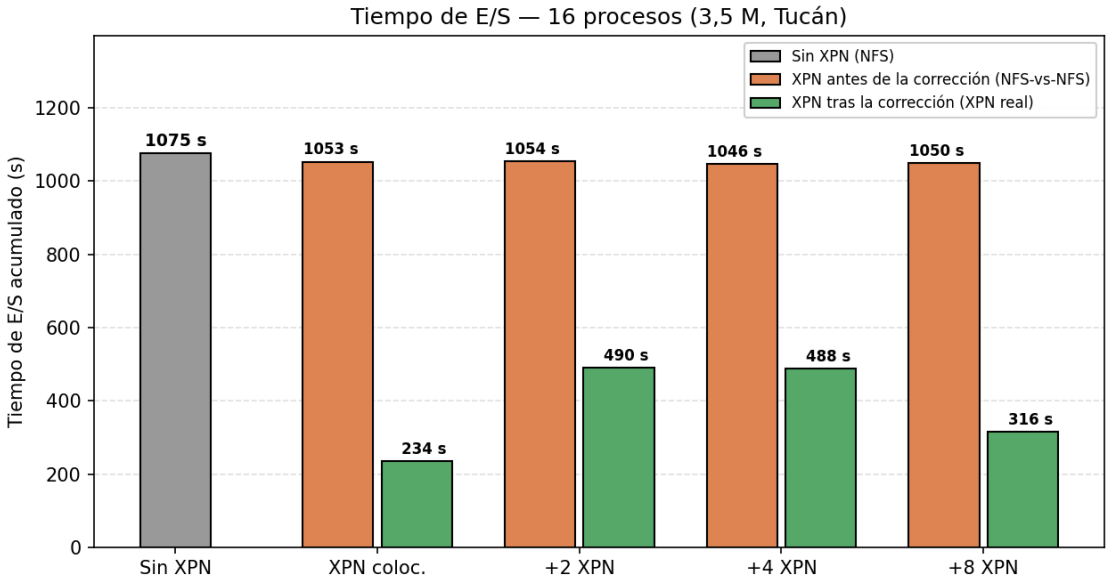


Fig. 5.5. Tiempo de E/S acumulado antes y después de corregir la intercepción de XPN.

La figura recoge también el comportamiento en modo exclusivo, en el que los servidores de XPN se ubican en nodos distintos de los que ejecutan la aplicación. Poner en marcha esta configuración exigió una corrección adicional específica del clúster: la creación del fichero de estado mediante E/S paralela abortaba cuando ningún servidor de XPN estaba colocalizado con los procesos, un supuesto de localidad que la biblioteca daba por

sentado. Al desactivar dicha suposición en el modo exclusivo, la escritura paralela pasó a completarse correctamente. Con ello, el tiempo de E/S en exclusivo desciende de 490 a 316 s al aumentar de dos a ocho el número de servidores dedicados, lo que evidencia que, una vez que XPN intercepta realmente la E/S, esta escala con el número de servidores de almacenamiento; la configuración colocalizada, en la que cada proceso dispone de un servidor local, es la que ofrece el menor tiempo de todos, con 234 s frente a los 1075 s de NFS, esto es, una reducción del 78 % del tiempo de E/S.

La ejecución multinodo real —con la aplicación y el almacenamiento repartidos entre máquinas físicas distintas— obligó a sortear las limitaciones de la red virtual de contenedores por defecto del clúster, que provocaba bloqueos en las operaciones colectivas de MPI a partir de cierto número de procesos. Para superarlas se recurrió a una red superpuesta gestionada por Docker Swarm, que sostiene la separación entre los nodos de cómputo y los de almacenamiento. La maleabilidad por creación dinámica de procesos requirió, a su vez, ajustar la forma en que FlexMPI reutilizaba los argumentos de invocación al lanzar los nuevos procesos y ampliar el número máximo de opciones admitidas; con estas correcciones, la expansión del número de procesos durante el pico de la infección funciona de forma estable en el clúster. El comportamiento cuantitativo de todas estas configuraciones se analiza en los casos de estudio (Sección 5.4).

TABLA 5.7. PRINCIPALES INCIDENCIAS ESPECÍFICAS DE LA PUESTA EN MARCHA SOBRE EL CLÚSTER TUCÁN, CON SU CAUSA Y LA SOLUCIÓN APLICADA.

Problema	**Causa**	**Solución**
Docker no era ejecutable sin privilegios de administrador en los nodos	Los nodos de cómputo estaban provisionados sin pertenencia al grupo de Docker y el planificador propagaba grupos que no incluían al usuario	Acceder por sesión interactiva al nodo, no a través del planificador, e instalar el complemento de composición en el directorio de usuario compartido por red
Los nodos quedaban inutilizables (en estado de drenaje) tras las campañas	La política de ahorro de energía, unida a la interrupción de la reserva, dejaba contenedores fuera del control del planificador	Cerrar de forma ordenada los contenedores y liberar las reservas al término de cada campaña

continúa en la página siguiente

TABLA 5.7 — *continuación*

Problema	**Causa**	**Solución**
La escritura del fichero de estado por E/S paralela abortaba en modo exclusivo	La biblioteca de intercepción asumía un servidor de XPN colocalizado con cada proceso, inexistente cuando los servidores ocupan nodos dedicados	Desactivar la suposición de localidad de XPN en el modo exclusivo
Bloqueo de las operaciones colectivas de MPI a partir de cierto número de procesos	Limitaciones de la red virtual de contenedores por defecto (puente) del clúster para el tráfico intensivo entre múltiples nodos	Emplear una red superpuesta gestionada por Docker Swarm para la ejecución multinodo
La maleabilidad por creación dinámica de procesos no llegaba a arrancar los nuevos procesos	FlexMPI reutilizaba los argumentos de invocación del proceso original al generar los hijos y se rebasaba el número máximo de opciones admitidas	Corregir el paso de argumentos en la creación de procesos y ampliar el número máximo de opciones

5.3. Matriz de trazabilidad

La matriz de trazabilidad de la Tabla 5.8 relaciona los requisitos definidos en la Sección 4.3 con los casos de estudio diseñados para la evaluación (Sección 5.4). De esta forma es posible comprobar que todos los requisitos quedan cubiertos por al menos un caso de estudio. Dado el carácter experimental de este trabajo, cada caso de estudio se corresponde con una campaña de experimentos sobre la integración de EpiGraph, FlexMPI y Expand Ad-Hoc:

- **CE1 — Caracterización de la E/S e interceptación transparente.**
- **CE2 — Configuraciones de despliegue del almacenamiento.**
- **CE3 — Maleabilidad del número de procesos.**
- **CE4 — Rebalanceo dinámico de carga.**

El diseño de cada una de estas campañas —qué se compara, sobre qué plataforma y con qué observables— se desarrolla en la Sección 5.4.

TABLA 5.8. MATRIZ DE TRAZABILIDAD ENTRE LOS REQUISITOS Y LOS CASOS DE ESTUDIO.

Requisito	**CE1**	**CE2**	**CE3**	**CE4**
RF-01 — Interceptación transparente de la E/S	✓	✓		
RF-02 — Maleabilidad del número de procesos			✓	
RF-03 — Rebalanceo dinámico de carga				✓
RF-04 — Persistencia del estado en XPN	✓		✓	
RF-05 — Configuración del despliegue del almacenamiento		✓		
RNF-01 — Reproducibilidad del entorno	✓	✓	✓	✓
RNF-02 — Equivalencia de resultados	✓	✓	✓	✓
RNF-03 — Portabilidad del entorno	✓	✓	✓	✓
RNF-04 — Tecnologías de implementación	✓	✓	✓	✓

Los cuatro casos de estudio se ejecutan sobre el entorno contenerizado y reproducible descrito en la Sección 3.2.4, tanto en el equipo local de desarrollo como en el clúster Tucán de la UC3M, y sobre la misma base tecnológica (C, MPI/MPICH y PAPI). Por ello, los requisitos no funcionales RNF-01 a RNF-04 quedan cubiertos de forma transversal por cualquiera de ellos. El requisito RF-01 se verifica en CE1, donde se comprueba la interceptación transparente de la E/S, y también en CE2, que opera necesariamente sobre XPN. RF-04 queda cubierto en CE1, al confirmarse que el estado del *checkpoint* se almacena en XPN, y en CE3, que se apoya en dicho mecanismo para la maleabilidad.

En cuanto al resto de requisitos funcionales, el RF-02, relativo a la maleabilidad del número de procesos, se verifica en CE3, donde se modifica en caliente el número de procesos de EpiGraph durante una misma simulación empleando los dos mecanismos considerados —el *checkpoint*/reinicio y el modo dinámico nativo de FlexMPI—, comprobando que la ejecución continúa con la nueva configuración sin necesidad de relanzarla manualmente. El RF-03, que exige el rebalanceo dinámico de la carga, queda cubierto en CE4: al contrastar ejecuciones idénticas con y sin la política de balanceo activa se comprueba que el mecanismo interviene y se cuantifica su efecto sobre el tiempo de ejecución y sobre el coste de coordinación. Por último, el RF-05, referente a la configuración del despliegue del almacenamiento, se valida en CE2, que ejecuta de forma sistemática los modos colocalizado y exclusivo y varía el número de servidores XPN, demostrando que ambas dimensiones de configuración son operativas y caracterizando su efecto sobre el rendimiento de la entrada/salida.

De este modo, la matriz confirma que cada uno de los requisitos definidos en la Sección 4.3 queda cubierto por al menos un caso de estudio, lo que garantiza la trazabilidad completa entre las capacidades exigidas al sistema y su validación experimental.

5.4. Casos de estudio

Los cuatro casos de estudio enunciados en la sección anterior constituyen la evaluación funcional del sistema. Cada uno de ellos se corresponde con una campaña de experimentos completa, con su propia configuración de despliegue, su conjunto de ejecuciones y sus métricas de interés. Esta sección describe el diseño de cada campaña —qué se compara, sobre qué plataforma y con qué observables—, mientras que los resultados cuantitativos se analizan en la Sección 5.5.

Todas las campañas simulan 130 días con 1440 iteraciones por día, lo que sitúa el final de la ejecución en la iteración 187 200, y recorren en ese intervalo dos olas epidémicas sucesivas. Es esta segunda característica la que hace de EpiGraph un caso de estudio adecuado para la maleabilidad: la demanda de cómputo varía en más de un factor de 3 a lo largo de una misma ejecución, de modo que ninguna configuración fija de recursos puede ser simultáneamente adecuada para todas sus fases.

La escala de la carga, en cambio, no es la misma en todas las campañas, y conviene precisarlo porque condiciona la comparación de resultados. Dos parámetros la fijan, y es preciso separarlos porque durante buena parte del trabajo estuvieron ligados. El primero es el tamaño de la población: la reducida, de 800 826 individuos, y la completa de la ciudad de Madrid, de 3 511 110. El segundo es la frecuencia con que se vuelca el estado, un único volcado por día simulado o diez, esto es, cada 1440 o cada 144 iteraciones. Las primeras campañas sobre el equipo local emplearon la población reducida con un volcado diario, y las del clúster Tucán la población completa con diez, de modo que ambos parámetros cambiaban a la vez que la máquina. CE4 rompe deliberadamente esa correspondencia y ejecuta las cuatro combinaciones sobre el clúster, por las razones que se exponen en la Sección 5.4.4. Conviene aclarar que la cifra mayor no pretende reflejar el censo actual de la ciudad: forma parte de la población sintética que acompaña a EpiGraph, generada a partir de los datos demográficos disponibles cuando se diseñó el escenario de simulación. Es, por tanto, un tamaño de problema y no una medida de población: lo relevante para esta evaluación es que la carga de cómputo y de entrada/salida sea la misma en todas las ejecuciones que se comparan entre sí. Como se justificó en la Sección 5.2.4, el aumento deliberado de la carga de entrada/salida —por ambos factores a la vez— es la condición necesaria para que el efecto del sistema de ficheros paralelo resulte apreciable. Las comparaciones que se establecen en cada caso de estudio se realizan siempre entre ejecuciones de la misma escala.

5.4.1. CE1 — Caracterización de la E/S e interceptación transparente

El primer caso de estudio verifica que XPN llega a intervenir realmente en la entrada/-salida de EpiGraph y cuantifica el efecto de hacerlo. Se comparan ejecuciones idénticas de la aplicación que solo difieren en el destino de las escrituras: el sistema de ficheros de referencia —NFS sobre el directorio de usuario en el clúster Tucán, y el volumen compartido entre contenedores en el equipo local— frente a XPN. La redirección se obtiene precargando la biblioteca de intercepción antes de la ejecución, sin alterar el código de entrada/salida de la aplicación ni recompilarla, lo que constituye en sí mismo la validación del requisito de transparencia.

El observable principal es el tiempo acumulado de entrada/salida, y merece la pena detallar cómo se obtiene, porque de ello depende la interpretación de todas las medidas que siguen. Es la propia aplicación la que se instrumenta: antes de cada volcado de estado, una barrera sincroniza a todos los procesos, y la escritura colectiva que viene a continuación queda encerrada entre dos lecturas del reloj de MPI, descontando el coste de la propia consulta al reloj. El proceso de rango cero emite entonces en el registro de ejecución una línea `IOPerf` con la latencia de esa escritura, el número de individuos volcados y el ancho de banda agregado que de ahí se deduce. Con 130 días simulados y diez volcados por día, cada ejecución produce 1300 de estas líneas, y el tiempo de entrada/salida que se compara entre configuraciones es sencillamente la suma de esas latencias.

Esta medida procede, pues, de EpiGraph y no de FlexMPI, y la distinción importa porque la traza de FlexMPI incluye un contador de entrada/salida propio que aparece a cero durante toda la ejecución. Ese cero no indica que no haya escrituras, sino que la biblioteca de maleabilidad solo contabiliza las operaciones que ella misma envuelve, y el volcado de estado de EpiGraph no pasa por esa vía. Es la instrumentación de la aplicación, y solo ella, la que proporciona la medida empleada en este trabajo. A este observable se añade una comprobación de carácter funcional: la inspección de los directorios de datos de cada servidor de XPN, que permite confirmar que el fichero de estado se ha materializado efectivamente sobre el sistema de ficheros paralelo y repartido entre sus servidores, y no sobre el sistema de ficheros de referencia.

Junto a ellos se registra también el tiempo total de la simulación, porque la campaña debe responder a dos preguntas distintas: si XPN acelera las escrituras y si esa aceleración acorta la ejecución. No son la misma pregunta. Un sistema de ficheros más rápido no mejora el tiempo total si la entrada/salida apenas pesa en él, que era justamente la situación de partida antes de aumentar la carga de escritura del modo descrito en la Sección 5.2.4. Medir las dos magnitudes en cada ejecución es lo que permite comprobar si van juntas.

Este caso de estudio comprende asimismo la campaña de medidas anterior a la corrección del mecanismo de intercepción. Su interés no reside en los valores obtenidos —carentes de validez, puesto que comparaban el sistema de ficheros por red consigo mismo— sino en que delimitan la frontera entre lo que se estaba midiendo y lo que se creía estar midiendo, y en que permiten cuantificar, por diferencia, el efecto real de la

intercepción, tal y como recoge la Figura 5.5.

5.4.2. CE2 — Configuraciones de despliegue del almacenamiento

El segundo caso de estudio parte de lo que establece CE1 —que XPN interviene efectivamente en la entrada/salida de la aplicación— para plantear la pregunta siguiente: cómo conviene desplegarlo. No se trata ya de comprobar si el sistema de ficheros paralelo se emplea, sino de determinar qué disposición de sus servidores rinde mejor, que es lo que exige el requisito RF-05. Se consideran para ello dos dimensiones de configuración: dónde se sitúan los servidores de almacenamiento respecto de los procesos de cómputo —los modos colocalizado y exclusivo de la Figura 5.2(a) y (b)— y cuántos servidores se despliegan.

El barrido se realiza sobre el clúster Tucán con la población completa, en dos tamaños de aplicación: 8 y 16 procesos de cómputo. Para cada uno de ellos se ejecuta el modo colocalizado, en el que cada contenedor de cómputo aloja además un servidor, y el modo exclusivo con un número creciente de servidores dedicados: 2, 4 y 8 en las ejecuciones de 8 procesos, y 2, 4, 8 y 16 en las de 16. La ejecución sobre el sistema de ficheros de referencia, ya empleada en CE1, sirve de anclaje común a ambas series. Conviene advertir que las dos dimensiones no son libres a la vez: en el modo colocalizado el número de servidores queda fijado por el de procesos, de modo que comparar sin más colocalizado y exclusivo confunde la ubicación con el número. Por eso cada uno de los dos barridos incluye una configuración exclusiva con tantos servidores dedicados como procesos de cómputo —ocho y dieciséis, respectivamente—, que iguala el número de servidores del modo colocalizado y deja la ubicación como única variable; son esas dos parejas de ejecuciones, y no la comparación global entre modos, las que permiten atribuir el efecto observado a una u otra dimensión. Ambas ejecuciones de referencia se repitieron, junto con las configuraciones que anclan, en un mismo despliegue, de modo que la comparación entre el sistema de ficheros de referencia y XPN no arrastra la variabilidad propia de comparar entre despliegues distintos. Cada uno de los dos barridos se ejecutó íntegramente dentro de un único despliegue, junto con su ejecución de referencia, de manera que ninguna de las comparaciones que aquí se establecen mezcla despliegues.

El barrido del número de servidores se realiza únicamente sobre las dos topologías que residen en un único nodo físico. La razón es de método: al mantener todos los contenedores en la misma máquina, el número de servidores queda como única variable, sin que intervengan ni el reparto de los contenedores entre nodos ni la fracción de escrituras que cada proceso llega a resolver sin salir de su propia máquina. Las topologías multinodo introducen precisamente ese segundo factor —la distribución de los servidores entre las dos máquinas, que determina cuánta entrada/salida atraviesa efectivamente la red—, por lo que se examinan por separado, como una dimensión propia, en la Sección 5.5, allí sobre las mismas configuraciones de servidores y comparando las cuatro disposiciones de la Figura 5.2.

Los observables son los de CE1 —el tiempo acumulado de entrada/salida y la latencia de cada volcado de estado—, ampliados con el tiempo total de la simulación y su descomposición en cómputo, entrada/salida y coordinación. Esta ampliación es necesaria porque las configuraciones que aquí se comparan no se distinguen únicamente por lo que tardan en escribir: colocalizar un servidor junto a cada proceso de cómputo hace que ambos compitan por el mismo núcleo, mientras que dedicar contenedores exclusivamente al almacenamiento elimina esa contención a costa de que ninguna escritura encuentre ya su destino en el propio contenedor. Son dos efectos de signo contrario que el tiempo total, por sí solo, no permite separar; la descomposición sí. Esta se obtiene atribuyendo a la entrada/salida la suma de los intervalos `IOPerf`, al cómputo el tiempo de procesador que FlexMPI acumula en su contador `RTIME` —validado como estimador del tiempo de CPU efectivo— y a la coordinación el residuo hasta el tiempo total, que recoge tanto la comunicación propia de la simulación como la espera en las barreras colectivas.

A estas dos dimensiones se añade un tercer parámetro de configuración del almacenamiento, evaluado de forma independiente: el tamaño de bloque con el que XPN reparte los datos entre sus servidores (*bsize*), descrito en la Sección 3.2.3, que fija el grano del entrelazado y, con él, cuántos servidores atienden en paralelo cada escritura. El valor por defecto de la distribución es de 512 KiB, y la duda razonable es si un grano mayor aprovecharía mejor los servidores disponibles. Para resolverla se repitieron con 1024 KiB todas las configuraciones de 8 y 16 procesos descritas más arriba, de manera que la comparación entre ambos valores se establece dentro de cada configuración y no entre configuraciones distintas. Como referencia externa, el sistema de ficheros NFS del clúster —montado sobre NFSv4.1— transfiere en bloques de 128 KiB, de modo que ambos candidatos se sitúan holgadamente por encima del grano del sistema con el que se compara. De esta comparación se sigue el valor que se fija para el resto de las campañas, que se justifica junto con las medidas que lo respaldan en la Sección 5.5.

Conviene precisar, por último, cuántas veces se midió cada configuración, porque de ello depende qué diferencias pueden afirmarse. La campaña inicial ejecutó una sola vez cada celda de los dos barridos, lo que basta para las diferencias grandes —las que separan el sistema de ficheros de referencia de XPN— pero no para las que se observan entre configuraciones vecinas del modo exclusivo, que son del orden de la dispersión entre ejecuciones. Esa insuficiencia obligó a repetir el barrido, y la forma de hacerlo no es indiferente. Repetir una configuración en otro despliegue no mide solo la variabilidad entre ejecuciones: arrastra también la que introduce el propio despliegue, y esta última resultó ser sistemática. Las tres configuraciones exclusivas de 8 procesos, repetidas en una campaña anterior, se desplazan las tres a la vez y en el mismo sentido respecto de la campaña posterior, lo que delata un efecto del despliegue y no de la configuración. Por ese motivo la campaña definitiva mide dos veces cada celda de los dos barridos dentro de un mismo despliegue, junto con sus ejecuciones de referencia, y las repeticiones procedentes de despliegues anteriores se reservan para estimar esa segunda fuente de variabilidad. Las dos se cuantifican por separado en la Sección 5.5, y el número de repeticiones acompaña

a cada medida.

5.4.3. CE3 — Maleabilidad del número de procesos

El tercer caso de estudio evalúa la capacidad del sistema para modificar el número de procesos de EpiGraph mientras la simulación está en marcha, que es lo que exige el requisito RF-02, y lo hace apoyándose en el estado que la aplicación persiste sobre XPN, con lo que cubre también RF-04. La pregunta que aquí se plantea no es únicamente si la reconfiguración es posible —eso se comprueba de una vez—, sino en qué condiciones compensa: con qué mecanismo, en qué momento de la simulación, hasta qué número de procesos y en qué dirección.

Se contrastan los dos mecanismos que ofrece FlexMPI. El primero es el ciclo de *checkpoint* y reinicio descrito en la Sección 4.4.4: la aplicación vuelca su estado sobre el sistema de ficheros paralelo, termina, y se relanza con el nuevo número de procesos, que reconstruyen la partición a partir de ese volcado y prosiguen la simulación desde el día guardado. El segundo es el modo dinámico, en el que la biblioteca crea o retira procesos sobre la aplicación en ejecución y redistribuye entre ellos el estado sin que esta llegue a detenerse. Ambos persiguen el mismo fin y difieren en el sobrecoste que introducen y en la infraestructura que exigen, de modo que la comparación entre ambos se realiza sobre una única máquina y con la misma configuración de cómputo, dejando el mecanismo como única variable. Las campañas de mayor escala, desplegadas sobre las topologías multinodo de la Figura 5.2(c) y (d), emplean solo el modo dinámico. El ciclo de *checkpoint* y reinicio no resultó viable sobre el clúster: con la población completa, la fase que sigue al reinicio debe repartir entre el nuevo conjunto de procesos el estado leído del sistema de ficheros paralelo, y esa redistribución no llegó a completarse en ningún intento, de modo que la simulación quedaba detenida antes de reanudar el cómputo. A la vista de ello se optó por aplicar la maleabilidad a esa escala únicamente con el modo dinámico, que redistribuye el estado en memoria entre los procesos existentes y los recién creados y no exige volver a leerlo.

En todas ellas la reconfiguración se solicita al controlador de FlexMPI en días simulados prefijados, expresados como una lista de pares día-número de procesos, de manera que una misma ejecución puede recorrer varias configuraciones sucesivas. Este control explícito del calendario es deliberado: si la decisión se dejase en manos de una política automática, el experimento mediría a la vez el acierto de esa política y el coste del mecanismo, y lo que aquí interesa es aislar el segundo.

Sobre esa base se recorren tres dimensiones. La primera es la magnitud del cambio: partiendo de 8 procesos, se expande a 16, 32, 40, 48 y 64, valores que van desde muy por debajo hasta rebasar los 40 núcleos físicos de que dispone cada nodo del clúster. La segunda es el momento: fijados el punto de partida y el de destino, la expansión se sitúa alternativamente en los días 30, 40 y 50, y se añade un cuarto barrido en el día 78 —ya rebasada la primera onda— con el fin de determinar en qué medida adelantar o retrasar la

decisión altera el resultado. Se ensaya asimismo la operación contraria, una contracción situada en el día 120. La tercera es la dirección. Además de las ejecuciones que únicamente crecen, se diseñan calendarios bidireccionales que expanden al comienzo de la primera onda epidémica, se contraen durante el valle que la sigue y vuelven a expandir para la meseta final, siguiendo así el perfil de demanda de la escena simulada.

A estas campañas se añade una última, específica de co-maleabilidad, en la que se reconfiguran a la vez los dos recursos que el diseño mantiene desacoplados (Sección 4.4.4): el mismo día en que EpiGraph expande su número de procesos, XPN reduce a la mitad su número de servidores de almacenamiento. La hipótesis que se somete a prueba es que las dos necesidades no coinciden en el tiempo —la fase inicial es barata en cómputo pero escribe con la misma intensidad que el resto, mientras que la fase de pico está dominada por el cómputo—, de modo que soltar almacenamiento justo cuando se pide cómputo no debería penalizar la ejecución. Se comparan para ello tres configuraciones idénticas en todo salvo en el almacenamiento: ocho servidores durante toda la ejecución, cuatro durante toda la ejecución, y la reducción de ocho a cuatro en el día de la expansión. Cada una de las tres se repite seis veces, sobre la topología distribuida de la Figura 5.2(d), porque los efectos que se pretende afirmar son del mismo orden de magnitud que la dispersión entre ejecuciones equivalentes.

El observable principal deja de ser aquí el tiempo total de la simulación. Una configuración estática dimensionada al máximo dispone de todos los recursos durante toda la ejecución y difícilmente puede ser superada en tiempo por otra que arranca con menos, de forma que juzgar la maleabilidad solo por el reloj es juzgarla precisamente por donde no pretende aportar. A cada ejecución se le asocia por ello una segunda magnitud, el consumo de recursos, calculada como la suma sobre las fases de la ejecución del producto del número de núcleos reservados por la duración de la fase, y expresada en núcleos $\times$ hora. Ese cómputo incluye tanto los procesos de la aplicación como los servidores de almacenamiento —cada servidor ocupa un contenedor anclado a un núcleo—, sin lo cual el efecto de la co-maleabilidad no sería visible por ninguna vía. Frente a ambas magnitudes se sitúan como referencia las ejecuciones estáticas con el número de procesos fijado en el valor de partida y en el del pico, que acotan lo que cabe esperar de cualquier calendario intermedio.

La comprobación de que la reconfiguración ha tenido lugar no se apoya en el código de retorno de las órdenes emitidas, sino en su efecto observable: las líneas `IOPerf` que la aplicación emite en cada volcado incluyen el número de procesos activos, la traza del controlador registra cada orden de creación o eliminación junto con el tiempo invertido en la redistribución de los datos, y la inspección de los servidores de XPN permite verificar que el fichero de estado se ha reconstruido sobre la nueva partición, que es el punto en el que RF-04 se pone realmente a prueba. A ello se añade una precaución metodológica que condiciona todas las comparaciones de esta campaña: tanto el tiempo como el consumo de recursos se miden en una misma iteración de la simulación, común a todas las ejecuciones que se contrastan, de modo que todas ellas se comparan sobre exactamente la misma

cantidad de trabajo simulado.

5.4.4. CE4 — Rebalanceo dinámico de carga

El cuarto caso de estudio evalúa la segunda capacidad que aporta FlexMPI, el rebalanceo dinámico de la carga entre un número constante de procesos, que es lo que exige el requisito RF-03. Conviene distinguirla de la anterior: CE3 modifica cuántos procesos ejecutan la simulación, mientras que aquí ese número permanece fijo y lo que se redistribuye es el trabajo entre ellos. Son dos mecanismos independientes de la misma biblioteca, y separarlos es condición para atribuir a cada uno lo que le corresponde. La pregunta, de nuevo, no es si el rebalanceo ocurre, sino en qué condiciones compensa, dado que el balanceador impone un coste de coordinación en cada evaluación.

La comparación se plantea como una ablación de un único parámetro. Se ejecutan parejas de simulaciones idénticas en todo —mismo binario, misma población, mismo número de procesos, mismo despliegue y mismo lanzador— que difieren únicamente en la política de balanceo que se pasa a FlexMPI, activada o desactivada. Esta exigencia no es retórica. Las primeras medidas de este caso de estudio se obtuvieron con lanzadores distintos para cada rama, de modo que entre ambas variaba algo más que la política y la comparación no constituía una ablación; las medidas que aquí se emplean rehacen esas parejas con un solo lanzador.

Sobre esa base se recorren tres factores que en las campañas iniciales aparecían confundidos entre sí. El primero es el hardware: el equipo local ejecuta ocho procesos sobre cuatro núcleos físicos con multihilo simultáneo, de modo que los procesos no disponen de recursos equivalentes, mientras que en el clúster cada proceso ocupa un núcleo dedicado. El segundo es la cadencia con la que se evalúa el balance, que toma los valores de 1440 y 144 iteraciones. El tercero es el tamaño de la población, en sus dos variantes. La dificultad es que las campañas iniciales variaban los tres a la vez —el equipo local ejecutaba la población reducida con un volcado por día, y el clúster la completa con diez—, de manera que cualquier diferencia admitía tres explicaciones distintas. CE4 los separa ejecutando sobre el clúster las combinaciones que faltaban, de modo que cada comparación deja un solo factor libre.

El segundo de esos factores merece una precisión, porque no se introdujo deliberadamente. El parámetro que fija cada cuántas iteraciones se vuelca el estado gobierna también, en el mismo código, cada cuántas se invoca al monitor del balanceador. Al pasar de uno a diez volcados por día para hacer visible el efecto del sistema de ficheros paralelo, tal y como se describe en la Sección 5.2.4, se multiplicó por diez también la frecuencia con que se evalúa el balance. La cadencia deja de ser así un detalle de instrumentación y pasa a ser un eje del experimento, que es la razón de que figure entre los factores anteriores.

El barrido se realiza sobre el modo colocalizado, con 8 y 16 procesos de cómputo. El tamaño menor sostiene el diseño factorial completo, por ser el que permite repetir las

ejecuciones a un coste razonable, y el mayor replica sobre la población completa la pareja con y sin balanceo, con el fin de comprobar que el resultado no depende del número de procesos.

Los observables sobre los que se apoyan las conclusiones son el tiempo total de la simulación y el tiempo de entrada/salida, que son medidas directas y se obtienen en los mismos términos que en CE2. El coste del mecanismo tendría un observable más específico en el tiempo de comunicación que FlexMPI contabiliza por separado, ya que cada evaluación del balance exige intercambiar con el controlador el estado de carga de cada proceso y, cuando la partición se modifica, migrar entre procesos las columnas afectadas. Esa métrica se registra y se examina, pero no sostiene aquí ninguna afirmación. Depende de los mismos contadores hardware cuyo acceso limita el entorno de contenedores, y los valores que produce no son estables entre intervalos consecutivos de una misma ejecución, de modo que solo admite una lectura cualitativa. Lo mismo vale, y por los mismos motivos, para la componente de coordinación obtenida por diferencia, según se detalla en la Sección 5.5. El diseño asume por tanto que el coste del rebalanceo se observa por su efecto en el tiempo total, una vez descontada la entrada/salida, y no por una medida separada del mecanismo.

Toda ejecución de esta campaña está sujeta además a una condición de validez previa. El balanceador decide a partir del número de operaciones en coma flotante que cada proceso comunica, obtenido de los contadores hardware por medio de PAPI, y en un contenedor esos contadores pueden no estar disponibles. Cuando eso ocurre la métrica llega a cero, el balanceador se queda sin entrada y la ejecución, aunque termine correctamente, no informa sobre el rebalanceo. Por ello se verifica en cada registro que la métrica es distinta de cero antes de admitir la ejecución en la comparación, y se descartan las campañas anteriores a la corrección que habilitó el acceso a los contadores.

Por último, las ejecuciones que sostienen las afirmaciones centrales se repiten, porque los efectos que se pretende afirmar deben distinguirse de la variabilidad del entorno. Esta se cuantifica en dos niveles: la dispersión entre ejecuciones equivalentes dentro de un mismo despliegue y la que se observa entre despliegues distintos de la misma configuración. El número de repeticiones disponible para cada celda se indica junto a los resultados en la Sección 5.5.

5.5. Resultados

Esta sección recoge las medidas de las cuatro campañas cuyo diseño se ha descrito en la sección anterior, y las presenta en el mismo orden. Antes de entrar en ningún caso de estudio se cuantifica la variabilidad del entorno de ejecución, porque es lo que determina qué diferencias pueden afirmarse y cuáles quedan por debajo del ruido de medida; sin esa referencia previa, buena parte de las comparaciones que siguen no admitiría interpretación.

Todas ellas se toman sobre un mismo punto de la simulación, el volcado de estado de la iteración 185 760, y no sobre la duración completa de la ejecución. La razón es metodológica: esa duración comprende la carga de la población y la terminación del trabajo, ajenas al cómputo que se pretende comparar, y medir en un punto fijo del calendario simulado garantiza que todas las cifras que se contrastan cubren exactamente la misma cantidad de trabajo. La diferencia no es menor: en la configuración colocalizada de 16 procesos, comparar duraciones completas arrojaba una discrepancia del 3,8 % entre dos despliegues, mientras que sobre el punto de medida común esa discrepancia se reduce al 1,8 %. De cada ejecución se toman, por tanto, la marca de tiempo de ese volcado y la suma del tiempo de entrada/salida acumulado hasta él.

Los observables son los que fija el diseño de cada campaña. El tiempo total y el tiempo de entrada/salida acompañan a todas las comparaciones, mientras que el consumo de recursos expresado en núcleos por hora aparece solo en la campaña de maleabilidad, donde una configuración estática dimensionada al máximo difícilmente puede ser superada en tiempo por otra que arranca con menos procesos, y juzgarla únicamente por el reloj sería juzgarla por donde no pretende aportar.

5.5.1. Variabilidad de las medidas

Ninguna de las diferencias que se presentan más adelante admite interpretación sin saber cuánto varía una medida por el solo hecho de repetirla. Esa variabilidad se manifiesta en dos niveles de magnitud distinta, y conviene separarlos: el que se observa entre ejecuciones equivalentes dentro de un mismo despliegue de contenedores, y el que aparece al medir una misma configuración sobre despliegues levantados en momentos distintos.

El primer nivel se cuantifica sobre la campaña de co-maleabilidad, en la que se reconfiguran de forma simultánea el número de procesos de EpiGraph y el número de servidores de almacenamiento de XPN, y de la que se dispone de seis ejecuciones de una misma configuración repartidas entre dos despliegues. Las cuatro que comparten despliegue presentan una desviación típica de 23,2 s sobre una media de 3455,4 s, un 0,7 %. La Figura 5.6 enfrenta las dos que más se alejan entre sí de esas cuatro, separadas por 54,1 s, un 1,6 %, que es la mayor discrepancia observada entre dos ejecuciones de un mismo despliegue. El panel superior recoge el tiempo transcurrido de cada una y el inferior su diferencia, que se acumula de forma gradual a lo largo de la simulación, sin que ningún tramo concentre el desvío.

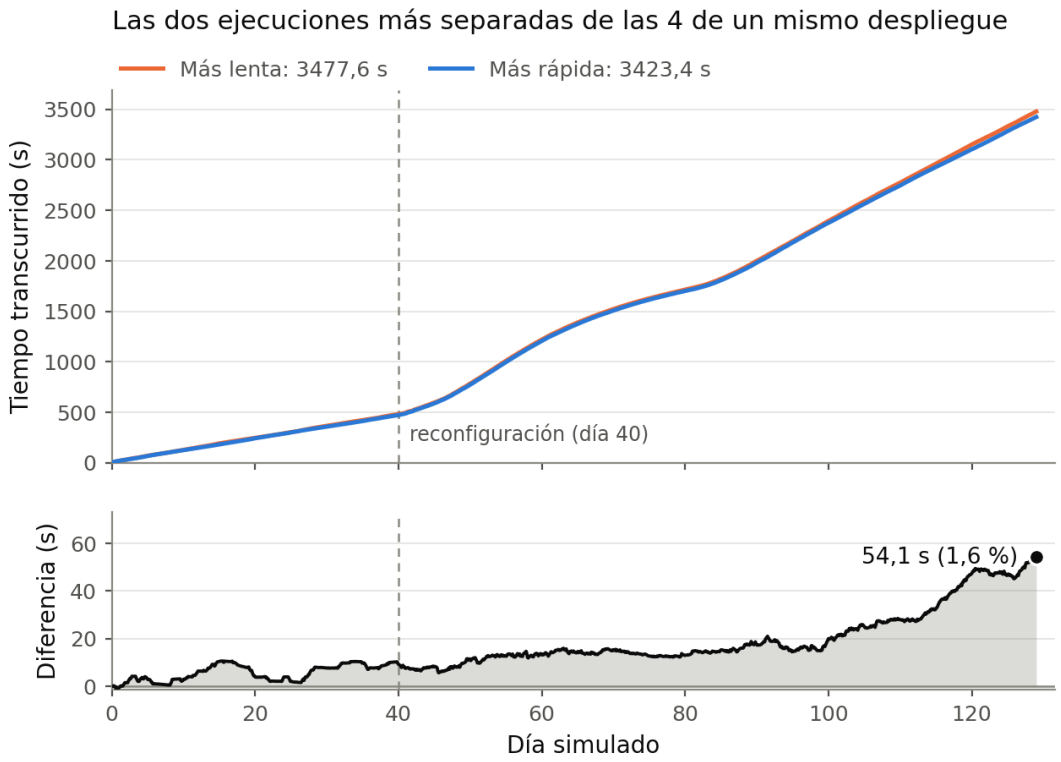


Fig. 5.6. Las dos ejecuciones más separadas entre sí de una misma configuración.

El segundo nivel se estima por dos vías. La primera es esa misma campaña, cuya configuración se midió también sobre un segundo despliegue: allí el tiempo medio es de 3490,9 s, frente a los 3455,4 s del despliegue anterior. La diferencia es del 1,0 % y los dos grupos de medidas no se solapan, de modo que el efecto es sistemático y no una fluctuación. La segunda vía son las diez configuraciones estáticas de la Tabla 5.9, medidas sobre dos despliegues distintos, cuya dispersión tiene una mediana del 0,7 % y un máximo del 5,0 %. Las dos estimaciones son coherentes entre sí: el desplazamiento que introduce cambiar de despliegue está en torno al 1 %, con una cola que en un caso alcanza el 5 %.

TABLA 5.9. DISPERSIÓN ENTRE DESPLIEGUES DE LAS CONFIGURACIONES ESTÁTICAS.

Configuración	**Medidas (s)**	**Media (s)**	**Dispersión**
8 procesos, NFS	8329,0 · 8369,8	8349,4	0,5 %
8 procesos, colocalizado	6951,8 · 6958,3	6955,1	0,1 %
8 procesos, sin balanceo	6497,0 · 6634,7	6565,9	2,1 %
16 procesos, NFS	6264,5 · 6297,6	6281,1	0,5 %
16 procesos, colocalizado	4629,8 · 4716,0	4672,9	1,8 %
16 procesos, sin balanceo	4135,4 · 4348,8	4242,1	5,0 %
16 procesos, 2 servidores	4909,0 · 4949,5	4929,2	0,8 %
16 procesos, 4 servidores	4893,1 · 4971,1	4932,1	1,6 %
16 procesos, 8 servidores	4725,3 · 4755,1	4740,2	0,6 %
16 procesos, 16 servidores	4727,7 · 4717,9	4722,8	0,2 %

Las dos medidas de cada fila proceden de despliegues distintos, que es lo que hace que la columna de dispersión sea informativa. La Figura 5.7 representa esas mismas dispersiones junto a las del tiempo de entrada/salida, con los dos umbrales adoptados marcados por líneas discontinuas.

El tiempo de entrada/salida resulta notablemente menos reproducible que el tiempo total: sobre esas mismas configuraciones su dispersión tiene una mediana del 3,7 % y un máximo del 12,6 %. Es un comportamiento esperable, por tratarse de una cantidad un orden de magnitud menor que el tiempo total y de la más expuesta a la contención con el resto de la actividad del nodo. Tampoco acompaña al tiempo total: la configuración de 16 procesos con 16 servidores es la más reproducible en tiempo, con un 0,2 %, y la segunda peor en entrada/salida, con un 10,7 %.

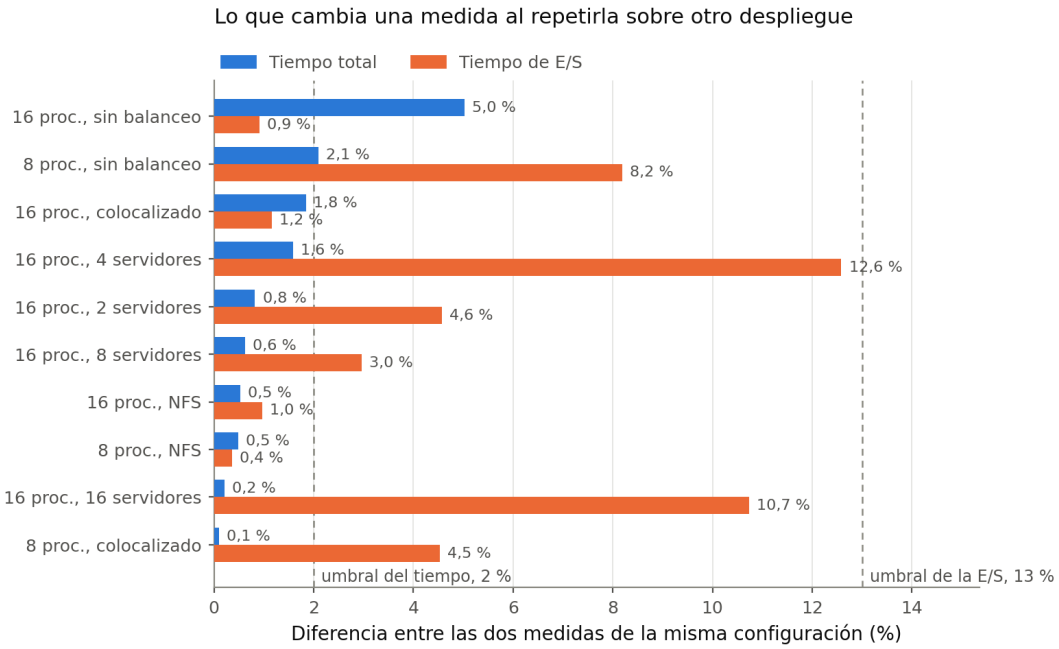


Fig. 5.7. Diferencia entre las dos medidas de cada configuración estática.

De estas cifras se sigue el criterio con el que debe leerse el resto de la sección. A cada configuración se le atribuye la media de sus ejecuciones, tomadas de cuantos despliegues se hayan empleado; al promediar sobre varios despliegues, el desplazamiento de en torno al 1 % que cada uno introduce no se confunde con un efecto de la configuración. Una diferencia entre esas medias se presenta como un resultado solo si supera el 2 %, porque por debajo de esa cifra repetir la misma medida ya produce diferencias de ese tamaño. Entre el 2 y el 5 % la diferencia se da, pero advirtiendo de que podría proceder del entorno, ya que la dispersión entre despliegues llegó a alcanzar el 5 % en una de las diez configuraciones medidas. Por encima del 5 % ninguna repetición ha producido nunca una diferencia semejante, de modo que sí se afirma cuál de las dos configuraciones es la más rápida, aunque no la magnitud exacta de la ventaja. El tiempo de entrada/salida, bastante menos reproducible, exige un umbral más alto: solo se afirman diferencias que superen holgadamente el 13 %.

5.5.2. CE1 — Efecto de la interceptación transparente

La primera pregunta del caso de estudio es de carácter funcional y admite una respuesta binaria. La intercepción que describe la Sección 4.4.3 —la precarga de una biblioteca que desvía hacia XPN las llamadas de entrada/salida que la aplicación dirige al sistema de ficheros— opera sobre EpiGraph: la aplicación se ejecuta sin recompilar ni modificar su código de entrada/salida, y el fichero de estado aparece repartido entre los directorios de datos de los servidores de XPN en lugar de sobre el sistema de ficheros de referencia. La redirección, por tanto, se produce, y se produce de forma transparente para la aplicación. La Sección 5.2.4 describe el defecto que impidió que así fuera durante buena parte de la

experimentación y la corrección que lo resolvió.

Establecido eso, la Tabla 5.10 recoge lo que cuesta cada alternativa. Las cuatro configuraciones son idénticas salvo en el destino de las escrituras, y cada una se presenta con el número de ejecuciones sobre el que se ha promediado.

TABLA 5.10. TIEMPO TOTAL Y TIEMPO DE ENTRADA/SALIDA CON NFS Y CON XPN.

Configuración	**n**	**Tiempo (s)**	Δ	**E/S (s)**	Δ
8 procesos, NFS	5	8307,5	−15,9 %	1017,4	−67,3 %
8 procesos, XPN	6	6986,0		332,8	
16 procesos, NFS	4	6284,2	−25,8 %	1075,2	−78,1 %
16 procesos, XPN	4	4665,9		235,7	

Las dos preguntas que el caso de estudio planteaba por separado reciben la misma respuesta. XPN acelera las escrituras, y lo hace por un margen que no admite discusión: la entrada/salida se reduce a la tercera parte con ocho procesos y a menos de la cuarta con dieciséis, muy por encima del 13 % que exige el observable menos reproducible de los dos. Y esa aceleración sí acorta la ejecución, en un 15,9 y un 25,8 % respectivamente, también holgadamente por encima del umbral del 5 % a partir del cual ninguna repetición ha producido nunca una diferencia semejante.

Que la ventaja crezca con el número de procesos se explica por el comportamiento de cada sistema de ficheros al añadirlos. El tiempo de entrada/salida sobre NFS es prácticamente el mismo con ocho procesos que con dieciséis, 1017,4 frente a 1075,2 segundos, de modo que duplicar los procesos no reduce en absoluto lo que cuesta escribir: el servidor remoto es el mismo y el volumen de datos también. Sobre XPN, en cambio, cae de 332,8 a 235,7 segundos, porque cada proceso añadido incorpora un servidor de almacenamiento sobre el que reparte el volcado. La entrada/salida es la única de las tres partidas que se comporta de manera distinta según el sistema de ficheros, y de ahí que su peso relativo, y con él la ventaja de XPN, aumente con el tamaño de la ejecución.

La Tabla 5.11 recoge esas mismas cuatro medidas comparadas en vertical, es decir, cada sistema de ficheros consigo mismo al pasar de ocho a dieciséis procesos. Duplicar los procesos acorta la ejecución en los dos casos, porque el cómputo sí se reparte entre ellos, pero la entrada/salida solo se abarata sobre XPN. Sobre NFS su variación queda muy por debajo del umbral que exige este observable, de modo que lo que cuesta escribir se mantiene igual con el doble de procesos.

TABLA 5.11. EFECTO DE DUPLICAR EL NÚMERO DE PROCESOS SOBRE CADA SISTEMA DE FICHEROS.

Sistema	**Observable**	**8 procesos**	**16 procesos**	Δ
NFS	Tiempo total (s)	8307,5	6284,2	−24,4 %
	Entrada/salida (s)	1017,4	1075,2	+5,7 %
XPN	Tiempo total (s)	6986,0	4665,9	−33,2 %
	Entrada/salida (s)	332,8	235,7	−29,2 %
Los mismos datos de la Tabla 5.10, comparados ahora entre tamaños de ejecución. El +5,7 % de NFS queda por debajo del umbral del 13 % que la Sección 5.5.1 exige a la entrada/salida, y se lee como ausencia de cambio.				

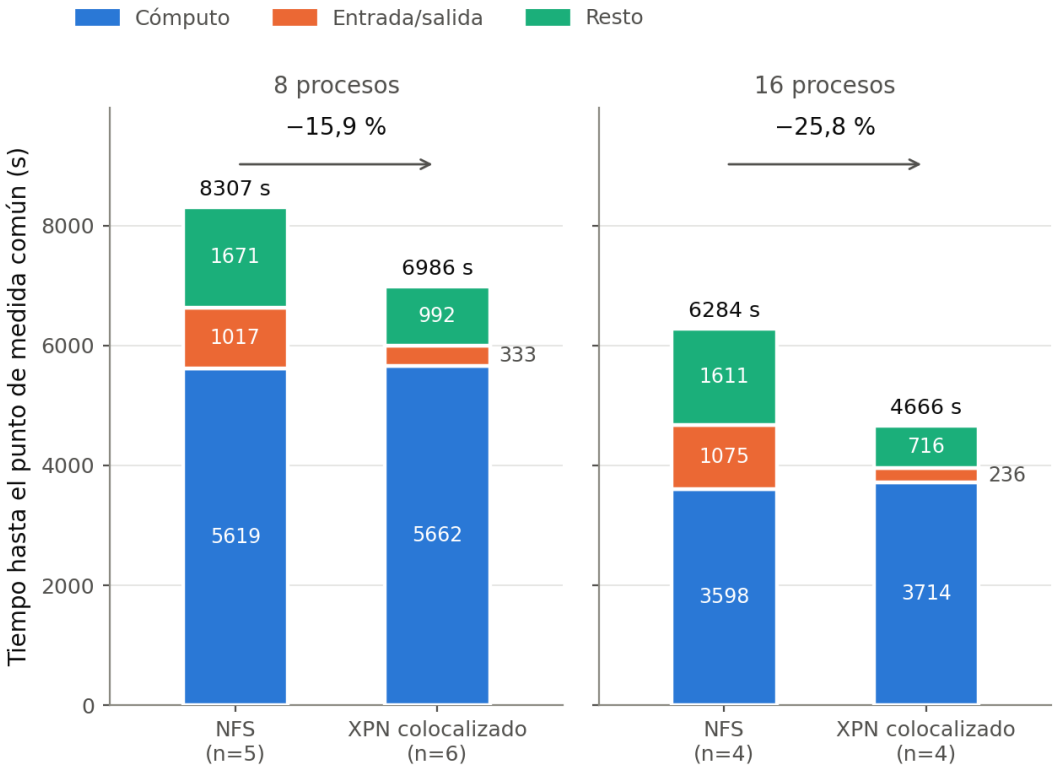


Fig. 5.8. Reparto del tiempo de ejecución entre cómputo, entrada/salida y resto.

La Figura 5.8 descompone esas mismas medidas y matiza la lectura anterior. El cómputo no se mueve, que es lo que cabía esperar de configuraciones que solo difieren en dónde escriben: 5619,2 frente a 5661,5 segundos con ocho procesos, una diferencia del 0,8 % que queda por debajo de lo que produce repetir la medida. Con dieciséis la diferencia llega al 3,2 %, dentro de la banda en la que no puede descartarse el efecto del entorno. El ahorro, en consecuencia, no procede del cómputo.

Procede de las otras dos partidas, y conviene decir en qué proporción. Con ocho procesos la entrada/salida se abarata en 684,6 segundos y el resto en 679,3, casi a partes iguales; con dieciséis, en 839,5 y 895,2 segundos respectivamente. De esas cantidades hay que descontar el ligero crecimiento del cómputo para llegar a los 1321,5 y 1618,3 segundos que XPN ahorra en total. El reparto no es el que cabría esperar. Lo único que distingue a las dos configuraciones es dónde se escribe, de modo que la ventaja debería aparecer entera dentro del cronómetro que envuelve las escrituras; y sin embargo la mitad de ella aparece en la partida que no se mide, sino que se obtiene restando.

La explicación más plausible está en dónde arranca ese cronómetro. La lectura del reloj se toma después de la barrera que sincroniza a los procesos, no antes, de manera que el tiempo que un proceso pasa esperando a los demás queda fuera de la ventana cronometrada y cae del lado del resto. Y hay motivo para que sobre NFS esa espera sea mayor, porque una escritura remota no termina de pagarse cuando la llamada retorna: queda trabajo pendiente en el cliente, en forma de vaciado de las memorias intermedias y de tráfico de red, que se cobra más tarde, cuando el proceso ya ha reanudado el cómputo. Ese coste diferido desincroniza a los procesos entre sí y se manifiesta en la barrera siguiente. La escritura sobre un servidor de XPN alojado en el propio nodo deja mucho menos trabajo de esa clase pendiente. Dicho brevemente, el cronómetro recoge la parte del coste de escribir que se paga en el acto, y con NFS esa parte es menor: una porción mayor del coste se cobra después, ya fuera de la ventana medida.

Conviene subrayar que lo anterior es una interpretación y no una medida. Lo medido es que la partida del resto se abarata en 679,3 segundos; que ese ahorro provenga de la barrera es la lectura más razonable de los datos disponibles, pero el resto se obtiene por diferencia y absorbe todo aquello que la instrumentación no atribuye explícitamente a ninguna de las otras dos partidas, de modo que no cabe descartar que intervenga algún otro factor. Nada de esto afecta a la conclusión del caso de estudio: el 15,9 y el 25,8 % son medidas directas del tiempo transcurrido y no dependen de cómo se reparta por dentro. Lo que queda sujeto a interpretación es dónde se produce el ahorro, no cuánto.

Las dos preguntas de CE1 quedan respondidas, y la segunda con más margen del que la primera hacía esperar. La interceptación funciona sin tocar la aplicación: EpiGraph se ejecuta sin recompilar y sin modificar su código de entrada/salida, y el fichero de estado aparece repartido entre los directorios de los servidores de XPN. Y no se paga por ello, sino al contrario: la entrada/salida se reduce un 67,3 % con ocho procesos y un 78,1 % con dieciséis, y el tiempo total un 15,9 y un 25,8 % respectivamente, todos ellos muy por encima de los umbrales que fija la Sección 5.5.1. Que la ventaja crezca con el tamaño de la ejecución tiene una sola causa, y es la que gobierna también los dos casos de estudio siguientes: la entrada/salida sobre el sistema de ficheros de referencia no se reparte al añadir procesos —1017,4 frente a 1075,2 segundos— mientras que sobre XPN cada proceso añadido incorpora un servidor sobre el que repartir el volcado. Queda una salvedad sobre la que conviene no equivocarse: solo la mitad del ahorro aparece dentro del cronómetro que envuelve las escrituras, y la explicación de la otra mitad —el coste

que NFS difiere hasta la barrera siguiente— es una interpretación y no una medida. Las cifras de tiempo total, que son las que sostienen la conclusión, no dependen de ella.

5.5.3. CE2 — Configuraciones de despliegue del almacenamiento

Establecido que XPN interviene en la entrada/salida de la aplicación, esta subsección responde cómo conviene desplegarlo. Recorre las tres dimensiones que plantea la Sección 5.4.2 —cuántos servidores se despliegan, dónde se sitúan respecto de los procesos de cómputo y cómo se reparten cuando intervienen dos máquinas— y termina con el tamaño de bloque, que se evalúa aparte porque no es una decisión de despliegue sino de configuración interna del sistema de ficheros. Todas ellas son los parámetros que introduce la Sección 4.4.3: el número de servidores entre los que se reparten los bloques de cada fichero, su ubicación colocalizada o exclusiva respecto de los procesos de cómputo, y el tamaño de esos bloques.

Las cifras del barrido proceden de la campaña de repeticiones, que mide dos veces cada celda dentro de un mismo despliegue y junto a su ejecución de referencia, de manera que ninguna de las comparaciones que siguen mezcla despliegues. Junto a cada tiempo aparece el rango de las dos ejecuciones, que es lo que permite juzgar si una diferencia es afirmable. Por ese motivo las dos configuraciones que esta subsección comparte con la Sección 5.5.2 aparecen aquí con un valor ligeramente distinto: allí se promedia sobre todas las ejecuciones válidas, procedan del despliegue que procedan, y aquí solo sobre las de este lote. Las diferencias no llegan al 1,1 %, por debajo del ruido entre despliegues que cuantifica la Sección 5.5.1, y la elección responde a que un barrido solo es interpretable si todas sus celdas comparten despliegue.

Número de servidores

La Tabla 5.12 recoge los dos barridos completos, con la descomposición del tiempo en las tres partidas que define la Sección 5.4.2.

TABLA 5.12. BARRIDO DEL NÚMERO DE SERVIDORES DE ALMACENAMIENTO SOBRE UN ÚNICO NODO, CON LA DESCOMPOSICIÓN DEL TIEMPO TOTAL.

Configuración	**n**	**Tiempo (s)**	**Cómputo (s)**	**E/S (s)**	**Coord. (s)**
Ocho procesos de cómputo					
Referencia (NFS)	2	8252,0 [8243,8–8260,2]	5546,8	1021,9	1683,3
2 servidores dedicados	2	7123,6 [7034,0–7213,3]	5516,5	544,1	1063,0
4 servidores dedicados	2	7126,3 [7102,7–7150,0]	5609,9	515,4	1001,0
8 servidores dedicados	2	7025,9 [7007,9–7043,8]	5623,1	419,6	983,1
Colocalizado (8 serv.)	2	6975,3 [6962,4–6988,2]	5616,9	335,3	1023,1
Dieciséis procesos de cómputo					
Referencia (NFS)	2	6287,3 [6258,3–6316,3]	3627,8	1086,4	1573,2
2 servidores dedicados	2	4907,4 [4894,8–4920,1]	3582,4	478,8	846,3
4 servidores dedicados	2	4913,8 [4904,4–4923,3]	3754,8	432,0	727,0
8 servidores dedicados	2	4785,4 [4749,1–4821,8]	3642,9	314,0	828,5
16 servidores dedicados	2	4706,5 [4689,1–4723,9]	3634,7	264,9	806,9
Colocalizado (16 serv.)	2	4658,8 [4636,2–4681,5]	3747,8	237,4	673,6

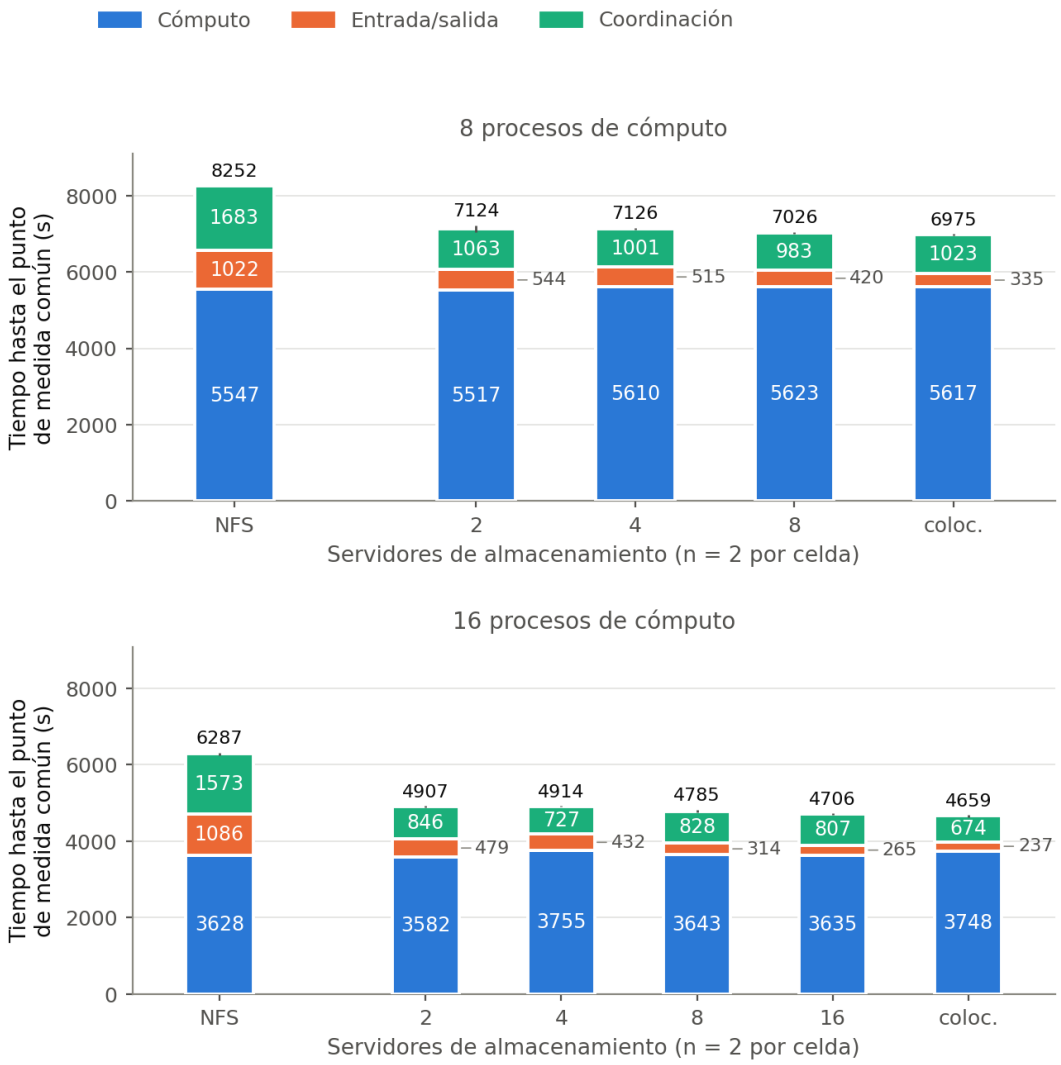


Fig. 5.9. Reparto del tiempo de ejecución según el número de servidores de almacenamiento.

La Figura 5.9 descompone esas mismas medidas —el remate de cada barra recoge el rango de las dos ejecuciones— y deja ver el resultado central del barrido antes de entrar en las cifras: el segmento de entrada/salida se encoge en cada escalón, y sin embargo la altura de la barra entera apenas cambia, porque el cómputo, que ocupa la mayor parte y no distingue configuraciones, fija esa altura.

El tiempo de entrada/salida ordena con el número de servidores y lo hace sin excepción. En las ejecuciones de ocho procesos cae de 544,1 a 335,3 segundos al pasar de dos servidores a ocho colocalizados, un 38,4 %; en las de dieciséis, de 478,8 a 237,4, un 50,4 %. Cada paso del barrido reduce esa partida, y ninguno la aumenta. Es el resultado que cabía esperar, ya que ampliar el número de servidores reparte cada escritura entre más destinos, y conviene subrayarlo porque es la única de las tres partidas que responde con esa regularidad.

Ese descenso, sin embargo, apenas se nota en el tiempo total. Recorrer el barrido entero, de dos servidores dedicados al modo colocalizado, ahorra 148 segundos en las eje-

cuciones de ocho procesos y 249 en las de dieciséis, un 2,1 y un 5,1 % sobre ejecuciones que duran casi dos horas y hora y veinte respectivamente. Entre dos escalones contiguos la diferencia es todavía menor: pasar de dos a cuatro servidores cambia el total en 2,7 segundos con ocho procesos y en 6,4 con dieciséis. Con el criterio de la Sección 5.5.1, el recorrido del barrido de ocho procesos se queda en el umbral y no permite atribuir nada al número de servidores, mientras que el de dieciséis lo supera.

La razón está en el peso que la entrada/salida tiene dentro del total, y la cuenta puede hacerse antes de mirar las medidas. Con dos servidores dedicados esa partida ocupa el 7,6 % del tiempo a ocho procesos y el 9,8 % a dieciséis; el resto se lo reparten el cómputo, que ronda el 75 %, y la coordinación. Si algo que pesa una décima parte del total se reduce a la mitad, el total no puede bajar más de un cinco por ciento. Eso es exactamente lo que ocurre: en el barrido de dieciséis procesos se ahorran 241 segundos de escritura y el total baja 249. En el de ocho se ahorran 209 y el total baja 148, y la diferencia tiene también explicación, ya que el cómputo de la configuración colocalizada resultó 100 segundos mayor que el de la de dos servidores. Esa oscilación del cómputo, que no sigue en ningún caso al número de servidores —entre 5516,5 y 5623,1 segundos con ocho procesos, entre 3582,4 y 3754,8 con dieciséis—, es la comprobación de que el barrido mide lo que pretende, ya que ninguna de estas configuraciones cambia el trabajo que hay que hacer, solo dónde se escribe.

Duplicar el número de procesos acentúa todo lo anterior, y por un solo motivo. El cómputo se reparte entre más procesos y cae de 5516,5 a 3582,4 segundos, mientras que la escritura no se reparte y apenas baja, de 544,1 a 478,8. Al encoger una partida y no la otra, la entrada/salida gana peso dentro del total, y con ella lo gana todo aquello que la abarate. Por eso el recorrido del barrido pasa del 2,1 al 5,1 %, y por eso la ventaja de XPN sobre el sistema de ficheros de referencia crece del 15,5 % de tiempo a ocho procesos al 25,9 % a dieciséis, con la entrada/salida un 67,2 % y un 78,1 % menor. La entrada/salida de la referencia, en cambio, es prácticamente la misma en las dos series, 1021,9 y 1086,4 segundos, pese a que la segunda dispone del doble de procesos.

Ubicación de los servidores

Comparar sin más el modo colocalizado con el exclusivo confundiría la ubicación con el número de servidores, porque en el primero cada proceso de cómputo aloja el suyo. Las dos parejas que aíslan la ubicación son las que igualan ese número: ocho servidores dedicados frente al colocalizado de ocho procesos, y dieciséis dedicados frente al colocalizado de dieciséis.

En las dos, la ubicación resulta indiferente para el tiempo total. A ocho procesos la diferencia es de 7025,9 frente a 6975,3 segundos, un 0,7 %; a dieciséis, de 4706,5 frente a 4658,8, un 1,0 %. Las dos quedan por debajo del umbral de la Sección 5.5.1, aunque los rangos de las dos ejecuciones no lleguen a solaparse en ninguno de los dos casos; con dos repeticiones por celda el rango observado subestima la dispersión real, y es el umbral,

y no el rango, lo que decide en diferencias de este tamaño. Lo que sí cambia es la entrada/salida, y de forma apreciable: colocalizar la reduce un 20,1 % a ocho procesos (419,6 frente a 335,3 segundos) y un 10,4 % a dieciséis (264,9 frente a 237,4). La explicación es directa, ya que en el modo colocalizado una fracción de las escrituras encuentra su destino en el propio contenedor y no llega a salir de él.

Que esa ventaja no llegue al tiempo total es coherente con lo anterior: la entrada/salida pesa poco dentro del total, de modo que reducirla una quinta parte apenas deja rastro en el reloj. Tampoco se aprecia en la descomposición el sobrecoste de cómputo que la Sección 5.4.2 anticipaba para el modo colocalizado, por competir cada servidor con su proceso por el mismo núcleo. Ese efecto, de existir, no resulta observable a esta escala. Los dos modos terminan empatados en tiempo, y la elección entre ellos no se decide ahí.

Se decide en lo que cada uno permite dimensionar. El modo colocalizado obtiene su ventaja precisamente porque cada servidor vive junto al proceso que escribe, y esa misma propiedad es la que lo limita: el número de servidores queda fijado por el de procesos de cómputo, tal como advierte la Sección 5.4.2. No cabe añadir capacidad de almacenamiento sin añadir cómputo, ni a la inversa. El modo exclusivo deja libres las dos dimensiones y permite fijar el número de servidores con independencia del de procesos, de manera que es el único de los dos que puede seguir creciendo por el lado del almacenamiento en escenarios donde la entrada/salida pese más que en este. El colocalizado ofrece, en suma, la mejor entrada/salida a igualdad de servidores, y el exclusivo, la capacidad de escalarla.

Topologías multinodo

Cuando la ejecución ocupa dos máquinas aparece un factor que las topologías de un solo nodo no tienen: qué fracción de las escrituras atraviesa la red. Las dos disposiciones que se comparan son las de la Figura 5.2(c) y (d). En la primera cada máquina aloja un único tipo de contenedor, de manera que todos los servidores de almacenamiento residen en un nodo separado del que ejecuta los procesos de cómputo y ninguna escritura puede resolverse sin salir de la máquina que la origina; en la segunda ambos tipos se reparten a partes iguales entre los dos nodos, de modo que cada proceso encuentra servidores en su propia máquina. La Tabla 5.13 las enfrenta a igual número de servidores, que es la comparación que aísla ese factor. Su última fila añade el modo colocalizado llevado a dos máquinas, con los procesos repartidos ocho por nodo y un servidor en cada contenedor; es el caso extremo de la segunda disposición, y por eso no tiene homólogo en la primera.

TABLA 5.13. DISPOSICIÓN DE LOS SERVIDORES DE ALMACENAMIENTO ENTRE DOS NODOS, CON DIECISÉIS PROCESOS DE CÓMPUTO. EN UNA MÁQUINA LOS REÚNE EN UN NODO SIN CÓMPUTO; REPARTIDOS, EN AMBOS.

	En una máquina		**Repartidos**		
Servidores	**Tiempo (s)**	**E/S (s)**	**Tiempo (s)**	**E/S (s)**	**Δ tiempo**
2	5586,2	1141,1	4890,5	597,6	−12,5 %
4	5563,0	1138,1	4922,5	567,8	−11,5 %
8	5584,4	1139,0	4717,2	442,0	−15,5 %
16	5610,5	1145,5	4776,4	431,8	−14,9 %
Colocalizado	—	—	4757,4	412,5	—

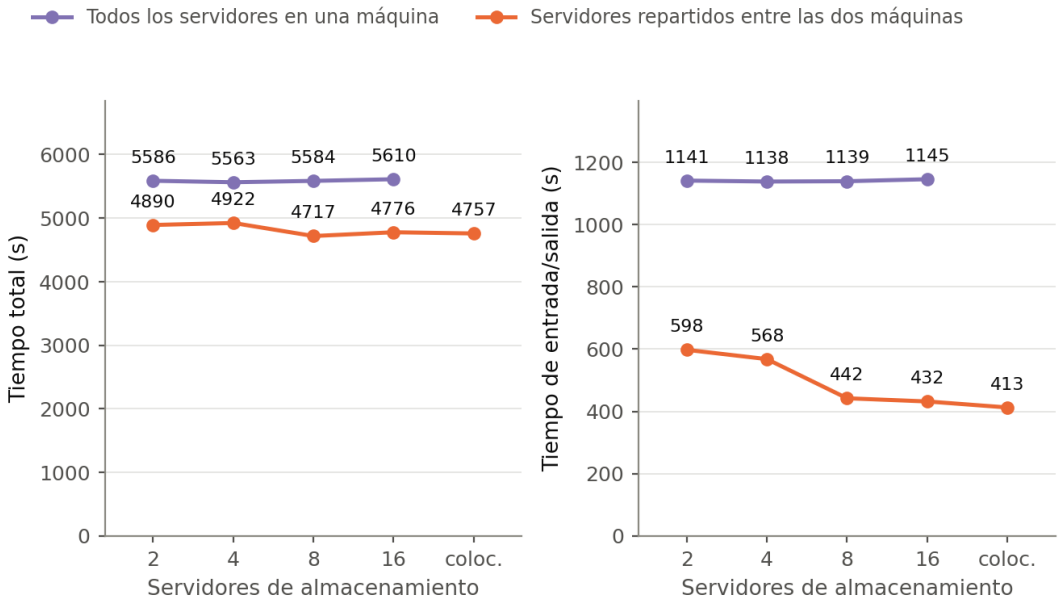


Fig. 5.10. Tiempo total y tiempo de entrada/salida de las dos disposiciones multinodo.

La Figura 5.10 enfrenta las dos series. La disposición que reúne todos los servidores en una única máquina es peor que la distribuida en las cuatro configuraciones, por márgenes de entre el 11,5 y el 15,5 % de tiempo, y repartir los servidores entre las dos máquinas reduce además la entrada/salida entre un 47,6 y un 62,3 %. Y no mejora al añadir servidores: las cuatro configuraciones caben en un intervalo del 0,8 % en tiempo y del 0,6 % en entrada/salida, de modo que multiplicar por ocho el número de servidores no cambia nada. Ésa es la señal característica de un cuello de botella que no está en los servidores, ya que toda la entrada/salida atraviesa el mismo enlace de red y ese enlace ya está saturado con dos. La disposición distribuida, en cambio, sí ordena con el número de

servidores —de 597,6 a 412,5 segundos— como ocurría en un solo nodo, lo que confirma que el cuello era el enlace y no el almacenamiento.

La lectura para el diseño es que el coste de repartir una ejecución entre dos máquinas no lo impone el número de máquinas, sino la fracción de escrituras que se ve obligada a cruzar la red. Situar servidores en ambos nodos deja esa fracción en manos del reparto de datos, y con ello recupera la mayor parte de la diferencia.

Conviene acotar el alcance de estas cifras. Las nueve configuraciones se midieron con una instrumentación distinta de la del barrido de un solo nodo: el contador de operaciones en coma flotante no estuvo disponible, de modo que el balanceador de carga trabajó sin su entrada principal. La circunstancia es común a las nueve, por lo que no afecta a las comparaciones de este apartado, que se establecen entre ellas, pero sí desaconseja enfrentar estos tiempos a los de la Tabla 5.12, tomados con el contador operativo: el balanceador que aísla el caso de estudio de la Sección 5.4.4 altera el tiempo total en esta población por un margen del mismo orden que el que se discute aquí, de manera que cuantificar el sobrecoste absoluto de la ejecución multinodo exigiría medir estas celdas en las condiciones de las demás. Lo que sí puede afirmarse, y es lo que sostiene el apartado, es cuál de las dos disposiciones conviene y por qué.

Tamaño de bloque

El último parámetro es el grano con el que XPN entrelaza los datos entre sus servidores. La Tabla 5.14 repite con 1024 KiB las nueve configuraciones de un solo nodo medidas con el valor por defecto de 512 KiB.

TABLA 5.14. TAMAÑO DE BLOQUE DE 512 FRENTE A 1024 KIB, SOBRE LAS NUEVE CONFIGURACIONES DE UN SOLO NODO. UNA EJECUCIÓN POR CELDA.

Configuración	**512 KiB (s)**	**1024 KiB (s)**	Δ	**E/S 512**	**E/S 1024**
8p, 2 servidores	7060,4	7224,9	+2,3 %	542,3	540,8
8p, 4 servidores	7165,4	7179,4	+0,2 %	533,4	535,4
8p, 8 servidores	7031,8	7154,2	+1,7 %	418,6	428,8
8p, colocalizado	6951,8	6977,9	+0,4 %	325,4	326,6
16p, 2 servidores	4909,0	4811,6	−2,0 %	486,2	488,1
16p, 4 servidores	4893,1	4892,3	0,0 %	484,5	478,9
16p, 8 servidores	4725,3	4753,1	+0,6 %	313,6	295,0
16p, 16 servidores	4727,7	4671,8	−1,2 %	299,8	302,9
16p, colocalizado	4629,8	4563,0	−1,4 %	232,5	235,5

Duplicar el tamaño de bloque no produce ningún efecto. El tiempo total se mueve entre un −2,0 y un +2,3 %, sin signo consistente: en las cuatro configuraciones de ocho procesos el grano mayor sale más lento, en media un 1,1 %, y en las cinco de dieciséis más rápido, en media un 0,8 %. Con una sola ejecución por celda y un umbral de ruido del 2 %, ninguna de las nueve diferencias es afirmable.

La entrada/salida, que es el observable directo y el que debería responder a un cambio de grano, tampoco se mueve: las nueve parejas difieren menos de un 6 %, y ese máximo lo alcanza una sola de ellas —dieciséis procesos con ocho servidores—, mientras que las ocho restantes se quedan por debajo del 3 %. Que nueve comparaciones independientes coincidan en no mostrar efecto es un argumento más sólido que cualquiera de ellas por separado, de modo que el resultado puede darse por establecido pese a la falta de repeticiones.

La explicación probable es que el tamaño de los volcados de EpiGraph es muy superior a los dos valores ensayados, de manera que ambos entrelazan de todas formas cada escritura entre todos los servidores disponibles y el grano deja de ser la variable que decide. Por ese motivo el resto de las campañas de este trabajo emplea el valor por defecto de 512 KiB, que no es peor que la alternativa y evita introducir una diferencia respecto de la configuración de referencia de la distribución.

Recapitulación

Las tres decisiones de despliegue no pesan lo mismo, y la que más pesa no es la que gobierna la entrada/salida. El número de servidores la ordena de forma monótona —hasta un 50,4 % menos entre los extremos del barrido de dieciséis procesos— pero traslada al tiempo total una fracción pequeña de esa mejora, un 5,1 % en el mejor caso, porque la entrada/salida es una partida menor del tiempo de la simulación. La ubicación no cambia el tiempo total en ninguno de los dos tamaños ensayados: colocalizar abarata la entrada/salida entre un 10 y un 20 % a igualdad de servidores, pero esa ventaja se diluye por la misma razón, de modo que la elección entre los dos modos no se decide en el reloj sino en lo que cada uno permite dimensionar, y solo el exclusivo puede crecer por el lado del almacenamiento sin añadir cómputo. Y la disposición entre máquinas resulta ser la decisión más costosa de todas: a igual número de servidores, reunirlos en un nodo separado del cómputo cuesta entre un 11,5 y un 15,5 % frente a repartirlos entre las dos máquinas, porque obliga a toda la entrada/salida a atravesar la red. El tamaño de bloque, evaluado aparte, no produce ningún efecto medible entre los dos valores ensayados.

5.5.4. CE3 — Maleabilidad del número de procesos

La reconfiguración funciona por los dos mecanismos y a todas las escalas ensayadas, de modo que la pregunta interesante no es si puede hacerse, sino cuándo compensa. Esta subsección recorre en ese orden el mecanismo, la magnitud del cambio, el momento en

que se solicita y su dirección, y termina con la campaña de co-maleabilidad. Todas las cifras están tomadas en el punto de medida común que fija la Sección 5.5.1, y junto a cada configuración aparece el número de ejecuciones sobre el que se ha promediado. Conviene anticipar que buena parte de las celdas de esta campaña son de una sola ejecución: cada una ocupa entre una y tres horas de un nodo completo, y las repeticiones se concentraron allí donde la diferencia que se quería afirmar era pequeña.

Los dos mecanismos

La Tabla 5.15 compara las dos vías por las que puede materializarse un cambio en el número de procesos. La primera es el ciclo de *checkpoint* y reinicio que detalla la Sección 4.4.4: la aplicación vuelca su estado y termina, y se relanza con el nuevo número de procesos, que reparten de nuevo los individuos en bloques contiguos y reconstruyen su estado local a partir del volcado. La segunda es el modo dinámico de FlexMPI (Sección 4.4.2), que incorpora los procesos sin abandonar la ejecución. Ambas se ensayan sobre la misma máquina y el mismo calendario, de manera que el mecanismo es la única variable. Se ensayan dos calendarios: uno que expande una sola vez y otro que alterna cuatro veces entre los dos tamaños, de modo que el segundo paga el coste del mecanismo cuatro veces en lugar de una.

TABLA 5.15. TIEMPO DE EJECUCIÓN CON CADA UNO DE LOS DOS MECANISMOS DE RECONFIGURACIÓN.

Calendario	**n**	**Reinicio (s)**	**Modo dinámico (s)**	Δ
$4 \rightarrow 8$	1	1359,2	1112,6	+22,2 %
$4 \rightarrow 8 \rightarrow 4 \rightarrow 8 \rightarrow 4$	1	1277,5	1138,2	+12,2 %
El primer calendario reconfigura una vez, en el día 41; el segundo cuatro, en los días 41, 65, 81 y 110.				

El modo dinámico es más rápido en los dos calendarios, y por un margen que en ambos casos supera con holgura la dispersión entre ejecuciones. La diferencia, además, está subestimada. El ciclo de *checkpoint* y reinicio no produce una única ejecución sino una por fase, cada una con su propio reloj, y el tiempo que aquí se le atribuye es la suma de los tramos de cómputo; queda fuera lo que cuesta terminar la ejecución anterior, lanzar la siguiente y reconstruir la partición a partir del volcado. Las cifras de la primera columna son, por tanto, una cota inferior del coste real del mecanismo. La comparación se realizó con la población reducida sobre una sola máquina, así que sus valores absolutos no son comparables con los del resto de la subsección; lo que se contrasta es la diferencia entre columnas.

Ese sobrecoste, unido a que la redistribución del estado tras el reinicio no llegó a completarse con la población completa sobre el clúster (Sección 5.4.3), es lo que llevó a emplear únicamente el modo dinámico en las campañas que siguen.

Magnitud del cambio

Fijado el mecanismo, la Tabla 5.16 sitúa las expansiones solicitadas el mismo día entre las configuraciones estáticas que las acotan: la del número de procesos de partida y la del pico. Va dividida en dos bloques, uno por cada campaña, por los motivos que se exponen más abajo; el análisis se apoya en el primero y recurre al segundo únicamente para el tramo que aquel no cubre.

TABLA 5.16. EXPANSIONES DE DISTINTA MAGNITUD, TODAS SOLICITADAS EL DÍA 45, FRENTE A LAS CONFIGURACIONES ESTÁTICAS QUE LAS ACOTAN. ENTRE CORCHETES, EL INTERVALO OBSERVADO.

Configuración	**n**	**Tiempo (s)**	**Núcleos × hora**
Campaña principal			
8 procesos, estática	2	7965,2 [7934,6–7995,8]	22,13
16 procesos, estática	2	5510,9 [5483,8–5538,1]	27,55
8 → 16 procesos	2	5575,2 [5573,1–5577,3]	26,03
8 → 32 procesos	2	3843,0 [3840,2–3845,7]	30,75
8 → 64 procesos (alcanza 32)	2	3866,0 [3861,7–3870,2]	30,85
64 procesos, estática	2	3333,8 [3331,6–3335,9]	61,12
Campaña anterior			
8 → 16 procesos	1	5669,8	26,54
8 → 32 procesos	1	3886,6	31,21
8 → 64 procesos (alcanza 60)	1	3523,0	48,81
64 procesos, estática	1	3353,8	61,49
Los núcleos × hora incluyen aquí los dos servidores de almacenamiento, a diferencia de la Tabla 5.18, donde son solo los de la aplicación.			

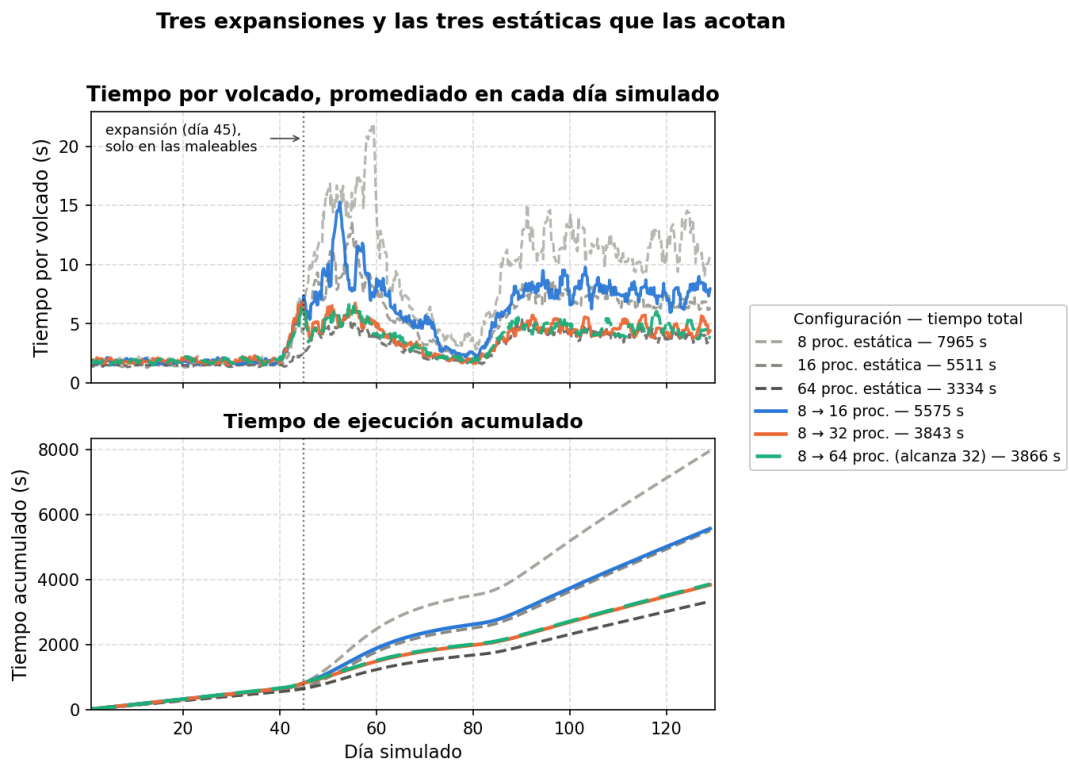


Fig. 5.11. Las tres expansiones y las tres estáticas que las acotan.

Las filas extremas del bloque principal delimitan lo que cabe esperar de cualquier calendario intermedio, y las centrales caen dentro. La estática de ocho procesos fija la cota superior, la de quien nunca amplía: expandir a dieciséis el día cuarenta y cinco recorta un 30,0 % de ese tiempo a cambio de un 17,6 % más de recursos. Frente a la estática de dieciséis, en cambio, expandir hasta ese mismo número cuesta un 1,2 % de tiempo y ahorra un 5,5 %. Ese sobrecoste queda en el umbral de la Sección 5.5.1, pero los intervalos de las dos configuraciones no llegan a solaparse, de modo que el signo es consistente aunque la magnitud sea pequeña. El mecanismo, en otras palabras, no regala tiempo: lo que devuelve es la parte de la ejecución que no necesitaba estar dimensionada al máximo.

El rendimiento de ampliar el destino decae rápido. Pasar de dieciséis a treinta y dos procesos recorta un 31,1 % del tiempo a cambio de un 18,1 % más de recursos. El tramo siguiente lo mide el segundo bloque de la tabla, cuya expansión es la única que llegó cerca de sesenta y cuatro procesos. Sus cifras se contrastan únicamente entre sí, sin cruzarlas con las del bloque principal: pasar de treinta y dos a sesenta recorta un 9,4 % del tiempo y cuesta un 56,4 % más de recursos, y frente a la estática de sesenta y cuatro de esa misma campaña la expansión sale un 5,0 % más lenta por un 20,6 % menos de recursos.

Los dos bloques se mantienen separados por dos motivos. El primero es de método: proceden de despliegues distintos, y la Sección 5.5.1 cuantifica en torno a un 1 % el desplazamiento sistemático que introduce cambiar de despliegue, de manera que cruzar celdas de uno y otro confundiría ese desplazamiento con el efecto que se quiere medir. Las dos estáticas de sesenta y cuatro procesos, que son la misma configuración medida

en cada campaña, dan la escala de esa diferencia: 3333,8 y 3353,8 segundos, un 0,6 % que concuerda con el ruido de despliegue. El segundo motivo es que la magnitud que alcanza una expansión no resulta reproducible. La creación de procesos se realiza por lotes sucesivos, y el mismo destino solicitado —sesenta y cuatro procesos— se resolvió en sesenta en aquella campaña y en treinta y dos en las dos repeticiones de esta, sin que mediara ningún cambio de configuración. La consecuencia se lee en la tabla: las dos repeticiones que se quedaron en treinta y dos procesos son indistinguibles de la expansión que pedía treinta y dos, con un 0,6 % de diferencia en tiempo y un 0,3 % en recursos. Lo que gobierna el resultado es el número de procesos que llegan a entrar, no el que se solicita, y esa incompletitud es también la que más adelante impide encoger desde el destino mayor.

La Figura 5.11 muestra ese rendimiento decreciente sin necesidad de acudir a los totales, y añade algo que la tabla no dice. Arriba recoge el coste de cada tramo de ciento cuarenta y cuatro iteraciones promediado por día simulado, y abajo el tiempo acumulado; las estáticas de referencia van en gris. Cada serie es la media de las dos ejecuciones de su celda, salvo la expansión que alcanzó sesenta procesos, que no aparece por proceder de otro despliegue, y la línea de puntos marca el día en que se solicita el cambio. Las tres expansiones son indistinguibles entre sí hasta el día en que se solicita el cambio, como corresponde a configuraciones que hasta ese instante son idénticas, y a partir de ahí la de dieciséis procesos se separa con claridad de las otras dos, que recorren superpuestas el resto de la simulación. Esa superposición no es una casualidad de la escena: las dos terminan con el mismo número de procesos, y por eso la serie de sesenta y cuatro se dibuja discontinua, para que no oculte a la otra. La separación con que terminan las curvas del panel inferior es, precisamente, la diferencia de tiempo total que recoge la tercera columna de la tabla.

Las estáticas de dieciséis y de sesenta y cuatro procesos, en cambio, corren por debajo de todas las maleables durante los primeros cuarenta y cinco días, porque ya disponen de todos sus procesos mientras las otras trabajan todavía con ocho. Desde la expansión, la de dieciséis se superpone a la maleable de su mismo tamaño, y ahí se ve sin necesidad de ningún cálculo por qué ninguna expansión puede batir en tiempo a la estática equivalente: lo que se acumula en la fase inicial ya no se recupera después, porque a partir del cambio las dos avanzan al mismo ritmo. Y se ve también que esa diferencia es pequeña, precisamente porque la fase inicial es la barata de la simulación. La estática de ocho procesos hace el papel contrario: acompaña a las maleables mientras ninguna ha ampliado y se despega de todas en cuanto lo hacen, de modo que la distancia que abre a partir del día cuarenta y cinco es exactamente lo que la ampliación compra.

Ese primer tramo permite además cuantificar un supuesto sobre el que se apoya todo el caso de estudio (Sección 5.4.3): mientras la epidemia no arranca, el número de procesos dedicados al cómputo es indiferente. Las dos estáticas recorren los cuarenta primeros días en 561,9 y 560,6 segundos, una diferencia del 0,23 %, y lo hacen con dieciséis y con sesenta y cuatro procesos respectivamente: cuadruplicar el cómputo no cambia abso-

lutamente nada, porque en esa fase apenas hay cómputo que repartir. Conviene precisar que la indiferencia empieza en dieciséis y no antes, ya que las tres maleables, que ahí corren con ocho, tardan 654,2 segundos de media, un 16,6 % más, y la estática de ocho procesos recorre ese mismo tramo en 665,3 segundos. Y conviene precisar también que ese tramo pesa entre la décima y la sexta parte de la ejecución según la configuración. En eso descansa el margen de la maleabilidad: los recursos que no se reservan durante esos días no se echan en falta, y por eso una expansión situada al comienzo de la primera onda no llega tarde.

Dónde deja exactamente de compensar ampliar es una pregunta que estas tres ejecuciones no pueden responder, porque todas expanden el mismo día y ninguna se detiene entre treinta y dos y sesenta y cuatro procesos. Se retoma más adelante, con las ejecuciones que sí la miden.

Momento de la reconfiguración

La Tabla 5.17 recoge el barrido del día en que se solicita la expansión, con el punto de partida y el de destino fijos.

TABLA 5.17. EFECTO DEL DÍA SIMULADO EN QUE SE SOLICITA LA EXPANSIÓN, CON DESTINO FIJO EN CUARENTA PROCESOS.

Día	n	Tiempo (s)	E/S (s)	Núcleos × hora
30	3	3439,3 [3395,3–3463,1]	364,5	34,61
40	6	3424,8 [3382,2–3482,2]	396,8	33,45
50	3	3802,3 [3774,2–3824,8]	442,8	31,41

El barrido no es simétrico. Adelantar la expansión diez días no cambia nada: los rangos del día treinta y del día cuarenta se solapan por completo, de modo que la diferencia no es distinguible de la variabilidad propia de la configuración. Retrasarla otros diez sí cuesta, un 11,0 % de tiempo sobre la media del día cuarenta, a cambio de un 6,1 % menos de recursos. La asimetría tiene una explicación directa en la tercera columna: el tiempo de entrada/salida crece de forma monótona al retrasar la expansión, de 364,5 a 442,8 segundos, porque cuantos más días se recorren con ocho procesos más volcados se pagan al ritmo de la configuración pequeña. Adelantarse solo añade procesos durante una fase que apenas cuesta; llegar tarde deja sin atender la parte cara de la simulación.

Se ensayó también la operación contraria al final de la ejecución, contrayendo a dieciséis procesos en el día ciento veinte. El resultado descarta esa política: la ejecución tarda 3596,9 segundos frente a los 3424,8 de media sin esa contracción, un 5,0 % más, y el ahorro de recursos de aplicación es de apenas un 2 %. Los últimos días de la meseta final son caros en cómputo, y recortar ahí sale mal por las dos vías.

Dirección de los cambios

Hasta aquí todas las reconfiguraciones han sido expansiones. Queda por ver si además de crecer conviene encoger, y para responderlo hay que distinguir dos preguntas que se parecen mucho pero no son la misma.

La primera es qué pasa si a un calendario se le añade una contracción. La respuesta no tiene mérito: cuesta tiempo y ahorra recursos. El perfil que expande a dieciséis procesos y vuelve a ocho tarda 6794,8 segundos y consume 23,25 núcleos × hora, frente a los 5669,8 y 26,54 que midió sobre ese mismo despliegue el que expande a dieciséis y ya no se mueve —segundo bloque de la Tabla 5.16—, un 19,8 % más de tiempo por un 12,4 % menos de recursos. Eso no dice nada del mecanismo ni de la política: la ejecución contraída recorre la mitad final de la simulación con la mitad de procesos, y tarda lo que corresponde a ese tamaño.

La segunda pregunta, que es la interesante, parte de un hecho de la simulación: no todos los días cuestan lo mismo. La epidemia tiene una fase inicial barata, una primera onda cara, un valle en el que la actividad decae y una meseta final que vuelve a ser cara. Si el coste cambia a lo largo de la ejecución, cabe preguntarse si conviene que el número de procesos cambie con él. Eso es lo que compara la Figura 5.12, y lo hace enfrentando dos maneras de gastar aproximadamente los mismos recursos. Sus tres paneles recogen, de arriba abajo, lo que cuesta avanzar un día simulado, el tiempo acumulado y los procesos realmente activos, con las cuatro fases de la escena sombreadas al fondo.

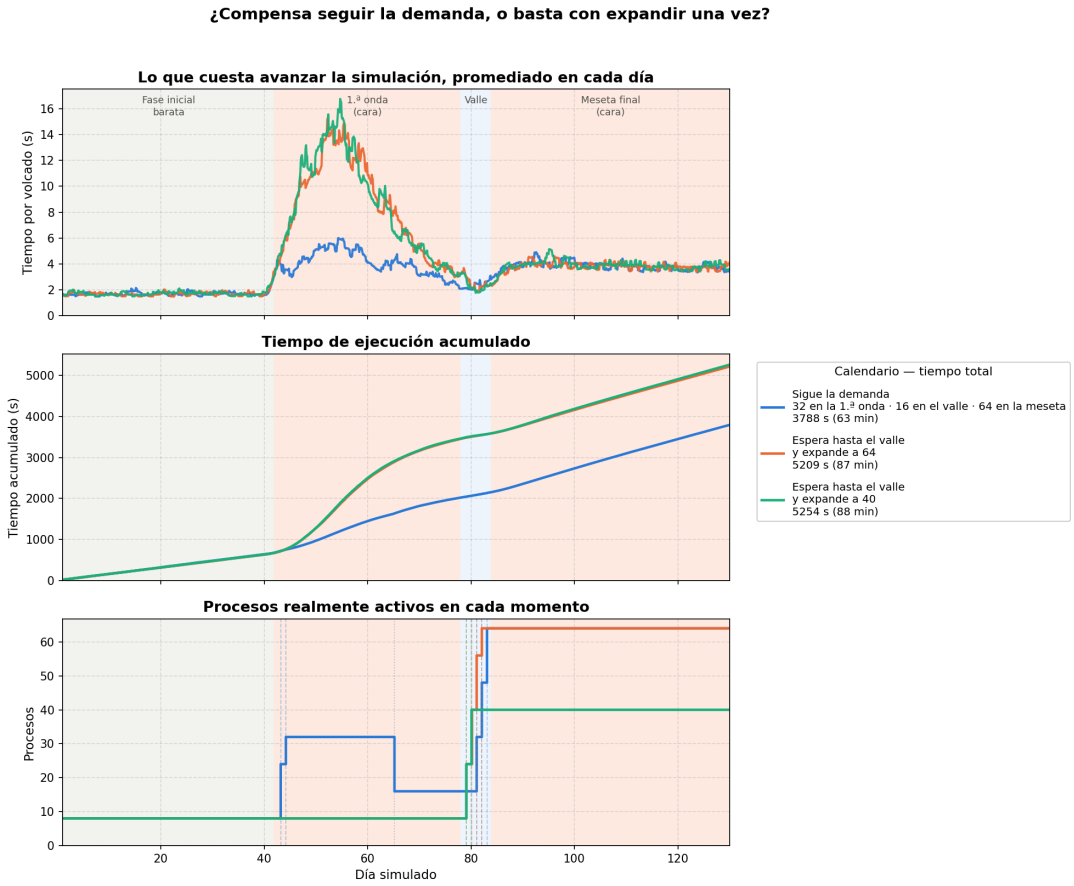


Fig. 5.12. Tres calendarios de expansión sobre la misma simulación.

El panel superior es el que explica el resultado. Los tres calendarios son indistinguibles mientras la epidemia no arranca. Se separan en la primera onda, y de forma brutal: los dos que siguen con ocho procesos llegan a costar entre catorce y dieciséis segundos por volcado, mientras que el que ya ha expandido a treinta y dos se queda en torno a cinco. Durante unos treinta y cinco días simulados, uno avanza tres veces más deprisa que los otros. Ahí se decide la ejecución: en el panel central, la separación entre curvas nace justo en ese tramo y después ya no se recupera, porque a partir del valle los tres corren con un número de procesos parecido.

Los totales lo confirman. El calendario que sigue la demanda —expande para la primera onda, se contrae en el valle y vuelve a expandir para la meseta final— tarda 3757,4 segundos y consume 41,02 núcleos × hora, frente a los 5178,9 y 37,46 del que espera al valle para expandir al mismo pico: un 27,4 % menos de tiempo por un 9,5 % más de recursos. Ambas cifras de consumo son las de los procesos de la aplicación, como las de la Tabla 5.18, de la que procede la segunda.

Conviene ser preciso con qué produce esa ganancia, porque el papel de la contracción se presta a confusión. Lo que hace ganar tiempo es haber expandido pronto, no haber encogido: encoger, por sí solo, siempre ralentiza. Lo que aporta la contracción es que esa expansión temprana no se pague durante todo el resto de la ejecución. Sin ella, los treinta

y dos procesos seguirían reservados durante los días del valle, en los que no hacen falta, y el consumo de recursos se dispararía muy por encima de ese 9,5 %. Dicho de otro modo, expandir pronto es lo que da la velocidad y encoger a tiempo es lo que la hace asequible.

Cuánto ampliar depende de cuándo se amplía

Los dos apartados anteriores han tratado por separado el tamaño del destino y el día en que se solicita el cambio, como si fueran dos decisiones independientes. No lo son. La Tabla 5.18 cruza los dos parámetros sobre todas las expansiones de una sola etapa medidas, y de ella sale el resultado central de este caso de estudio: la misma ampliación devuelve cosas muy distintas según cuándo se pida, y el día pesa más que el tamaño.

TABLA 5.18. TODAS LAS EXPANSIONES DE UNA SOLA ETAPA, CRUZANDO EL DÍA EN QUE SE SOLICITAN CON EL DESTINO AL QUE LLEGAN.

Día	**Destino**	**n**	**Tiempo (s)**	**Núcleos × hora**
Ocho servidores de almacenamiento				
30	32	3	3592,5 [3544,3–3617,9]	29,33
30	40	3	3439,3 [3395,3–3463,1]	34,61
40	32	2	3598,7 [3566,6–3630,7]	28,56
40	40	6	3424,8 [3382,2–3482,2]	33,45
40	64	3	2927,6 [2875,4–2994,6]	43,47
50	32	3	3956,6 [3939,9–3985,6]	26,98
50	40	3	3802,3 [3774,2–3824,8]	31,41
78	40	2	4934,3 [4919,6–4949,1]	26,63
78	48	2	4788,0 [4787,6–4788,4]	28,31
78	64	2	4605,2 [4592,6–4617,7]	32,07
Dos servidores de almacenamiento				
78	40	1	5222,6	26,90
78	48	2	5268,8 [5263,7–5273,9]	31,24
78	64	1	5178,9	37,46
Todas las ejecuciones parten de ocho procesos y recorren la misma escena simulada. Las tres últimas repiten el día setenta y ocho sobre una topología de almacenamiento distinta y se incluyen para aislar su efecto; los núcleos × hora son siempre los de la aplicación.				

Conviene empezar por las tres ejecuciones del día setenta y ocho, que son las que

plantean el problema. Piden la expansión el mismo día simulado y solo se diferencian en el destino: cuarenta, cuarenta y ocho y sesenta y cuatro procesos. Ampliar les sirve de algo, pero poco. Pasar de cuarenta a sesenta y cuatro procesos recorta un 6,7 % del tiempo, con dos escalones intermedios ordenados y por encima del ruido, y cuesta un 20,4 % más de recursos. Ese mismo salto, solicitado el día cuarenta, recorta un 14,5 %. El destino es idéntico y el retorno se queda en menos de la mitad por haberlo pedido treinta y ocho días simulados más tarde.

La explicación inmediata sería que la máquina no admite más. Cada nodo del clúster dispone de cuarenta núcleos físicos (Sección 5.1.2), de manera que por encima de esa cifra los procesos comparten hilos lógicos del mismo núcleo, y el destino de cuarenta caería justo en la frontera. La tabla la descarta: si el techo estuviera en el hardware, los procesos añadidos por encima de cuarenta rendirían peor con independencia de cuándo se pidan, y el mismo salto no podría devolver el doble por adelantarlo. El límite, por tanto, no está en la máquina.

Está en cuánta simulación queda por delante cuando los procesos nuevos empiezan a trabajar. La escena no cuesta lo mismo todos los días, como muestra el panel superior de la Figura 5.13, con las cuatro fases sombreadas al fondo y marcados los días en que se solicita el cambio: hasta el día cuarenta y dos avanzar un día simulado cuesta alrededor de dos segundos por volcado, entre el cuarenta y dos y el setenta y ocho sube hasta los catorce o dieciséis, y a partir de ahí vuelve a caer a valores intermedios. Los procesos añadidos solo pueden actuar sobre lo que todavía no se ha hecho, y esa fracción se agota deprisa porque la parte cara está concentrada en la primera onda. Medido sobre cada ejecución, cuando el destino queda operativo se ha consumido ya el 11 % del tiempo total si se pidió el día treinta, el 15 % si se pidió el cuarenta, el 32 % si se pidió el cincuenta y entre el 64 y el 70 % si se pidió el setenta y ocho. Pedir la ampliación el día setenta y ocho es pedirla con dos tercios del trabajo hecho.

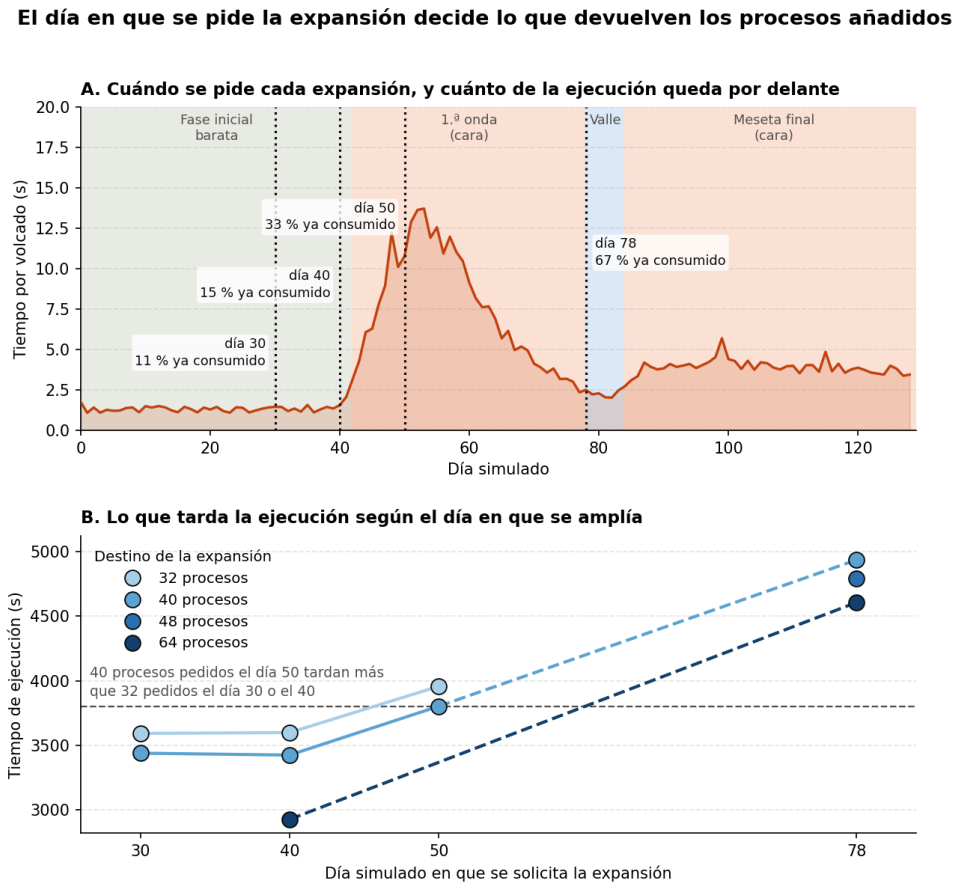


Fig. 5.13. Efecto del día en que se solicita la expansión.

Ese reparto explica el resultado anterior, y conviene hacer la cuenta en segundos de reloj. Los procesos que se añaden solo pueden acelerar lo que todavía queda por hacer. En la ampliación del día cuarenta empiezan a trabajar a los nueve minutos de una ejecución que dura cincuenta y siete, de modo que tienen por delante cuarenta y ocho minutos con la primera onda incluida: los veinticuatro procesos adicionales ahorran allí 497 segundos. En la del día setenta y ocho empiezan a los cincuenta y tres minutos de una ejecución de ochenta y dos, les quedan veintinueve, y ahorran 329.

Lo revelador es la proporción. Sobre el tramo que cada ampliación tiene por delante, los procesos añadidos recortan un 17,2 % en el primer caso y un 18,8 % en el segundo. Es decir, rinden igual de bien en los dos, e incluso algo mejor en el tardío. La ampliación del día setenta y ocho no rinde menos porque los procesos trabajen peor ni porque los días que quedan sean baratos: rinde menos porque quedan pocos. Esto acota además la duda del párrafo anterior. El reparto de hilos lógicos, tenga el efecto que tenga sobre cada proceso añadido, es el mismo se pida la expansión el día que se pida, de modo que no puede explicar por qué el mismo salto devuelve más del doble cuando se adelanta.

El barrido cruzado lo confirma sobre tres días distintos. El precio de ampliar apenas cambia con el momento: pasar de treinta y dos a cuarenta procesos cuesta un 18,0 %, un

17,1 % y un 16,4 % más de recursos según se pida el día treinta, el cuarenta o el cincuenta. Lo que se recibe a cambio es un 4,3 %, un 4,8 % y un 3,9 % de tiempo. Esas tres cifras no son distinguibles entre sí, ya que la separación entre ellas cabe dentro de la dispersión entre ejecuciones; lo que sí se distingue, y por un margen amplio, es el contraste con el 6,7 % que devuelve el día setenta y ocho un salto que es tres veces mayor.

De ahí sale la lectura que resume el apartado. El panel inferior de la Figura 5.13 muestra su mitad visible, la del tiempo, y la tabla añade la otra. En él, la línea horizontal marca el tiempo de la expansión a cuarenta procesos del día cincuenta, y los tramos entre ese día y el setenta y ocho se trazan discontinuos porque no hay medidas intermedias; todos los puntos comparten la topología de ocho servidores de almacenamiento. Las dos ejecuciones que amplían a treinta y dos procesos antes de la onda terminan antes y consumen menos que la que amplía a cuarenta el día cincuenta: 3592,5 y 3598,7 segundos frente a 3802,3, con 29,33 y 28,56 núcleos $\times$ hora frente a 31,41. La primera de las dos gana un 5,5 % de tiempo con un 6,6 % menos de recursos. Son los dos únicos pares de la campaña en los que una configuración es a la vez más rápida y más barata que otra, y en los dos el ganador es el que amplía menos y antes. Dentro del rango ensayado, llegar a tiempo con ocho procesos menos bate a llegar tarde con ocho más.

Que el techo no lo imponga el hardware no significa que el tamaño sea indiferente. Su efecto existe y es grande cuando la ampliación llega a tiempo, aunque con retorno decreciente: sobre la campaña de la Tabla 5.16, pasar de dieciséis a treinta y dos procesos recorta un 31,1 % y pasar de treinta y dos a sesenta solo un 9,4 %. Ahí sí encaja el reparto de hilos lógicos por encima de los cuarenta núcleos físicos, que actúa como un peaje sobre lo que rinde cada proceso añadido y no como un límite al número de procesos útiles. El orden de las dos decisiones, en la práctica, es que el día fija cuánto se puede ganar y el destino reparte esa ganancia con rendimientos decrecientes.

Las tres últimas filas de la tabla repiten el día setenta y ocho con dos servidores de almacenamiento en lugar de ocho, y permiten separar ese factor del anterior. Multiplicar por cuatro el número de servidores acorta la ejecución un 5,5 % con cuarenta procesos, un 9,1 % con cuarenta y ocho y un 11,1 % con sesenta y cuatro. El almacenamiento influye, por tanto, y su peso crece con el número de procesos, en la misma dirección que ya mostraba la Tabla 5.10: cuantos más procesos escriben, mayor es la parte del tiempo que depende del sistema de ficheros. Pero ninguna de esas diferencias altera el orden de las tres configuraciones ni la conclusión del apartado, porque la comparación entre destinos se hace siempre dentro de una misma topología.

Conviene acotar el alcance. Las tres filas de dos servidores provienen de una campaña anterior y dos de ellas son de una sola ejecución, de modo que la magnitud del efecto del almacenamiento se enuncia con reserva; el resto de la tabla está medido sobre la misma topología y ninguna de sus celdas baja de dos ejecuciones. El consumo que se compara es siempre el de la aplicación, porque en estas ejecuciones los servidores también se reducen a la mitad el día de la expansión y su coste no distingue entre destinos.

A ese límite de rentabilidad se suma otro, de mecanismo, que no depende de cuánta simulación quede por delante. Una expansión amplia no puede solicitarse de una sola vez: cuando se pide en una única operación un salto como el de ocho a sesenta y cuatro procesos, la creación se completa solo en parte y el número de procesos que quedan activos varía de una ejecución a otra. La forma de acercarse al destino con fiabilidad es pedirlo en lotes de tamaño acotado y releer, después de cada uno, cuántos procesos hay realmente activos antes de solicitar el siguiente, de manera que cada lote cubre lo que el anterior dejó sin cubrir. El procedimiento reduce el problema, aunque no siempre lo elimina, como muestra la ejecución de la Tabla 5.16 que se quedó en sesenta procesos. Su coste es que el destino no se alcanza el día en que se pide, sino dos o tres días simulados después, lo que en las ampliaciones tardías agrava todavía un poco más el problema anterior.

Por qué el reloj no es el observable adecuado

Todas las comparaciones anteriores dan por ganadora a la configuración estática, y conviene detenerse en por qué. Una estática dimensionada al pico dispone de todos sus procesos desde el primer instante, incluida la fase inicial en la que no hacen ninguna falta. Ninguna ejecución que empiece con menos puede recuperar esa ventaja, porque el trabajo que la otra adelantó ya está hecho. Medida solo con el reloj, la maleabilidad pierde siempre, y pierde por construcción: el resultado está decidido antes de ejecutar nada.

El reloj, además, responde a una pregunta que no es la única que importa. Dice cuánto tarda la simulación en terminar, pero no dice cuánto ocupa mientras tanto, y en un clúster compartido lo segundo es tan relevante como lo primero: los procesos que una ejecución no reserva quedan disponibles para otra. Esa es la magnitud que el caso de estudio introdujo junto al tiempo, y se obtiene multiplicando los procesos reservados por el tiempo que lo están.

Con las dos magnitudes sobre la mesa el orden se invierte. El calendario que sigue la demanda de la Figura 5.12 tarda 3757,4 segundos frente a los 3353,8 que midió sobre ese mismo despliegue la estática de sesenta y cuatro procesos —el segundo bloque de la Tabla 5.16—, un 12,0 % más, y consume 43,11 núcleos × hora frente a 61,49, un 29,9 % menos. Estos dos consumos incluyen los dos servidores de almacenamiento, como los de esa tabla; de ahí que el del calendario maleable no coincida con el que se ha dado en el apartado anterior, donde se contaban solo los procesos de la aplicación. Dicho de la forma en que se nota en una máquina compartida: en los recursos que ocupan tres ejecuciones estáticas caben cuatro que siguen la demanda, a cambio de que cada una termine unos siete minutos más tarde. Eso es lo que la maleabilidad ofrece, y explica por qué juzgarla por el reloj es juzgarla precisamente por donde no pretende aportar.

La Figura 5.14 pone las dos magnitudes en el mismo dibujo. Representa los procesos que cada ejecución tiene reservados a lo largo del tiempo de reloj, de manera que el área encerrada bajo cada curva es exactamente su consumo, y el punto en que la curva termina, su duración. La franja gris superior son los dos servidores de almacenamiento,

que ocupan un núcleo cada uno durante toda la ejecución y también cuentan. La estática es un rectángulo: sesenta y cuatro procesos ocupados de principio a fin, también durante los primeros setecientos segundos, en los que la simulación avanza igual de rápido con ocho. La otra es una escalera que solo sube cuando la carga lo pide y baja cuando deja de pedirlo. Se ve a la vez que la escalera se prolonga algo más hacia la derecha y que deja sin ocupar buena parte del rectángulo.

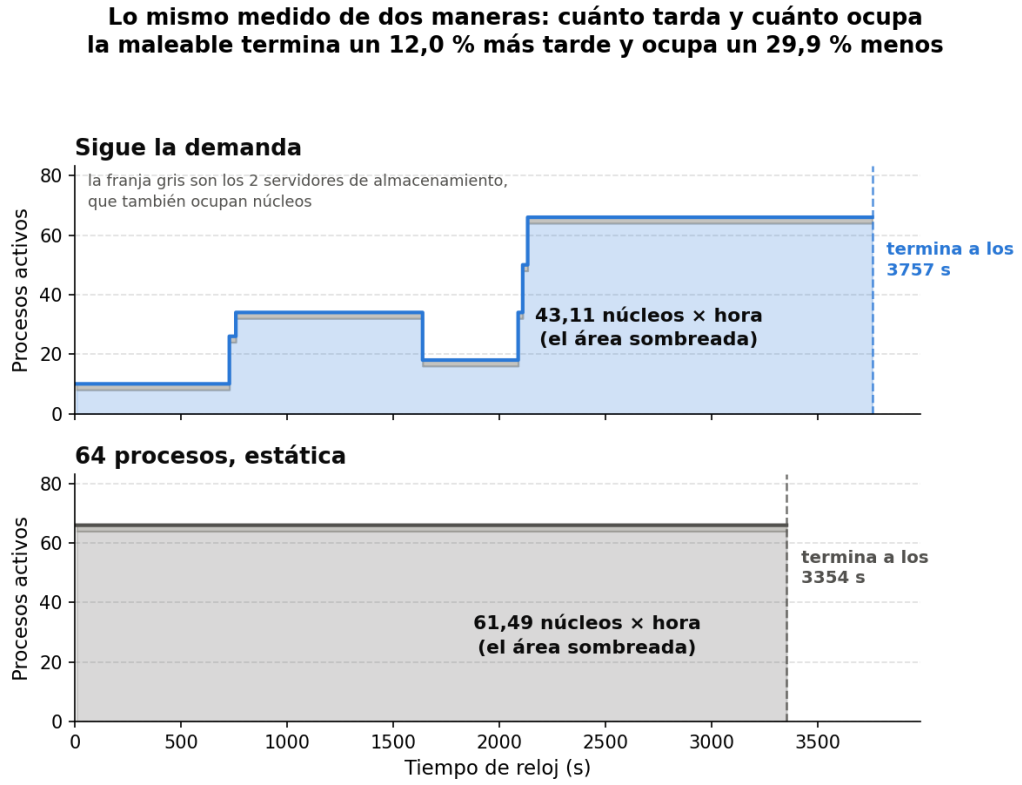


Fig. 5.14. Procesos reservados por cada configuración a lo largo del tiempo de reloj.

Queda precisar cómo se ha calculado ese consumo, porque el resultado depende de ello. Los procesos reservados no se toman del calendario solicitado sino del número que la aplicación declara tener activo en cada volcado, y se integran a lo largo del reloj. Son dos cosas distintas: el calendario recoge lo que se pide y es una escalera de saltos instantáneos en los días configurados, mientras que el recuento declarado recoge lo que efectivamente hubo. Las separa la creación por lotes descrita más arriba, que hace que una expansión amplia atraviese varios escalones intermedios antes de llegar a su destino. Integrar el calendario daría por reservados desde el primer momento unos procesos que todavía no existían y atribuiría a la configuración maleable un consumo que no llegó a producirse. El criterio no cambia el signo del resultado, pero sí su magnitud, y por eso se hace explícito.

Co-maleabilidad

Queda la campaña que reconfigura a la vez los dos recursos. La Tabla 5.19 recoge las tres configuraciones, idénticas en cómputo y distintas solo en el almacenamiento, cada una repetida seis veces.

TABLA 5.19. EFECTO DE RETIRAR LA MITAD DE LOS SERVIDORES DE ALMACENAMIENTO EL DÍA EN QUE LA APLICACIÓN EXPANDE.

Almacenamiento	**n**	**Tiempo (s)**	**E/S (s)**	**Aplic.**	**Total**
8 servidores fijos	6	3370,8 [3364,2–3379,1]	386,6	32,98	40,47
8 → 4 en el día 40	6	3467,3 [3423,4–3494,7]	421,5	34,00	38,38
4 servidores fijos	6	3507,2 [3489,2–3520,2]	458,5	34,27	38,17
Las dos últimas columnas son núcleos × hora de la aplicación y del total, con un núcleo por servidor.					

Frente a mantener los ocho servidores durante toda la ejecución, retirar la mitad cuesta un 2,86 % de tiempo y ahorra un 5,16 % de recursos. Ese es el compromiso que la campaña sostiene, y va en la dirección que la hipótesis anticipaba: el almacenamiento que sobra cuando la simulación pasa a estar dominada por el cómputo puede devolverse a un precio menor que el que se ahorra.

Frente a arrancar directamente con cuatro servidores el resultado es distinto y hay que enunciarlo con cuidado. La configuración que se contrae es un 1,14 % más rápida y un 0,6 % más cara, pero sus rangos observados se solapan y las dos diferencias son del mismo tamaño que el ruido entre despliegues medido en la Sección 5.5.1, de modo que ninguno de los dos signos puede afirmarse. Lo que sí queda establecido es que las dos configuraciones que terminan con cuatro servidores cuestan lo mismo, y que la que empieza con ocho no paga por ello: los cuatro servidores adicionales solo están reservados durante el primer catorce por ciento de la ejecución.

Los dos resultados juntos dicen más que cada uno por separado. Ninguna de las tres configuraciones domina a las otras dos: la que se contrae queda entre las dos fijas en las dos magnitudes, más lenta y más barata que la de ocho servidores e indistinguible de la de cuatro. Lo que la co-maleabilidad aporta, por tanto, no es una configuración mejor que las estáticas, sino no tener que acertar el dimensionado antes de arrancar: se empieza con el almacenamiento que exige la fase de carga inicial y se devuelve la mitad cuando deja de hacer falta, al mismo precio que si se hubiera arrancado ya con esa mitad.

Conviene, eso sí, no pedirle a esta política más de lo que puede dar. El almacenamiento pesa entre el 10 y el 19 % del consumo total según cuántos servidores haya desplegados, de manera que ninguna decisión sobre él puede mover el resultado tanto como las del número de procesos de cómputo, que en los apartados anteriores cambiaban el tiempo

en más de un 30 %. La aportación de la co-maleabilidad es que esa segunda decisión no obligue a sobredimensionar la primera.

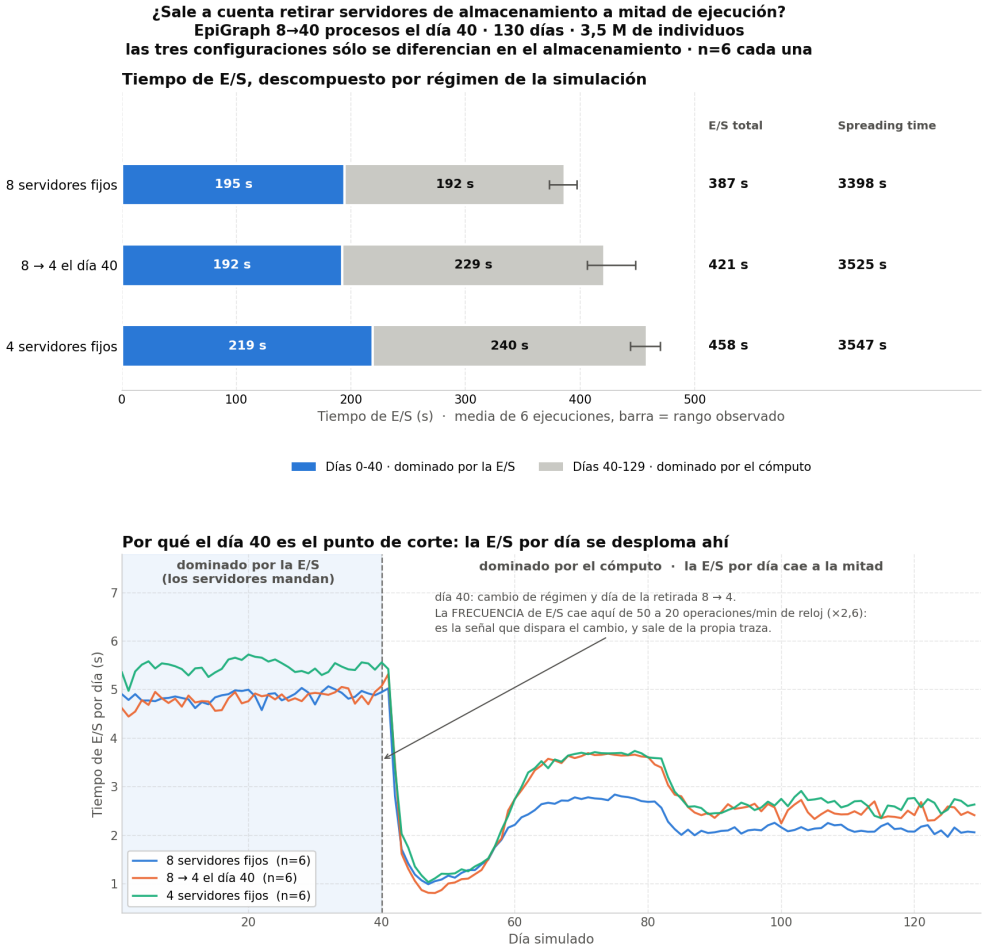


Fig. 5.15. Tiempo de entrada/salida al retirar servidores de almacenamiento a mitad de ejecución.

El panel superior de la Figura 5.15 separa el tiempo de entrada/salida de las tres configuraciones en los dos regímenes de la simulación, y la barra de error recoge el rango de las seis ejecuciones. El panel inferior recoge ese mismo tiempo por día simulado y añade el motivo por el que el día cuarenta es el punto de corte, y no otro. El coste diario de la entrada/salida se desploma ahí a la mitad, y con él la frecuencia con que la aplicación escribe, que pasa de unas cincuenta a unas veinte operaciones por minuto de reloj. Esa señal se obtiene de las trazas que la aplicación ya emite, sin instrumentar nada nuevo, lo que la hace candidata a gobernar la decisión de forma automática. Conviene advertir de una limitación: la señal no es monótona, sino que repunta en el valle entre las dos ondas epidémicas, de manera que una política que añadiese servidores cada vez que la frecuencia sube volvería a expandir en ese punto. Ese comportamiento no se ha medido.

Recapitulación

La reconfiguración del número de procesos funciona por los dos mecanismos, y el modo dinámico es entre un doce y un veintidós por ciento más rápido que el ciclo de *checkpoint* y reinicio, con esa diferencia subestimada. Su beneficio no aparece en el reloj: frente a una configuración estática dimensionada al pico, el mejor calendario tarda un 12,0 % más y consume un 29,9 % menos. Del resto de dimensiones, la magnitud rinde mientras quede simulación cara por delante y su retorno se reduce a menos de la mitad cuando la expansión llega tarde, el momento penaliza el retraso y no el adelanto, y la dirección importa en la medida en que permite sostener una expansión temprana sin pagarla durante las fases en que no hace falta. La reconfiguración simultánea del almacenamiento cuesta un 2,86 % de tiempo y ahorra un 5,16 % de recursos frente a mantener los ocho servidores toda la ejecución; frente a arrancar directamente con cuatro termina en el mismo tiempo y al mismo precio dentro de la variabilidad medida, de modo que lo que aporta no es rendimiento, sino poder empezar sobrado sin pagarlo.

5.5.5. CE4 — Rebalanceo dinámico de carga

La pregunta de esta campaña no admite una respuesta única. El rebalanceo de carga a número constante de procesos —el mecanismo descrito en la Sección 4.4.2, en el que el controlador agrega el vector de costes que EpiGraph le entrega al término de cada día simulado y desplaza columnas del grafo desde los procesos sobrecargados hacia los menos cargados, sin alterar cuántos son— acelera la simulación en unas condiciones y la frena en otras, y el signo lo deciden los tres factores que la Sección 5.4.4 enumera: el hardware sobre el que se ejecuta, la cadencia con que se evalúa el balance y el tamaño de la población. Esta subsección los recorre uno a uno, siempre sobre parejas de ejecuciones que difieren únicamente en la política de balanceo, y cierra con lo que se puede afirmar sin cualificar. Todas las ejecuciones de esta campaña son estáticas: el número de procesos se fija al arrancar y no varía en ningún momento, de modo que nada de lo que aquí se mide procede de la maleabilidad que evalúa la Sección 5.5.4. Todas las cifras están tomadas en el punto de medida común de la Sección 5.5.1 y junto a cada configuración figura el número de ejecuciones promediadas. Ninguna de las ejecuciones que aquí se emplean presenta la métrica de coma flotante a cero, de modo que en todas ellas el balanceador dispuso de la entrada que necesita para decidir.

El hardware

La Tabla 5.20 enfrenta las dos plataformas con la población reducida, un volcado por día y ocho procesos, que es la única combinación medida en ambas.

TABLA 5.20. EFECTO DEL BALANCEO SEGÚN LA PLATAFORMA. POBLACIÓN DE 800 826 INDIVIDUOS, OCHO PROCESOS, EVALUACIÓN CADA 1440 ITERACIONES. TIEMPOS EN SEGUNDOS.

Plataforma	**n**	**Con balanceo**	**Sin balanceo**	Δ
Equipo local	1	1122,5	1498,7	−25,1 %
Clúster Tucán	1	1394,2	1388,7	+0,4 %
Los valores negativos de Δ indican que el balanceo reduce el tiempo.				

En el equipo local el balanceo recorta la cuarta parte del tiempo; en el clúster la diferencia es de cinco segundos sobre mil cuatrocientos, por debajo del ruido de despliegue cuantificado en la Sección 5.5.1, de modo que no tiene signo. La explicación está en el reparto de recursos. El equipo local ejecuta ocho procesos sobre cuatro núcleos físicos con multihilo simultáneo, así que dos procesos comparten núcleo y avanzan a menos de la mitad de velocidad que los que lo ocupan solos; la partición en bloques contiguos de tamaño equivalente que EpiGraph construye al arrancar (Sección 4.4.1) entrega el mismo número de individuos a unos y a otros, y el desbalanceo resultante es grande. En el clúster cada proceso dispone de un núcleo dedicado, los ocho avanzan al mismo ritmo y la partición inicial está ya cerca del óptimo. El balanceador no crea equilibrio donde ya lo hay: solo puede recuperar el que falta, y si no falta nada, su trabajo es puro coste. Que ese coste no llegue a hacerse visible aquí es consecuencia de la cadencia baja de esta comparación, y es precisamente lo que examina el apartado siguiente.

La cadencia de evaluación

El parámetro que fija cada cuántas iteraciones se vuelca el estado gobierna también, en el mismo punto del código, cada cuántas se invoca al monitor que evalúa el balance. Multiplicar por diez la frecuencia de volcado para hacer visible el sistema de ficheros paralelo, según se describe en la Sección 5.2.4, multiplicó por diez, sin pretenderlo, tres cosas a la vez: el volumen de entrada/salida, el número de evaluaciones del monitor y el número de oportunidades de rebalancear. La Tabla 5.21 recoge las cuatro celdas que resultan de cruzar la cadencia con la política.

TABLA 5.21. EFECTO DE LA CADENCIA DE EVALUACIÓN SOBRE CADA UNA DE LAS DOS POLÍTICAS. CLÚSTER TUCÁN, POBLACIÓN DE 800 826 INDIVIDUOS, OCHO PROCESOS.

Configuración	n	Tiempo (s)	E/S (s)	Δ frente a 1440
Con balanceo, 1440 iteraciones	1	1394,2	6,9	—
Con balanceo, 144 iteraciones	2	1271,5	69,5	−8,8 %
Sin balanceo, 1440 iteraciones	1	1388,7	6,8	—
Sin balanceo, 144 iteraciones	2	1574,8	39,0	+13,4 %

Las dos políticas responden al mismo cambio en sentidos opuestos. Sin balanceo, evaluar diez veces más a menudo cuesta 186,1 segundos, de los cuales 32,2 son entrada/salida adicional y los 153,9 restantes son el precio del monitor, que se ejecuta igualmente cuando la política está desactivada. Con balanceo, el mismo cambio ahorra 122,7 segundos pese a absorber 62,6 segundos más de entrada/salida, de modo que el rebalanceo devuelve del orden de 185 segundos: paga el instrumento que lo habilita y aún deja beneficio. La separación de los dos términos supone que la entrada/salida no se solapa con el cómputo, lo que la convierte en una estimación y no en una medida directa.

De aquí se sigue una advertencia sobre cómo leer la comparación pareada. A la cadencia alta la diferencia entre políticas es de un 19,3 % a favor del balanceo, pero ese margen no es íntegramente mérito suyo: la rama de referencia paga un monitor que se ejecuta diez veces más a menudo sin que nadie aproveche sus medidas. La mejora neta que el balanceo aporta sobre la mejor alternativa disponible es el 8,8 % de la tabla, no el 19,3 %.

El tamaño de la población

Fijados el hardware y la cadencia, queda el factor que invierte el signo. La Tabla 5.22 reúne todas las parejas medidas sobre el clúster con evaluación cada 144 iteraciones.

TABLA 5.22. EFECTO DEL BALANCEO SEGÚN EL TAMAÑO DE LA POBLACIÓN. CLÚSTER TUCÁN, EVALUACIÓN CADA 144 ITERACIONES. TIEMPOS EN SEGUNDOS.

Población	Procesos	n	Con balanceo	Sin balanceo	Δ
800 826	8	2	1271,5	1574,8	−19,3 %
3 511 110	8	2	7011,2	6538,4	+7,2 %
3 511 110	16	1	4716,0	4348,8	+8,4 %
La pareja de dieciséis procesos se toma de un mismo despliegue; las repeticiones sobre otro despliegue se emplean en la Tabla 5.9.					

Con la población reducida los rangos de las dos ramas quedan separados por 261,5 segundos y no se solapan, frente a una dispersión interna del 2,8 y el 3,0 %, de modo que el signo es seguro. Con la población completa el signo se invierte, y lo hace tanto a ocho como a dieciséis procesos, con diferencias que superan con holgura el ruido de despliegue.

El sentido de esa inversión se ve al separar lo que el mecanismo gana de lo que cuesta. Lo que gana es el desequilibrio que llega a corregir, y ese desequilibrio lo produce el reparto de recursos, que aquí no cambia. Lo que cuesta es mover columnas de un proceso a otro, y eso sí crece con la población, porque cada columna que cambia de dueño arrastra el estado de más individuos. Al pasar de ochocientas mil personas a tres millones y medio el precio se multiplica mientras el beneficio se mantiene igual, de manera que el balanceo resulta tanto más útil cuanto menor es la población. Es una interpretación coherente con las medidas y no un resultado, porque el coste de migrar no se ha medido por separado.

La entrada/salida no interviene en ninguna de estas diferencias. Nunca supera el 5,5 % del tiempo total, y con la población reducida es mayor en la rama que gana —69,5 frente a 39,0 segundos— de manera que apunta en sentido contrario al resultado. Las diferencias son, por tanto, de cómputo y de coordinación. La descomposición interna de esos dos términos no se emplea aquí para sostener ninguna afirmación, por las limitaciones que el entorno de contenedores impone a los contadores hardware y que se discuten en la Sección 5.5; a título indicativo, el tiempo de comunicación que FlexMPI contabiliza es sistemáticamente mayor en las ejecuciones sin balanceo, lo que resulta compatible con procesos desiguales que se esperan unos a otros en las operaciones colectivas.

Recapitulación

La Figura 5.16 reúne las comparaciones de los tres apartados anteriores. Todas las barras miden lo mismo —cuánto varía el tiempo al activar el balanceo, sobre parejas de ejecuciones que solo difieren en esa política— de manera que son directamente comparables entre sí: los valores negativos indican que el balanceo acelera la simulación y los positivos que la encarece, y junto a cada configuración figura el número de parejas medidas. La banda gris es el ruido de despliegue de la Sección 5.5.1, y lo que cae dentro se representa en gris porque no tiene signo. La celda del clúster con volcado diario aparece en los dos primeros grupos porque es el punto que ambos comparten, y es la misma pareja de ejecuciones en los dos casos.

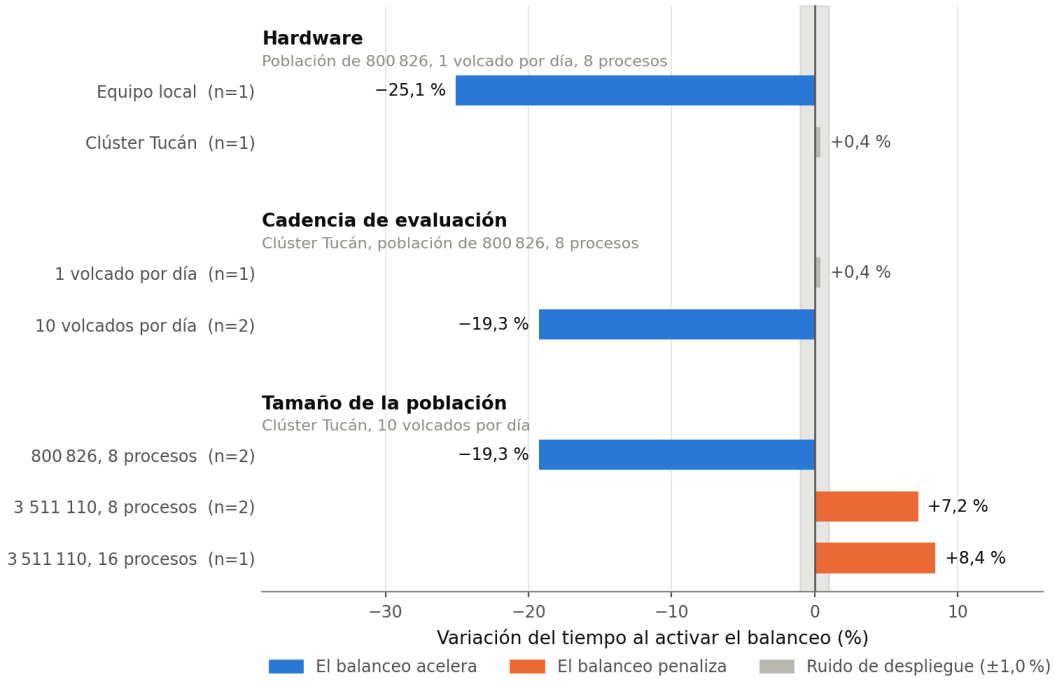


Fig. 5.16. Variación del tiempo de ejecución al activar el balanceo de carga.

El resultado de esta campaña es que el enunciado «el balanceo de carga a número fijo de procesos compensa» carece de valor de verdad mientras no se acompañe de las condiciones, y que las dos que lo deciden son el tamaño de la población y la cadencia con que se evalúa el balance. La población fija el signo: con ochocientas mil personas el balanceo ahorra un 19,3 % y con tres millones y medio cuesta entre un 7 y un 8 %, sin que cambie ni la máquina ni ninguna otra opción. La cadencia decide si el mecanismo llega a actuar: con una evaluación por día simulado la diferencia entre activarlo y no hacerlo es nula, y solo al evaluar diez veces por día aparecen los efectos anteriores. Las dos gobiernan además el coste: evaluar diez veces más a menudo acelera un 8,8 % cuando el balanceo está activo y penaliza un 13,4 % cuando no lo está, de modo que el precio del instrumento y el beneficio del mecanismo son del mismo orden. El hardware queda por debajo de las dos como condición de partida, porque es el que determina si existe algún desequilibrio que corregir.

Esto no resta valor a FlexMPI, lo delimita. Las dos capacidades que la biblioteca ofrece son independientes y no se comportan igual, tal y como anticipa la Sección 4.4.2: la maleabilidad del número de procesos, que CE3 evalúa, mejora la eficiencia en todas las condiciones ensayadas, mientras que el rebalanceo a número fijo solo compensa cuando existe un desbalanceo que corregir y mover el estado es barato. En el despliegue de esta memoria, con un proceso por núcleo dedicado y la población completa, esa condición no se cumple.

Conviene señalar, por último, el alcance de las medidas. El eje del hardware y el de la cadencia descansan sobre celdas de una sola ejecución en las configuraciones de volcado diario, y la réplica a dieciséis procesos también. Las diferencias que sostienen son grandes

—entre cinco y veinticinco veces el ruido de despliegue— y en dos de los tres ejes el signo se repite en parejas independientes, pero repetir esas celdas estrecharía los márgenes, en particular el de la magnitud con la población completa.

5.6. Limitaciones del estudio

Los resultados de la sección anterior se han obtenido en unas condiciones concretas, y conviene delimitar hasta dónde alcanzan. Las limitaciones que siguen se agrupan en cuatro bloques: las que impone el entorno sobre el que se ha medido, las que proceden de la integración con el sistema de ficheros, las que impuso la disponibilidad de la plataforma y las que afectan a la interpretación de las propias medidas. Ninguna invalida las conclusiones alcanzadas, pero varias acotan su alcance y una de ellas impide cruzar entre sí dos de los casos de estudio.

5.6.1. Entorno de ejecución

La limitación de mayor alcance es el acceso restringido a los contadores *hardware* del procesador. Un contenedor no dispone de la unidad de monitorización del anfitrión en las mismas condiciones que un proceso nativo, y cuando esa lectura no está disponible el número de operaciones en coma flotante llega a cero y arrastra consigo a los tiempos que se derivan de él, descritos en la Sección 5.1.5. Las consecuencias son dos. La primera afecta al mecanismo: el balanceador de carga decide a partir de esa métrica, de modo que una ejecución sin ella termina correctamente pero no informa sobre el rebalanceo, y por eso el caso de estudio de la Sección 5.5.5 solo admite ejecuciones en las que el contador estuvo operativo. La segunda afecta a la comparación: todas las ejecuciones multinodo presentan la métrica a cero y ninguna de las de un solo nodo lo hace, de manera que las cifras de una topología no pueden enfrentarse a las de la otra, tal y como se advierte en el apartado correspondiente de la Sección 5.5.3. Es también el motivo por el que el coste del rebalanceo se observa por su efecto sobre el tiempo total y no por una medida propia del mecanismo.

La red de contenedores impone la segunda. La red virtual por defecto no sostiene las operaciones colectivas de MPI por encima de unas decenas de procesos, lo que dejó fuera de alcance las configuraciones grandes sobre una sola máquina y obligó a recurrir a la red superpuesta de *swarm* descrita en la Sección 5.2.4. Esa red tiene un precio medido: entre dos nodos del clúster, un enlace directo alcanza 1175 MB/s y el mismo enlace a través de la red superpuesta se queda en 875 MB/s, un 25 % menos. El efecto sobre los resultados no es tanto la pérdida de ancho de banda como el hecho de que las dos cosas vayan juntas: pasar de una máquina a dos cambia a la vez el número de núcleos disponibles y la red por la que viaja la entrada/salida, de modo que las diferencias entre las topologías de uno y de dos nodos no pueden atribuirse limpiamente a ninguno de los dos factores por separado. Por esa razón las comparaciones se establecen siempre dentro de una misma topología.

Ambas limitaciones tienen un origen común, que es la tercera: el trabajo se ha realizado sin privilegios de administración sobre los nodos del clúster. Ajustar los parámetros de red del anfitrión o conceder capacidades adicionales a los contenedores habría permitido abordar las dos anteriores, y ninguna de las dos cosas estaba al alcance del usuario que ejecuta los experimentos.

Por último, el número de procesos que puede emplearse con provecho no lo fija el número de núcleos de las máquinas, sino el que queda libre después de situar los servidores de almacenamiento. Con la topología de ocho servidores, sobre dos nodos de cuarenta núcleos físicos, ese margen sitúa el techo práctico en torno a los sesenta y cuatro procesos de aplicación, que es el mayor destino ensayado en la Sección 5.5.4. Dimensionar la aplicación y dimensionar el almacenamiento no son, por tanto, dos problemas independientes.

5.6.2. Integración con el sistema de ficheros

Expand Ad-Hoc admite un modo de operación en el que un cliente que comparte máquina con un servidor escribe directamente sobre él, sin recurrir al protocolo de red. Ese modo no es utilizable aquí: cliente y servidor residen en contenedores distintos aunque compartan nodo físico, y cada contenedor mantiene su propio sistema de ficheros local, de manera que el atajo apunta a una ruta que no existe y la ejecución aborta en el primer volcado. Todas las medidas de esta memoria se han tomado, en consecuencia, con esa vía desactivada. La consecuencia interpretativa es relevante para el caso de estudio de la Sección 5.5.3: la ventaja que muestra la configuración colocalizada no procede de que el dato se escriba en el propio contenedor, sino de repartir el tráfico entre más servidores y de no atravesar una red física, y la localidad propiamente dicha queda sin evaluar.

La interceptación cubre únicamente la entrada/salida binaria, que es la del volcado de estado y la que domina el tiempo. Los ficheros de texto que la simulación escribe siguen dirigiéndose al sistema de ficheros compartido, de modo que la comparación entre almacenar sobre XPN y almacenar sobre el sistema por red no es completa, aunque la fracción que queda fuera es pequeña frente al volumen de los volcados.

Conviene además precisar qué es y qué no es el ancho de banda que la aplicación registra en cada volcado. La escritura se da por concluida cuando la llamada retorna, no cuando los datos han llegado a su destino, y por eso la cifra que aparece en la traza llega a superar lo que la red física puede transportar. Sirve para comparar configuraciones entre sí, porque el sesgo es el mismo en todas, y no debe leerse como una medida de rendimiento de la red.

5.6.3. Disponibilidad de la plataforma

El número de veces que puede medirse una configuración no es una decisión libre del experimentador, sino el resultado de lo que la plataforma permite. Conviene por ello dejar constancia de las condiciones en que se obtuvieron las medidas, porque explican la distribución desigual de repeticiones que se detalla en el apartado siguiente.

El coste unitario de una medida es alto. Una ejecución con la población completa ocupa un nodo entero entre una y tres horas según la configuración, de modo que una campaña de repeticiones consume varios días de nodo. El clúster, además, es una infraestructura compartida a la que se accede mediante reserva, y sus nodos permanecen en un estado de ahorro de energía mientras no hay ninguna activa. Cada lote de experimentos hubo de dimensionarse, por tanto, para caber dentro de una ventana de reserva acotada, lo que en la práctica fijó cuántas celdas podían medirse de una vez.

De ahí se sigue la forma que adoptaron los experimentos. Los lotes se diseñaron para ejecutarse de forma desatendida y reanudable, saltando las configuraciones cuyo registro ya estuviera completo, porque su duración excede la de una sesión de trabajo y una interrupción de la conexión no debe costar la campaña entera. La finalización de la reserva exigió además un cuidado particular: si expira mientras los contenedores siguen en marcha, el nodo queda en un estado de drenaje que lo retira del planificador hasta que un administrador lo restablece, con el consiguiente perjuicio para el resto de usuarios. Fue necesario, en consecuencia, un proceso auxiliar que vigilase el tiempo restante y detuviera ordenadamente el despliegue antes de que la reserva concluyese.

Por último, la disponibilidad de la plataforma no está garantizada. Una avería en el sistema de ficheros de usuario del clúster lo dejó inaccesible durante los últimos días del trabajo e impidió completar las repeticiones que quedaban previstas, en particular las de las configuraciones que el apartado siguiente identifica como medidas una sola vez. Las conclusiones que estas sostienen se enuncian, por ese motivo, con la reserva que les corresponde.

5.6.4. Alcance e interpretación de las medidas

El número de repeticiones no es uniforme, por las razones que expone el apartado anterior. Las afirmaciones centrales descansan sobre configuraciones medidas entre dos y seis veces, y el criterio con el que se decide si una diferencia es un resultado o es ruido se establece en la Sección 5.5.1, pero quedan celdas medidas una sola vez: el barrido de tamaño de bloque, las nueve configuraciones multinodo del caso de estudio de la Sección 5.5.3 y alguna fila suelta de los apartados de maleabilidad y de balanceo, señalada en cada tabla mediante la columna que recoge el número de ejecuciones promediadas. Las diferencias que esas celdas sostienen son en general varias veces mayores que el ruido medido, pero repetirlas estrecharía los márgenes.

La segunda magnitud que se emplea, el consumo de recursos expresado en núcleos × hora, se obtiene integrando a lo largo del reloj el número de procesos que la aplicación declara tener activos, y no el que fija el calendario solicitado. La distinción importa porque una expansión no alcanza siempre el destino que pide. Además, esa integral recoge solo los procesos de la aplicación, salvo en la comparación de co-maleabilidad de la Sección 5.5.4, donde el número de servidores de almacenamiento es precisamente la variable que se manipula y se contabilizan también ellos. Las cifras de consumo de uno y otro apartado no son, por tanto, directamente comparables entre sí.

De la mecánica de la expansión se derivan dos salvedades más, ya expuestas en el apartado correspondiente. La primera es que el destino se alcanza dos o tres días simulados después del día en que se solicita, porque los procesos se crean en lotes sucesivos; la desviación es pequeña frente a los ciento treinta días de la escena, pero juega siempre en contra de la configuración que amplía. La segunda es que la magnitud alcanzada no siempre coincide con la solicitada, lo que obliga a describir esas ejecuciones por el número de procesos que llegaron a entrar y no por el que se pidió.

Las medidas tomadas sobre despliegues distintos arrastran un desplazamiento sistemático en torno al 1 %, cuantificado en la Sección 5.5.1. Cuando ha sido posible se han emparejado las ejecuciones dentro de un mismo despliegue, y donde no lo ha sido se indica expresamente. Por el mismo motivo no se cruzan cifras absolutas entre campañas levantadas con distinto número de servidores de almacenamiento, aunque sí la dirección del efecto dentro de cada una.

Queda finalmente el límite más general, que es el de la carga evaluada. Todo lo anterior se ha medido sobre una única aplicación, con una única escena epidémica de ciento treinta días y dos tamaños de población. EpiGraph combina un coste de cómputo que sigue la curva de infección con una entrada/salida proporcional al total de individuos, y esa asimetría es justamente lo que hace pertinente la maleabilidad. Los resultados describen el comportamiento de esa combinación y no autorizan a extrapolar magnitudes a aplicaciones con otro perfil de carga, si bien el procedimiento de evaluación y los mecanismos empleados sí son trasladables a ellas.

6. CONCLUSIONES Y TRABAJO FUTURO

6.1. Conclusiones

Los cuatro casos de estudio se han presentado por separado, y conviene reunir ahora lo que cada uno permite afirmar. Lo que sigue enuncia esos resultados y señala a qué objetivo de los formulados en la Sección 1.2 responde cada uno; las cifras que los sostienen están en la Sección 5.5.

La entrada/salida de un simulador puede redirigirse hacia un sistema de ficheros paralelo sin modificar su código. EpiGraph se ejecuta sin recompilar y sin tocar sus rutinas de escritura, y su estado aparece repartido entre los directorios de los servidores de Expand Ad-Hoc. Esto responde a la parte del objetivo general que pedía integrar ambas piezas, y su dificultad no estuvo en el diseño sino en verificar que la redirección ocurría de verdad.

Escribir más rápido acorta también la ejecución completa. No era una consecuencia automática: un sistema de ficheros más veloz no mejora el tiempo total si la escritura apenas pesa en él, y comprobar que las dos magnitudes se mueven juntas era precisamente lo que el primer caso de estudio se propuso.

La ventaja de hacerlo no es una cantidad fija, sino que crece con el tamaño de la ejecución. La razón está en cómo responde cada sistema de ficheros a que se añadan procesos: el almacenamiento por red mantiene el mismo coste de escritura porque el servidor sigue siendo uno, mientras que el sistema *ad hoc* lo reduce, porque cada proceso añadido incorpora un servidor sobre el que repartir el volcado. Es el mecanismo que explica también los resultados de los dos casos de estudio siguientes.

Al desplegar el almacenamiento importa más dónde se sitúan los servidores que cuántos son. El número de servidores ordena la entrada/salida de forma monótona, pero traslada al tiempo total una fracción pequeña de esa mejora, mientras que reunir el almacenamiento en una máquina separada del cómputo es, con diferencia, la decisión más costosa de las evaluadas. Lo que decide es si la entrada/salida tiene que atravesar la red, y no cuántos servidores la atienden una vez que la atraviesa. Es un criterio de dimensionamiento, y como tal responde al objetivo de diseñar estrategias de adaptación.

La maleabilidad no debe juzgarse por el tiempo total de ejecución. Frente a una configuración estática dimensionada al pico, una ejecución que arranca con menos procesos tarda siempre algo más, y ese resultado está decidido antes de ejecutar nada, porque el trabajo que la estática adelanta mientras la otra crece ya no puede recuperarse. Lo que la maleabilidad aporta se ve en la otra magnitud, la de los recursos que la ejecución ocupa mientras dura, que en un sistema compartido pesa tanto como el tiempo. Reconocer esto es lo que hizo falta para poder afirmar que el mecanismo cumple su objetivo.

Dentro de la maleabilidad, llegar a tiempo vale más que llegar con muchos. Ampliar

poco y pronto resultó mejor que ampliar mucho y tarde en las dos magnitudes a la vez, algo que ninguna otra pareja de la campaña consiguió. La explicación es que los procesos que entran solo pueden actuar sobre el trabajo que queda por delante, y la parte cara de la simulación se concentra en la primera onda. Es el criterio más útil de los obtenidos para decidir un calendario de reconfiguración, y responde de lleno al objetivo de diseñar estrategias de adaptación dinámica.

El almacenamiento admite también reconfiguración en marcha, y lo que permite no es ir más deprisa, sino empezar sobrado sin pagarlo durante toda la ejecución. Retirar servidores cuando la escritura deja de dominar cuesta algo de tiempo y ahorra recursos frente a mantenerlos, y termina igualado con haber arrancado ya con pocos. Su valor está, por tanto, en no tener que acertar el dimensionamiento de antemano. Con ello el objetivo general queda cubierto también por el lado del almacenamiento: no solo se ajusta durante la ejecución el número de procesos que calculan, sino también el de servidores que atienden la escritura, si bien conforme a un calendario establecido de antemano.

El rebalanceo de la carga a número fijo de procesos es el único mecanismo evaluado que no compensa siempre. Su signo depende de las condiciones, y las dos que lo deciden son el tamaño de la población y la frecuencia con que se evalúa el balance: con una población reducida ahorra tiempo y con la completa lo encarece, sin que cambie ninguna otra cosa. El enunciado «el balanceo de carga compensa» carece por tanto de valor de verdad mientras no se acompañe de las condiciones en que se afirma.

De ahí se sigue que las dos capacidades que ofrece FlexMPI no son intercambiables. La maleabilidad del número de procesos mejoró la eficiencia en todas las condiciones ensayadas, mientras que el rebalanceo solo compensa cuando existe un desequilibrio que corregir y mover el estado entre procesos resulta barato. Esto no resta valor a la biblioteca, sino que delimita cuál de las dos conviene emplear en cada caso, que es lo que el objetivo de evaluar los resultados perseguía.

Queda una conclusión que no se refiere a los mecanismos sino a cómo se ha medido: sin cuantificar antes la variabilidad del entorno no hay resultado que afirmar. Varias conclusiones de este trabajo se dieron por firmes con una sola ejecución por configuración y no sobrevivieron al repetirlas, de modo que hubo que retirarlas. La cuantificación del ruido de la Sección 5.5.1 y la comparación en un punto común de la simulación no son preliminares del análisis, sino la parte que decide qué puede afirmarse de todo lo demás.

6.2. Grado de consecución de los objetivos

A continuación se recuperan los objetivos formulados en la Sección 1.2 y se razona en qué medida se han alcanzado.

El objetivo general era integrar el simulador EpiGraph con el sistema de ficheros Expand Ad-Hoc y dotar al conjunto de maleabilidad, de manera que tanto el cómputo como el almacenamiento se adaptasen a una carga que varía con la fase de la epidemia. La

integración se describe en la Sección 5.2 y quedó operativa sin modificar el código de entrada/salida de la aplicación, con el fichero de estado materializado y repartido entre los servidores de XPN. La adaptación dinámica se ejercita en los dos sentidos y sobre los dos recursos: el número de procesos de la aplicación se amplía y se reduce durante la ejecución, y el número de servidores de almacenamiento también, de forma simultánea, en la campaña de co-maleabilidad de la Sección 5.5.4. El objetivo se considera cumplido. Los objetivos específicos se detallan a continuación.

1. **Adquirir las competencias necesarias.** El trabajo ha exigido comprender el funcionamiento interno de las tres piezas y no solo su interfaz. Las modificaciones descritas en la Sección 5.2 —la corrección del mecanismo de interceptación, la de la serialización de argumentos que impedía la creación dinámica de procesos sobre la red superpuesta y los ajustes en la cadencia de la monitorización— son consecuencia directa de esa comprensión, y ninguna de ellas habría sido posible tratando los componentes como cajas negras. El objetivo se ha cumplido.

2. **Diseñar estrategias de adaptación dinámica.** El diseño se recoge en la Sección 4.4 y se concreta en los calendarios de reconfiguración que evalúa la Sección 5.5.4: expansiones de distinta magnitud, solicitadas en distintos días de la simulación, reducciones posteriores al pico y la reconfiguración simultánea del número de servidores de almacenamiento. La evaluación de esas estrategias ha permitido además ordenarlas, y el criterio resultante —ampliar a tiempo antes que ampliar mucho— es uno de los resultados del trabajo. El objetivo se ha cumplido, con la salvedad de que las estrategias se fijan por calendario y no se deciden automáticamente en ejecución, lo que se recoge en la Sección 6.3.

3. **Representación de los resultados.** Los resultados se presentan en la Sección 5.5 mediante tablas que indican en todos los casos el número de ejecuciones promediadas y el intervalo observado, y mediante figuras que descomponen el tiempo en sus partidas, siguen la evolución de la simulación día a día o representan el consumo de recursos como el área bajo la curva de procesos activos. El objetivo se ha cumplido.

4. **Evaluar los resultados.** La evaluación se organiza en los cuatro casos de estudio de la Sección 5.4 y se apoya en un criterio explícito para distinguir un resultado del ruido de medida, establecido en la Sección 5.5.1. Las conclusiones alcanzadas y los límites dentro de los que son válidas se recogen en las Secciones 6.1 y 5.6. El objetivo se ha cumplido.

6.3. Trabajo futuro

La primera línea de continuación consiste en repetir las campañas en un despliegue que no imponga las dos restricciones descritas en la Sección 5.6. Disponer de los contadores *hardware* del procesador permitiría observar el coste del rebalanceo por una medida

propia del mecanismo, en lugar de por su efecto sobre el tiempo total, y haría comparables entre sí las topologías de uno y de dos nodos, que hoy no lo son. Habilitar la escritura local de Expand Ad-Hoc, que el aislamiento entre contenedores impide, permitiría a su vez evaluar la localidad de los datos, que es la única de las tres decisiones de despliegue del segundo caso de estudio que ha quedado sin medir. Ambas cosas requieren ejecutar los componentes directamente sobre los nodos, sin la capa de contenedores, o bien disponer de privilegios de administración sobre ellos.

La segunda línea es automatizar la decisión de reconfigurar. En este trabajo el día en que se amplía el número de procesos, y el destino al que se amplía, se fijan de antemano en un calendario, lo que obliga a conocer la escena epidémica antes de simularla. El resultado de que el momento pesa más que la magnitud sugiere que esa decisión puede tomarse en ejecución a partir de una magnitud que la propia simulación ya produce: el número de individuos infectados, que es lo que determina el coste de cómputo de cada iteración. Una política que solicitase procesos al detectar el crecimiento de la curva, y los devolviese al remitir, haría innecesario el calendario y extendería el mecanismo a escenas desconocidas de antemano.

La tercera línea es ampliar el alcance de la evaluación a otras cargas. Todo lo medido aquí procede de una única aplicación y una única escena, y las magnitudes obtenidas describen el comportamiento de una combinación concreta: un coste de cómputo que sigue la curva de infección y una entrada/salida proporcional al total de individuos. El procedimiento de evaluación, en cambio, no depende de esa combinación, y aplicarlo a cargas con otro perfil —por ejemplo, con una entrada/salida que crezca con la fase de cómputo en lugar de mantenerse constante— permitiría comprobar cuáles de las conclusiones alcanzadas son propias de esta aplicación y cuáles pertenecen a los mecanismos empleados.

BIBLIOGRAFÍA

[1] MPI Forum, *MPI: A Message-Passing Interface Standard Version 4.1*, Accedido: 2024-06-14, 2023. [En línea]. Disponible en: https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf.

[2] H. Ather et al., "Parallel I/O Characterization and Optimization on Large-Scale HPC Systems: A 360-Degree Survey," *arXiv preprint arXiv:2501.00203*, 2024, Accedido: 2025-05-20. [En línea]. Disponible en: https://arxiv.org/abs/2501.00203.

[3] D. Camarmas-Alonso, F. Garcia-Carballeira, A. Calderón-Mateos y J. Carretero, "Sistema de almacenamiento Ad-Hoc para entornos HPC basado en el sistema de ficheros paralelo Expand," en *XXXII Jornadas de Paralelismo (JP22)*, Alicante, España: Zenodo, 2022. DOI: 10.5281/zenodo.6862882.

[4] D. Muñoz-Muñoz, F. Garcia-Carballeira, D. Camarmas-Alonso, A. Calderon-Mateos y J. Carretero, "Malleability and fault tolerance in ad-hoc parallel file systems," *Cluster Computing*, vol. 28, n.º 860, pp. 1-19, 2025. DOI: 10.1007/s10586-025-05575-8.

[5] P. Sánchez-Checa, G. Sánchez-Gallegos, J. Garcia-Blas, J. Carretero y D. E. Singh, "Explorando la maleabilidad combinada de aplicaciones y sistemas de ficheros," en *Jornadas SARTECO*, Sociedad de Arquitectura y Tecnología de Computadores, 2025.

[6] A. Brinkmann et al., "Ad Hoc File Systems for High-Performance Computing," *Journal of Computer Science and Technology*, vol. 35, n.º 1, pp. 4-26, 2020. DOI: 10.1007/s11390-020-9801-1.

[7] J. Garcia-Blas, G. Sanchez-Gallegos, C. Petre, A. R. Martinelli, M. Aldinucci y J. Carretero, "Hercules: Scalable and Network Portable In-Memory Ad-Hoc File System for Data-Centric and High-Performance Applications," en *Euro-Par 2023: Parallel Processing*, J. Cano et al., eds., ép. Lecture Notes in Computer Science, vol. 14100, Springer, 2023, pp. 679-693. DOI: 10.1007/978-3-031-39698-4_46.

[8] J. Fernandez Muñoz, A. Cascajo García y J. Carretero Pérez, "Dynamic Management of Processes and Communicators in Malleable MPI Applications," en *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, 2023, pp. 848-855. DOI: 10.1109/ICPADS60453.2023.00127.

[9] G. Martín, D. E. Singh, M.-C. Marinescu y J. Carretero, "Enhancing the Performance of Malleable MPI Applications by Using Performance-aware Dynamic Reconfiguration," *Parallel Computing*, vol. 46, pp. 60-77, 2015. DOI: 10.1016/j.parco.2015.04.003.

[10] K. El Maghraoui, T. J. Desell, B. K. Szymanski y C. A. Varela, "Dynamic Malleability in Iterative MPI Applications," en *Seventh IEEE International Symposium on Cluster Computing and the Grid (CCGrid '07)*, 2007, pp. 591-598. DOI: 10.1109/CCGRID.2007.45.

[11] P. Lemarinier, K. Hasanov, S. Venugopal y K. Katrinis, "Architecting Malleable MPI Applications for Priority-driven Adaptive Scheduling," en *Proceedings of the 23rd European MPI Users' Group Meeting (EuroMPI 2016)*, 2016, pp. 74-81. DOI: 10.1145/2966884.2966907.

[12] D. Huber, M. Streubel, I. Comprés, M. Schulz, M. Schreiber y H. Pritchard, "Towards Dynamic Resource Management with MPI Sessions and PMIx," en *EuroMPI/USA'22: 29th European MPI Users' Group Meeting*, 2022, pp. 57-67. DOI: 10.1145/3555819.3555856.

[13] B. B. Rad, H. J. Bhatti y M. Ahmadi, "An Introduction to Docker and Analysis of its Performance," *IJCSNS International Journal of Computer Science and Network Security*, vol. 17, n.º 3, pp. 228-235, mar. de 2017.

[14] D. Walsh, *Podman in Action: Secure, Rootless Containers for Kubernetes, Microservices, and More*. Manning Publications, 2023.

[15] G. M. Kurtzer, V. Sochat y M. W. Bauer, "Singularity: Scientific Containers for Mobility of Compute," *PLoS ONE*, vol. 12, n.º 5, e0177459, 2017. DOI: 10.1371/journal.pone.0177459.

[16] I. Martín-Álvarez, J. I. Aliaga, M. Castillo y S. Iserte, "Efficient Data Redistribution for Malleable Applications," en *SC-W 2023: Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 416-426. DOI: 10.1145/3624062.3624110.

[17] G. Sánchez-Gallegos, J. García-Blas, C. Petre y J. Carretero, "Malleable and Adaptive Ad-Hoc File System for Data Intensive Workloads in HPC Applications," en *Euro-Par 2023: Parallel Processing Workshops*, ép. LNCS, Springer, 2023, pp. 56-70. DOI: 10.1007/978-3-031-40843-4_5.

[18] ADMIRE Consortium, *Deliverable D2.3: Short-lived Ad-Hoc Storage Systems*, 2024. [En línea]. Disponible en: https://admire-eurohpc.eu/wp-content/uploads/2024/02/D2_3_admire.pdf.

[19] D. E. Singh y J. Carretero, "Combining Malleability and I/O Control Mechanisms to Enhance the Execution of Multiple Applications," *Journal of Systems and Software*, vol. 148, pp. 21-36, 2019. DOI: 10.1016/j.jss.2018.10.057.

[20] ADMIRE Consortium, *ADMIRE: Adaptive Multi-tier Intelligent Data Manager for Exascale*, European Union's Horizon 2020 JTI-EuroHPC, Grant No. 956748, 2024. [En línea]. Disponible en: https://admire-eurohpc.eu/.

[21] D. E. Singh, C. Olmedo Luceron, A. Limia Sanchez, M. Guzman Merino, C. Duran Gonzalez et al., "Evaluation of vaccination strategies for the metropolitan area of Madrid via agent-based simulation," *BMJ Open*, vol. 12, n.º 12, e065937, 2022. DOI: 10.1136/bmjopen-2022-065937.

[22] G. Martín, M.-C. Marinescu, D. E. Singh y J. Carretero, "FLEX-MPI: An MPI Extension for Supporting Dynamic Load Balancing on Heterogeneous Non-dedicated Systems," en *Euro-Par 2013 Parallel Processing*, F. Wolf, B. Mohr y D. an Mey, eds., ép. Lecture Notes in Computer Science, vol. 8097, Springer, 2013, pp. 138-149.

[23] K. Sherratt, H. Gruson, R. Grah et al., "Predictive performance of multi-model ensemble forecasts of COVID-19 across European nations," *Epidemiology and Global Health*, vol. 12, n.º 6, e81916, 2023. DOI: 10.7554/eLife.81916.

[24] Universidad de Alcalá, *Curriculum Vitae del Personal Docente e Investigador (PDI) - Departamento de Automática*, 2024. [En línea]. Disponible en: https://www.uah.es/export/sites/uah/es/empleo-publico/PDI/.galleries/2024/CVs-Dpto.-Automatica.pdf.

[25] G. Sánchez-Gallegos, J. García-Blas, C. Petre y J. Carretero, "Sistema distribuido de archivos ad hoc para cargas de trabajo intensivas en datos en aplicaciones HPC," en *XXXIII Jornadas de Paralelismo (JP23)*, Ciudad Real, España, 2023. [En línea]. Disponible en: https://www.researchgate.net/publication/382265489_Sistema_distribuido_de_archivos_ad_hoc_para_cargas_de_trabajo_intensivas_en_datos_en_aplicaciones_HPC.

[26] Innovative Computing Laboratory, University of Tennessee, *PAPI: Performance Application Programming Interface — BSD 3-Clause License*, Accedido: 2025-05-27, 2024. [En línea]. Disponible en: https://github.com/icl-utk-edu/papi/blob/master/LICENSE.

[27] Docker, Inc., *Moby (Docker Engine) — Apache License 2.0*, Accedido: 2025-05-27, 2024. [En línea]. Disponible en: https://github.com/moby/moby/blob/master/LICENSE.

[28] ARCOS, Universidad Carlos III de Madrid, *EpiGraph v1.3 — Licencia Académica UC3M (LAUC3M)*, Fichero `LICENSE.txt` de la distribución del código fuente. Accedido: 2026-07-07, 2017. [En línea]. Disponible en: https://www.arcos.inf.uc3m.es/epigraph/.

[29] ARCOS, Universidad Carlos III de Madrid, *FlexMPI — GNU General Public License versión 3*, Fichero `LICENSE.txt` de la distribución. Accedido: 2026-07-07, 2024. [En línea]. Disponible en: https://www.arcos.inf.uc3m.es/flexmpi/.

[30] ARCOS, Universidad Carlos III de Madrid, *Expand Ad-Hoc (XPN) — GNU Lesser General Public License versión 3*, Accedido: 2026-07-07, 2024. [En línea]. Disponible en: https://github.com/xpn-arcos/xpn/blob/master/LICENSE.

[31] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos*, Diario Oficial de la Unión Europea, L 119, 4 de mayo de 2016, 2016. [En línea]. Disponible en: https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679.

[32] Jefatura del Estado, *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, BOE núm. 294, de 6 de diciembre de 2018, 2018. [En línea]. Disponible en: https://www.boe.es/eli/es/lo/2018/12/05/3/con.

[33] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial*, Diario Oficial de la Unión Europea, L 1689, 12 de julio de 2024, 2024. [En línea]. Disponible en: https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32024R1689.

[34] G. Gopinath, *A Disrupted Global Recovery*, Blog del Fondo Monetario Internacional, 25 de enero de 2022. Accedido: 2025-05-27, 2022. [En línea]. Disponible en: https://www.imf.org/en/Blogs/Articles/2022/01/25/blog-a-disrupted-global-recovery.

[35] Asamblea General de las Naciones Unidas, *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible. Resolución A/RES/70/1*, Aprobada el 25 de septiembre de 2015. Accedido: 2025-05-27, 2015. [En línea]. Disponible en: https://documents.un.org/doc/undoc/gen/n15/291/93/pdf/n1529193.pdf.

[36] International Energy Agency, *Data Centres and Data Transmission Networks*, Accedido: 2025-05-27, 2024. [En línea]. Disponible en: https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks.

[37] Glassdoor, *Sueldo: Ingeniero de Software en España*, Media de los sueldos comunicados por los usuarios. Accedido: 2026-08-20, 2026. [En línea]. Disponible en: https://www.glassdoor.es/Sueldos/espa%C3%B1a-ingeniero-de-software-sueldo-SRCH_IL.0,6_IN219_KO7,28.htm.

[38] Tesorería General de la Seguridad Social, *Bases y tipos de cotización*, Régimen General. Accedido: 2026-08-20, 2026. [En línea]. Disponible en: https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/36537.